

**MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE
REPUBLIC OF KAZAKHSTAN**

**“Kazakh-British Technical University” JSC
School of Information Technology and Engineering
Educational program 6B07103 “Automation and Control”**

ADMITTED TO DEFENCE

Dean of SITE,

_____ A. Zh. Imanbayev
“ _____ ” _____ 2026

**EXPLANATORY NOTE
TO GRADUATION PROJECT (work)**

Theme: “Development of multirobot control system for painting parts with real time monitoring”

Authors:

_____ A. Abdimalik
_____ R. Zeinulla
_____ C. Masimba
_____ A. Rakhmetkali
“ _____ ” _____ 2026 y.

Supervisor:

Associate Professor, PhD
_____ Samigulin T.I.
“ _____ ” _____ 2026 y.

Educational program:

**6B07103 – “Automation and
Control”**

Norms Compliance Monitor:

Lecturer, MSc
_____ D.D.Bekeshov
“ _____ ” _____ 2026

Almaty, 2026

**MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE
REPUBLIC OF KAZAKHSTAN
“Kazakh-British Technical University” JSC
School of Information Technology and Engineering
Educational program 6B07103 “Automation and Control”**

ADMITTED TO DEFENCE

Dean of SITE,

_____ A. Zh. Imanbayev

“ _____ ” _____ 2026

DIPLOMA PROJECT ASSIGNMENT

Students: Abdimalik Almat, Zeinulla Rassul, Masimba Collins, Rakhmetkali Ayan

Project title: “Development of multirobot control system for painting parts with real time monitoring”

Approved by the KBTU order: № _____ dated “ _____ ” _____

Submission deadline: _____

List of issues addressed in the diploma project or its brief content:

1. Description of the technological process;
2. Description of the control plant;
3. Simulation and stability tests;
4. Information about the controller type;
5. Technical equipment;
6. Implementation of safety instrumented systems;
7. Econometric part;
8. Occupational health and safety.

Consultations regarding the project with indication of respective reasons:

Section	Consultant (academic degree, title)	Timeline	Signature
Technological part	T. I. Samigulin, Associate Professor, PhD	02.09.2025 – 28.09.2025	
Mathematical model	T. I. Samigulin, Associate Professor, PhD	30.09.2025 – 02.11.2025	
Technical equipment	T. I. Samigulin, Associate Professor, PhD	04.11.2025 – 07.12.2025	
Implementation of safety instrumented systems	T. I. Samigulin, Associate Professor, PhD	09.12.2025 – 22.02.2026	
Econometric part	T. I. Samigulin, Associate Professor, PhD	24.02.2026 – 08.03.2026	
Occupational health and safety	T. I. Samigulin, Associate Professor, PhD	10.03.2026 – 29.03.2026	

DIPLOMA PROJECT WORK SCHEDULE

Sections, issues addressed	Submission deadline	Notes
Technological part	29.09.2025	
Mathematical model	03.11.2025	
Technical equipment	08.12.2025	
Implementation of safety instrumented systems	23.02.2026	
Econometric part	09.03.2026	
Occupational health and safety	30.03.2026	

Dean of SITE

_____ A. Zh. Imanbayev
(signature)

Project Supervisor

_____ T. I. Samigulin
(signature)

Authors

_____ A. Abdimalik
(signature)

_____ R. Zeinulla
(signature)

_____ C. Masimba
(signature)

_____ A. Rakhmetkali
(signature)

Date « _____ » _____ 2026

Project sheet

№	Format	Designation	Denomination	Quantity	Note
1	A4		Explanatory note	1	159 pages
2	.pdf		Electronic copy of the diploma project	1	
3	.pdf		Presentation File	1	
4	A3		Drawings for the project	13	

АНДАТПА

Бұл зерттеу жабынның сапасын, процестің қайталануын және жұмысшылардың қауіпсіздігін жақсартуға бағытталған автоматтандырылған өнеркәсіптік бояу желісі үшін нақты уақыттағы мониторингі бар таратылған көп роботты басқару жүйесін әзірлеу болып табылады. Жүйе Siemens Simatic S7-1500 бағдарламаланатын логикалық контроллері, индуктивті жақындық датчиктерімен, BLDC конвейер қозғалтқышымен және HMI басқару панелімен нақты уақыт режимінде визуализация және процесті басқару арқылы дайындамаларды тасымалдауды, бір уақытта бүріккіш бояуды, термиялық кептіруді және орнында жинауды автоматтандырады. Дифференциалдық теңдеулерді, Лаплас аймағындағы беріліс функциясын және күй кеңістігіндегі көріністі қамтитын конвейердің жетек қозғалтқышының математикалық моделі жасалды, барлық компоненттер екінші ретті динамикалық жүйелер ретінде модельденді. Контроллер параметрлері үш әдісті қолдана отырып синтезделді: Zigler–Nicols тұйық циклін баптау процедурасы, MATLAB автоматты күйге келтіру және сызықтық-квадраттық реттегіш. Тұрақтылық меншікті мәндерді талдау, Хурвиц критерийі, жиілік аймағындағы Боде және Найквист әдістері және нөлдік полюсті көрсету арқылы аналитикалық түрде тексерілді. Негізгі шешім ретінде автоматты түрде реттелетін PID реттегіші таңдалды, ол нөлдік тұрақты қатені, асып кетудің болмауын және талап етілетін спецификация шегінде орнату уақытын қамтамасыз етеді. Жүйелік Байланыс нақты уақыт режимінде Жергілікті деректерді бөлісу үшін PROFINET және MATLAB/Simulink-пен басқару интеграциясы үшін OPC UA көмегімен жүзеге асырылды.

Құбырды басқарудан басқа, екі Waveshare roarm-M2-s бояу манипуляторын және бір автоматтандырылған цикл шеңберінде Kuka kr10 r1100-2 өнеркәсіптік роботын үйлестіру үшін MATLAB негізіндегі басқару деңгейі әзірленді. Сурет салу траекториялары quintic smoothstep интерполяциясы арқылы жасалды және ортақ Wi-Fi желісі арқылы HTTP/JSON командалары арқылы RoArm манипуляторларына жіберілді. KUKA роботының орнын таңдау ерекшеліктерді алып тастауды ескере отырып, негізгі позицияларды аналитикалық параметрлеу арқылы жоспарланған және KukaVarProху TCP/IP аралық бағдарламалық жасақтамасының көмегімен нақты уақыт режимінде 83 Гц жиілікте орындалды. Siemens TIA Portal V19-да жасалған PLC бағдарламасы конвейерді индекстеуді, кептіру аймағын басқаруды және Роботты сигнал беруді қоса алғанда, өтудің әр кезеңінде қолданылатын қауіпсіздік құлыптарымен толық технологиялық дәйектілікті жүзеге асырады.

Бұл компоненттер бірге PLC негізіндегі өнеркәсіптік сенімділікті, MATLAB негізіндегі басқаруды үйлестіруді және бір өндірістік цикл шеңберінде бірнеше роботтарды синхрондау үшін нақты уақыттағы байланысты пайдаланатын бірыңғай бояу платформасын құрайды. Экономикалық талдау ұсынылған жүйенің тікелей еңбек шығындарын 45% -

дан астамға төмендететінін, шамамен 6 миллион теңге көлемінде жыл сайынғы экономикалық тиімділікті қамтамасыз ететінін және өнеркәсіптік бояуда қолдану үшін технологиялық орындылығын да, коммерциялық өміршеңдігін де көрсете отырып, 2,2 жыл ішінде бастапқы инвестицияларды қайтаратынын растайды.

ABSTRACT

This research presents the development of a distributed multirobot control system with real-time monitoring for an automated industrial painting line, aimed at improving coating quality, process repeatability, and worker safety. The system automates workpiece transportation, simultaneous spray painting, thermal drying, and pick-and-place handling using a Siemens SIMATIC S7-1500 programmable logic controller integrated with inductive proximity sensors, a BLDC conveyor motor, and an HMI operator panel for real-time visualization and process control. A mathematical model of the conveyor drive motor was developed, including differential equations, a Laplace-domain transfer function, and a state-space representation, with all components simulated as second-order dynamic systems. Controller parameters were synthesized using three methods: the Ziegler–Nichols closed-loop tuning procedure, MATLAB auto-tuning, and a Linear Quadratic Regulator approach. Stability was verified analytically through eigenvalue analysis, the Hurwitz criterion, Bode and Nyquist frequency-domain methods, and pole-zero mapping. The auto-tuned PID controller was selected as the primary solution, achieving zero steady-state error, no overshoot, and a settling time within the required specification. System communication was implemented using PROFINET for real-time field-level exchange and OPC UA for supervisory integration with MATLAB/Simulink.

Beyond conveyor control, a MATLAB-based supervisory layer was developed to coordinate two Waveshare RoArm-M2-S painting manipulators and a KUKA KR10 R1100-2 industrial robot within a single automated cycle. Painting trajectories were generated using quintic smoothstep interpolation and dispatched to the RoArm manipulators via HTTP/JSON commands over a shared Wi-Fi network. Pick-and-place motion for the KUKA robot was planned through analytical key-pose parameterization with singularity avoidance, and executed in real time at 83 Hz via the KukaVarProxy TCP/IP middleware. The PLC program, developed in Siemens TIA Portal V19, implements the full technological sequence including conveyor indexing, drying zone management, and robot handshake signals, with safety interlocks enforced at every stage transition.

Together, these components form a unified precision painting platform that leverages PLC-based industrial reliability, MATLAB-based supervisory coordination, and real-time communication to synchronize multiple robots within one production cycle. An economic analysis confirms that the proposed system reduces direct labor costs by more than 45%, achieves an annual economic effect of approximately 6 million KZT, and returns the initial investment within 2.2 years, demonstrating both technological feasibility and commercial viability for industrial painting applications.

АННОТАЦИЯ

Это исследование представляет собой разработку распределенной мультироботной системы управления с мониторингом в режиме реального времени для автоматизированной промышленной линии окраски, направленной на повышение качества покрытия, повторяемости процесса и безопасности работников. Система автоматизирует транспортировку заготовок, одновременную окраску распылением, термическую сушку и сборку на месте с помощью программируемого логического контроллера Siemens SIMATIC S7-1500, интегрированного с индуктивными датчиками приближения, двигателем конвейера BLDC и панелью управления HMI для визуализации в режиме реального времени и управления процессом. Была разработана математическая модель приводного двигателя конвейера, включающая дифференциальные уравнения, передаточную функцию в области Лапласа и представление в пространстве состояний, при этом все компоненты моделировались как динамические системы второго порядка. Параметры контроллера были синтезированы с использованием трех методов: процедуры настройки по замкнутому циклу Циглера–Николса, автоматической настройки MATLAB и линейно-квадратичного регулятора. Стабильность была проверена аналитически с помощью анализа собственных значений, критерия Гурвица, методов Боде и Найквиста в частотной области и отображения нулевого полюса. В качестве основного решения был выбран автоматически настраиваемый ПИД-регулятор, обеспечивающий нулевую установившуюся погрешность, отсутствие превышений и время установления в пределах требуемой спецификации. Системная коммуникация была реализована с использованием PROFINET для обмена данными на местах в режиме реального времени и OPC UA для диспетчерской интеграции с MATLAB/Simulink.

Помимо управления конвейером, был разработан диспетчерский уровень на базе MATLAB для координации двух окрасочных манипуляторов Waveshare RoArm-M2-S и промышленного робота KUKA KR10 R1100-2 в рамках одного автоматизированного цикла. Траектории рисования были сгенерированы с использованием интерполяции quintic smoothstep и отправлены манипуляторам RoArm с помощью команд HTTP/JSON по общей сети Wi-Fi. Выбор места для робота KUKA был спланирован с помощью аналитической параметризации ключевых позиций с учетом исключения сингулярностей и выполнялся в режиме реального времени на частоте 83 Гц с помощью промежуточного программного обеспечения KukaVarProxy TCP/IP. Программа ПЛК, разработанная в Siemens TIA Portal V19, реализует полную технологическую последовательность, включая индексацию конвейера, управление сушильной зоной и подачу сигналов роботом, с блокировками безопасности, применяемыми на каждом этапе перехода.

Вместе эти компоненты образуют единую платформу точной покраски, которая использует промышленную надежность на базе ПЛК, координацию

управления на базе MATLAB и связь в режиме реального времени для синхронизации нескольких роботов в рамках одного производственного цикла. Экономический анализ подтверждает, что предлагаемая система снижает прямые затраты на оплату труда более чем на 45%, обеспечивает ежегодный экономический эффект в размере около 6 миллионов тенге и возвращает первоначальные инвестиции в течение 2,2 лет, демонстрируя как технологическую осуществимость, так и коммерческую жизнеспособность для применения в промышленной покраске.

CONTENT

INTRODUCTION.....	13
1 LITERATURE REVIEW.....	14
2 TECHNOLOGICAL PART	16
2.1 Statement of the problem	16
2.2 Description of the technological process	16
2.3 Block diagram process	18
2.4 Description of the parameters	19
3 DESCRIPTION OF THE CHOSEN CONTROL OBJECT	21
3.1 Feedback loop diagram	22
3.2 Structural scheme of the control plant	22
3.3 Specification table	23
3.4 Conclusion.....	24
4 MATHEMATICAL MODEL OF CONTROL OBJECT	25
4.1 Transfer function of a DC motor instead of the TF of a conveyor belt	25
4.2 Differential equation	26
4.3 Transfer Function	28
4.4 State Space	30
4.5 Stability	31
4.5.1 Step Response	32
4.5.2 Impulse Response.....	33
4.5.3 Frequency Response.....	34
4.5.4 Nyquist	34
4.5.5 Hurwitz stability criterion	36
4.5.6 Pole-zero map.....	37
4.5.7 Controllability and Observability.....	38
4.6 Desired parameters.....	38
4.7 Conclusion.....	39
5 REGULATOR SYNTESIS	40
5.1 Modelling of the mathematical model and control system (PID).....	40
5.1.1 PID controller with Ziegler- Nichols Method.....	41
5.1.2 PID controller with auto-tuning	44
5.2 Linear-Quadratic Regulator	46
5.3 Comparison of methods: results and choice	48
5.3.1 Simulink Verification of the Auto-Tuned PID Controller.....	49
5.4 Conclusion.....	50
6 COMPLEX OF THE TECHICAL DEVICES	51
6.1 Information about the controller type	52
6.2 Description of controller modules.....	55
6.3 Information about data networks	59
6.4 Operator interface.....	60
6.4 Specification of sensors with basic characteristics	61
6.5 Specifications of actuators	63

6.5.1 Specification Brushless DC Motor	63
6.5.2 Specifications of manipulators	65
6.6 Economic analyzes and cost estimation	68
6.7 Conclusion.....	69
7 PROGRAM REALIZATION	70
7.1 PLC–MATLAB integration and supervisory control architecture	70
7.1.1 Software architecture overview.....	70
7.1.2 PLC-MATLAB communication and safety handshake	71
7.1.3 System workflow and execution sequence	72
7.2 MATLAB-based pick and place process using KUKA KR10 R1100-2	72
7.2.1 Network and middleware setup.....	72
7.2.2 TCP/IP Communication protocol implementation	73
7.2.3 Pick and place trajectory generation	74
7.2.4 Real time trajectory execution and monitoring.....	75
7.2.5 Real time telemetry and end-effector feedback	76
7.3. MATLAB-based painting process using RoArm M2s manipulators	77
7.3.1 Hardware and network configuration	77
7.3.2 HTTP/JSON communication protocol.....	78
7.3.3 Object oriented MATLAB control classes.....	78
7.3.4 Painting trajectory generation and execution.....	79
7.4 Industrial configuration of the controller	80
7.4.1 Integration of conveyor and fan	80
7.5 Control system development in TIA Portal	81
7.5.1 Program variables and tags	81
7.5.2 Control logic implementation	83
7.5.4 HMI visualization design	86
7.6 Motor PID programming.....	89
7.6.1 PID variables	89
7.6.2 PID logic	90
7.6.3 PID’s Human Machine Interface visualization.....	92
7.7 Real-life implementation.....	93
7.8 Conclusion.....	96
8 LABOR PROTECTION	97
8.1 Characteristics of the workplace of the automated line operator.....	97
8.2 Dangerous and harmful factors in the operation of the robotic system	97
8.3 Analysis of compliance of working conditions with regulatory requirements	99
8.4 Security measures.....	101
8.5 Electrical and fire safety.....	102
8.6 Calculation of the ventilation of the painting area.....	102
8.7 Conclusion.....	104
9 ECONOMIC SECTION.....	105
9.1 Technical and Economic Justification of the Proposed System	105
9.2 Calculation of Capital Expenditures for System Implementation	105
9.2.1 Labor Cost Calculation for the Implementation Team	105

9.2.2 Equipment Procurement and Installation Costs	106
9.2.3 Operating Cost Calculation	108
9.2.4 Electricity Cost Calculation	109
9.2.5 Total Capital Expenditure	109
9.2.6 Economic Efficiency of APCS Implementation	109
9.3 Investment Attractiveness Assessment	111
9.4 Conclusion.....	112
CONCLUSION	114
References	115
APPENDIX A	118
APPENDIX B	120
APPENDIX C	121
APPENDIX D	124
APPENDIX E.....	126
APPENDIX F.....	130
APPENDIX G	131
APPENDIX H	134
APPENDIX I.....	137
APPENDIX J.....	142
APPENDIX K	144
APPENDIX L.....	147
APPENDIX M.....	148
APPENDIX N	149
APPENDIX O	150
APPENDIX P	151
APPENDIX Q	152
APPENDIX R	153
APPENDIX S.....	154
APPENDIX T.....	155
APPENDIX U	156
APPENDIX V	157
APPENDIX W	158
APPENDIX X	159

INTRODUCTION

Robot manipulators have become an important part of modern automation. They are widely used in manufacturing and related industries because of their accuracy, repeatability, and ability to perform dangerous or repetitive tasks. Industrial robots are now used in many operations, such as assembly, material handling, welding, painting, dispensing, and inspection. Their application continues to grow in the automotive, electronics, aerospace, logistics, and other industries.

Welding is one of the best examples of how automation can improve both process quality and workplace safety. Robotic welding provides accurate trajectory control, high repeatability of technological parameters, and reduces the risk of human exposure to heat, fumes, and other dangerous working conditions. Recent studies also show that modern welding robots use data driven technologies, and smart control methods to improve weld quality, productivity, and flexibility in industrial environments.

Another important and technologically challenging application is robotic spray painting. In this paper, it is considered as an example of a high-precision industrial process. Studies show that the demand for robotic painting manipulators has increased because they offer better stability, quality, repeatability, and overall efficiency. Compared to manual painting, robot-controlled motion allows more uniform paint application and better repeatability.

In industrial spray painting, product quality and productivity strongly depend on the repeatability of technological parameters such as spray distance, spray angle, overlap ratio, and end-effector speed. In addition, recent research has focused on collision-free trajectory generation and optimal robot placement in collaborative painting systems. At the same time, spray painting still creates serious occupational safety risks. Paint materials may contain volatile organic compounds (VOCs) and aerosols, so exposure control is necessary. The main risks are connected with inhalation of paint vapors and contact with the skin. These risks should be reduced through engineering controls, organizational measures, and properly selected personal protective equipment (PPE).

However, even when protective measures are used in real working conditions, some residual risk may still remain. For this reason, robotic painting systems are attractive not only because they improve process quality and repeatability, but also because they reduce continues human exposure to hazardous environments.

1 LITERATURE REVIEW

According to the International Federation of Robotics, collaborative robots reached a market share of 10.5% of all industrial robots installed worldwide in 2023 [1]. This shows that industrial robots are becoming more important in modern manufacturing. They are widely used because they improve productivity, accuracy, and repeatability. They can also work for long periods, reduce human error, and make production more stable. A recent systematic review also shows that global robot installations have remained above 500,000 units per year since 2020, while collaborative robots can reduce assembly time by 30% and improve quality by 15% in some applications [2]. In the automotive industry, this is especially important because production lines need both high speed and stable quality [9].

One important application of industrial robotics is spray painting. Robotic painting systems can provide more uniform coating, better repeatability, and better control of process parameters than manual painting. Research shows that the design of the robot, the control method, and the teaching process all affect painting quality [4], [5]. In addition, full robotic painting systems have already been developed for industrial use, which shows that automated painting is a practical direction for modern production [6]. Research conducted in China shows that coating quality also depends on process parameters such as spray distance, movement speed, and trajectory accuracy [7]. For this reason, path planning is an important part of robotic painting, especially for parts with complex surfaces [8].

Another important issue is the use of several robots in one system. In collaborative painting cells, robots must move safely, avoid collisions, and cover the workpiece efficiently. Some studies focus on collision-free trajectory generation for robotic car painting, while others study optimal robot base placement to improve coverage and reduce motion problems [10], [11]. These works are useful for multi-robot systems, but most of them focus mainly on the painting task itself. They do not always include the full technological sequence, where painting must also be connected with conveyor movement, drying, and the final pick-and-place operation performed by an industrial manipulator.

Painting processes are also connected with health and safety risks. According to recent research, volatile organic solvents may form more than 30% of paint ingredients, and some paint production areas show high non-cancer and cancer risks for workers [12]. Another recent study examined 80 exposed workers and 80 control subjects and found significantly higher DNA damage indicators in the exposed group [13]. Because of this, robotic painting is important not only for productivity, but also for worker safety. By using robots, it is possible to reduce direct human contact with hazardous materials and make the process safer.

From the control point of view, industrial robotic systems are usually built with more than one level of control. PLC-based control is widely used in industry because it is reliable, stable, and easy to connect with sensors, actuators, and safety logic [14], [19], [20]. However, PLC-only control is not always enough when the system needs more accurate trajectory calculations or more flexible supervision. For this reason, MATLAB/Simulink is often used for robot control, inverse kinematics, robot

programming, and testing of advanced control methods [15][16][17]. Studies also show that MATLAB and PLC can be connected through OPC UA, which allows one system to keep industrial reliability and, at the same time, use higher-level computational tools [18]. This idea is very suitable for a multi-robot painting line, where PLC can manage sequence control and safety, while MATLAB can support accurate calculations, monitoring, and trajectory-related tasks for both painting and final pick-and-place handling.

Based on the reviewed literature, it can be seen that many researchers study robotic painting, coating quality, robot trajectory planning, and industrial control. However, there are fewer works that combine all these parts in one distributed system. In particular, there is still a need for solutions where several robots work in one coordinated process, where painting is followed by drying and then by a pick-and-place operation using an industrial manipulator. This research gap supports the development of a distributed multi-robot control system with real-time monitoring for painting parts.

2 TECHNOLOGICAL PART

This chapter presents the project “Distributed Multirobot Control System with Real-Time Monitoring.” It describes how the technological process is organized, formulates the engineering problem addressed by the proposed system, explains the sequence of operations through a block diagram, and summarizes the general parameters maintained throughout the process. In this way, the chapter provides the technological basis for the design and implementation of the control system. Figure 1 shows the general workflow of the automated multi-robot painting line. It presents the main technological stages, starting from painting and ending with the pick-and-place operation.

2.1 Statement of the problem

Robotic manipulators can be controlled in industry in different ways. One of the most common methods is PLC-based control, where the PLC sends commands to the manipulator and defines the sequence of operations. This type of control is reliable, widely used in real industrial production, and can be easily integrated with sensors, actuators, and safety logic. It helps maintain stable and repeatable operation. However, when several manipulators must work together in one system, PLC-only control is not always the best solution for complex calculations and flexible coordination.

Another method is MATLAB/Simulink-based control, where the software is used to test control algorithms and then send commands to the manipulator. This approach is effective for trajectory planning, model-based control, data processing, and monitoring. Because of this, it is useful for experiments, analysis, and fast system development. However, for continuous industrial operation, MATLAB-only control may be less practical than PLC-based control, since it usually depends on a PC environment and is not mainly designed for industrial reliability and safety.

For this reason, this project uses a combined PLC and MATLAB approach to coordinate several robots in one automated technological system. The PLC is responsible for sequence control, interlocks, and device-level coordination, while MATLAB is used for higher-level supervision, trajectory-related calculations, monitoring, and tasks that require higher precision. Communication between the PLC and MATLAB is carried out through a standard industrial interface. As a result, the system keeps industrial stability while also using the computational advantages of MATLAB.

2.2 Description of the technological process

As an example, this technological process is based on a real industrial process used in automobile manufacturing at Hyundai Trans Kazakhstan. The general idea of the system was taken from a real industrial production line. However, within this diploma project, the process is implemented on a laboratory scale. This is because

the available hardware at the university is limited to laboratory equipment. At the same time, the software part was developed according to industrial standards, so the system can be easily adapted to a real industrial process in the future. For this reason, the project refers to a large industrial process, but its practical implementation is carried out at the laboratory level.

The process consists of three main stages: the painting process, the drying process, and the pick-and-place process.

Figure 2.1 shows the general workflow of the automated multi-robot painting line. It presents the main technological stages, starting from painting and ending with the pick-and-place operation.

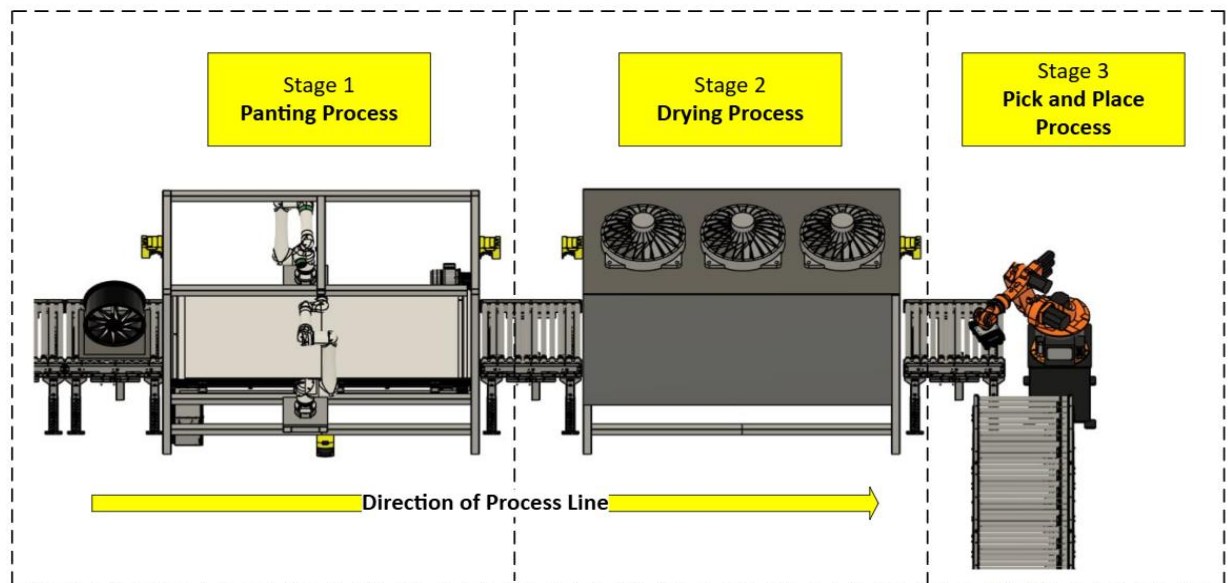


Figure 2.1 – industrial technological process

The manufacturing process begins when a workpiece is placed on the conveyor line. The conveyor then moves the workpiece to the painting station. When the product reaches the predefined position, the conveyor stops to ensure accurate placement. This positioning is controlled with the help of proximity sensors installed at each technological stage.

At the painting station, two RoArm-M2-S manipulators apply the coating to the workpiece at the same time. Their movements are coordinated by a distributed control layer, which also controls the spray on and spray off modes.

After the painting operation is finished, the conveyor transfers the workpiece to the drying area. Drying is carried out by a set of industrial fans that provide airflow from several directions. The drying cycle follows a predefined technological recipe to stabilize the applied coating.

At the final stage, the KUKA manipulator removes the processed workpiece from the conveyor and places it onto the output platform for finished products. This step completes the technological cycle and prepares the system for the next workpiece.

2.3 Block diagram process

To better understand how all subsystems in the painting line interact with each other, the technological sequence is presented in the form of a block diagram. Unlike the descriptive explanation given in the previous section, the block diagram provides a compact and visual representation of the process. It shows the main stages, their logical order, and the decision points that affect the further flow of operations.

The block diagram in Figure 2.2 presents the full working cycle of the multi-robot painting line from start to finish. It shows the main actions, control decisions, and the transfer of the workpiece between the conveyor, the painting station, the drying unit, and the pick-and-place station. The diagram is read from top to bottom and from left to right. Each diamond represents a condition, each rectangle shows an action, and the circles indicate transition points to the next stage.

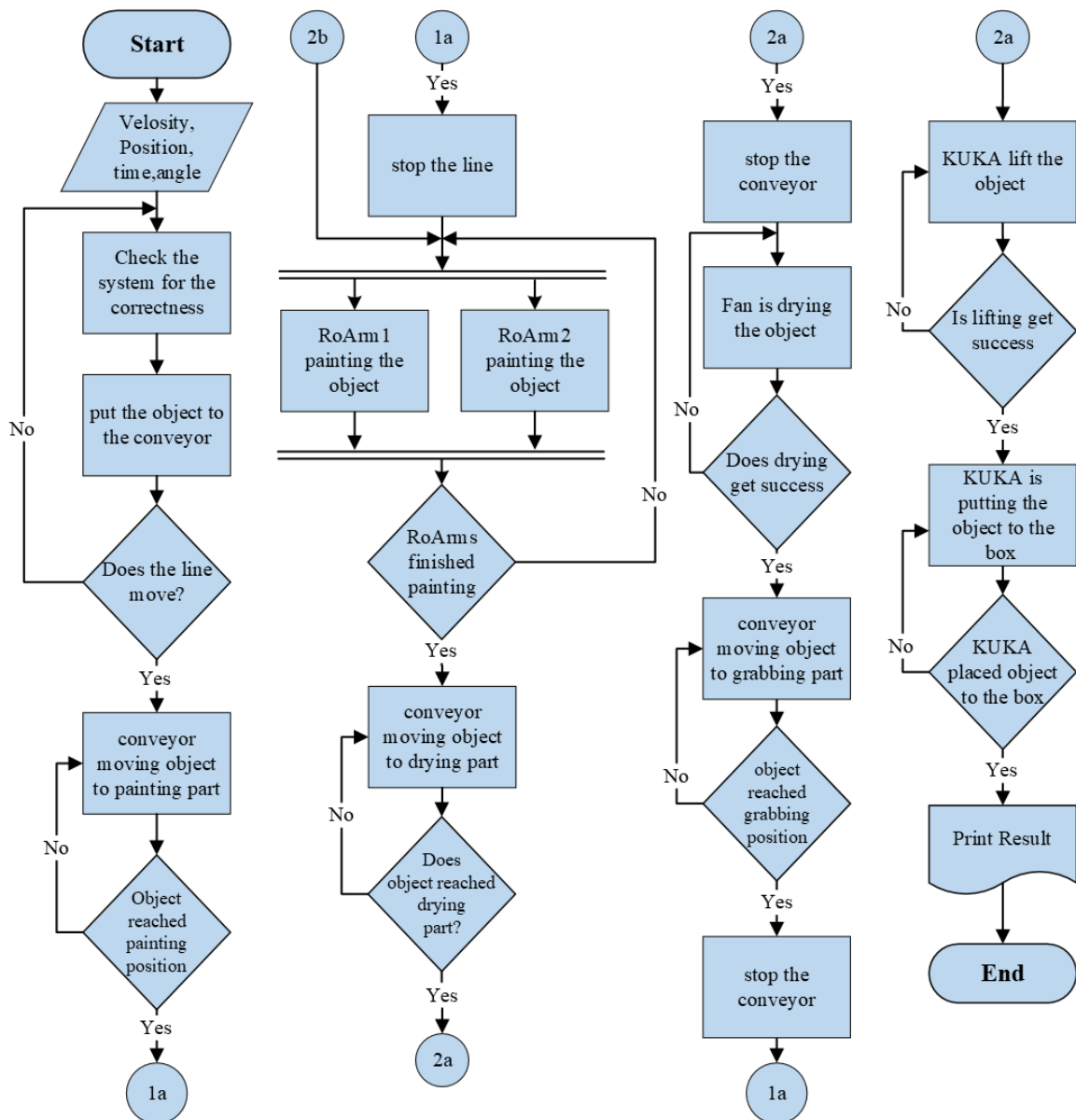


Figure 2.2 - A visualized form of the block diagram

This type of representation makes it easier to follow the process logic, identify possible bottlenecks, and improve the coordination between the conveyor, manipulators, and control devices. By using standard graphical symbols, the flowchart clearly explains the system logic, which is useful both for engineers who configure the control software and for operators who monitor the production line.

The process begins with parameter setup, including velocity, position, time, and angle, followed by a short system check. After that, an object is placed on the conveyor. If the conveyor is running, it moves the object to the painting position and then stops. At this stage, two RoArm M2S manipulators paint the object simultaneously. When both manipulators complete their task, the conveyor moves the object to the drying zone.

In the drying zone, the conveyor stops again. The coated surface is dried by an industrial fan arch. If the drying process is completed successfully, the conveyor moves the object to the grabbing position.

At the final stage, the conveyor stops once more. The KUKA robot picks up the object and places it on the platform for finished parts. If the robot fails to lift the object, the system repeats the action or sends an error signal. After successful placement, the result is recorded and the cycle ends. During all stages, the distributed control system synchronizes start and stop commands, reads sensor signals, and maintains a stable takt time.

2.4 Description of the parameters

In any automated industrial system, especially in a multi-robot painting process, it is important to define and monitor the main process parameters. This helps ensure stable operation, safety, and product quality. These parameters may include different aspects of the system, such as mechanical motion, process states, and control signals. Table 2.1 below presents the main operational parameters used in the system, their possible values, and their role in the overall process. Maintaining these parameters within the required limits is necessary for the correct operation of the painting line

Table 2.1 - Operational parameters

№	Name of parameter	Range	Technical recommendations	Description	I/O
1	2	3	4	5	6
1	Painting Process	True / False	Program Logic Controller parameter	State of process	DO
2	Drying Process	True / False	Program Logic Controller parameter	State of process	DO

Continuation of table 2.1

1	2	3	4	5	6
3	Pick and Place Process	True / False	Program Logic Controller parameter	State of process	DO
4	Conveyor belt state	True / False	Program Logic Controller parameter	State of conveyor belt	DO
5	Inductive sensor 1	True / False	Program Logic Controller parameter	State of conveyor belt sensor	DI
6	Inductive sensor 2	True / False	Program Logic Controller parameter	State of conveyor belt sensor	DI
7	Inductive sensor 3	True / False	Program Logic Controller parameter	State of conveyor belt sensor	DI
8	Inductive sensor 4	True / False	Program Logic Controller parameter	State of conveyor belt sensor	DI

Real-time monitoring and control of these values allow the system to respond quickly to changes in the process and possible deviations. As part of the distributed control system, these parameters are continuously checked to support accurate painting, reduce defects, and ensure the safe operation of all robotic subsystems.

2.5 Conclusion

In this chapter, the technological process was described with detailed information about the general logic of the system, the main operational parameters, and the overall structure of the multirobot painting line. The main stages of the process were presented, including painting, drying, and pick-and-place operations. Each stage was explained according to its role in the full technological cycle.

The chapter also described the interaction between the conveyor, sensors, robotic manipulators, and control devices. Special attention was given to the sequence of operations, communication protocols, and connection methods between the main parts of the system. This makes it possible to understand how the system performs coordinated actions during the automated process.

Overall, this chapter provides the technological basis for the further development of the distributed control system. The described process logic and parameters can be used for the implementation of PLC-based control, MATLAB/Simulink supervision, and real-time monitoring of the robotic line.

3 DESCRIPTION OF THE CHOSEN CONTROL OBJECT

The main stage of installing a multirobot painting is painting the incoming object, as well as moving the object. That is, the main control objects are robotic manipulators for painting, a manipulator for lifting and lowering an object to another location, and a conveyor belt. But all robot manipulators are a non-linear system, just like a conveyor belt. Therefore, the main control object will be the conveyor belt motor. Figure 3.1 shows the control object.



Figure 3.1 - 57BLF01 BLDC motor

Manipulators are nonlinear because their movement and dynamics are determined by nonlinear relationships between joint positions, speeds, and forces. In robot kinematics, the angles of joints are transformed into the position of the final control using trigonometric functions which are inherently nonlinear. In the dynamics of robots, the equations of motion include the product of variables that depend on the configuration of inertial matrices and centrifugal effects, as well as gravitational coefficients that depend on the angles of articulation. As a result, the behavior of the system changes depending on its configuration and speed, which means that the principle of superposition is not fulfilled, and the response is not proportional to the input data - this is why the robot arm is inherently a nonlinear system.

The conveyor belt system is non-linear because the ratio between input and output is not proportional and varies depending on external conditions. In practice, friction is often non-linear, so small costs may not cause movement until the separation threshold is exceeded, and then the movement changes dramatically. The belt can also behave as a flexible element with variable tension and possible slippage, so the effective dynamics depends on the load, belt tension and contact conditions. For more information, see section 4.1.

A BLDC motor is an electric motor that operates without mechanical brushes and commutators found in traditional DC motors. In traditional DC motors, brushes and commutators are essential for directing current to the motor winding. This method involves physical contact of brushes and the commutator, which leads to friction and wear and tear. However, BLDC motors use electronic controllers to manage the direction of current in the stator's winding. It allows for more precise control, high efficiency and long lifespan of these motors.

3.1 Feedback loop diagram

The controlled output quantities feature the control object. Such values at a particular point, define the present position of the control object. Further, the object contains control inputs to which control actions are done in an attempt to intentionally modify its state. Figure 3.2 gives a graphical display of the feedback control system. The BLDC motor of the conveyor belt is the primary control object in this project. The speed of the conveyor belt would require regulation to make sure that they work in unison with the robotic manipulators in the painting process. The actual speed is measured by an encoder and transmitted to the controller. The controller, on its part, compares the measured value against the desired one and produces the control action which is fed to the motor driver to control the motor speed.

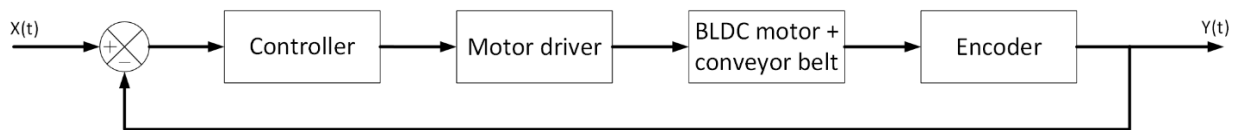


Figure 3.2 - Feedback loop diagram

$X(t)$ is the reference input signal that defines the desired conveyor belt speed. $Y(t)$ is the output signal that describes the actual state of the system, in this case the real conveyor belt speed measured by the encoder. $U(t)$ is the control action generated by the controller and applied to the motor driver. $Z(t)$ represents external disturbances affecting the system, such as changes in the weight of the transported object, friction variations, and supply voltage fluctuations. Due to the feedback loop, the system can compensate for these disturbances and maintain stable conveyor belt motion.

3.2 Structural scheme of the control plant

The control object is characterized by output controlled values. These values of the values at a given time determine the current state of the control object. In addition, the object has control inputs, to which control actions are applied to purposefully change the state of the object. Figure 3.3 shows a structural scheme of control plant. The main control object is the DC motor of the conveyor belt.

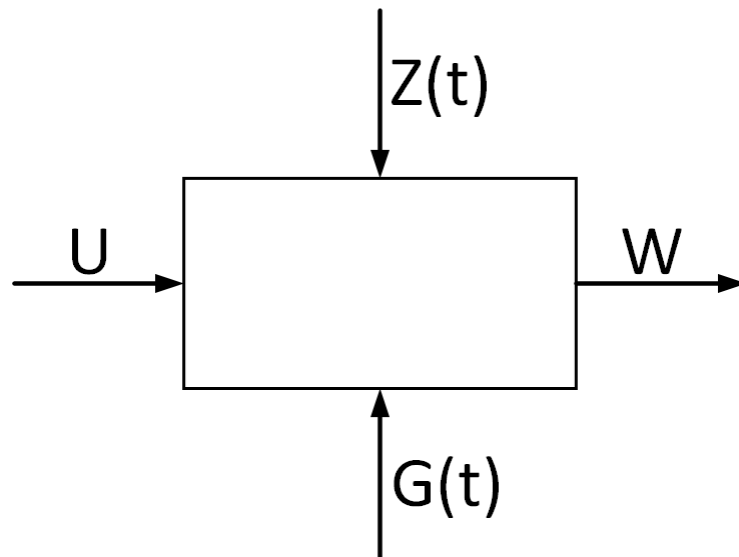


Figure 3.3 - Structural scheme of control plant

In the considered system, the conveyor belt is driven by a BLDC motor. The main controlled output is the conveyor belt speed, which must remain constant to ensure synchronized operation with robotic manipulators. External disturbances, such as changes in the weight of the transported object, friction variations, and supply voltage fluctuations, affect the system dynamics.

The control objective is to adjust the motor speed automatically so that, when a heavier object is placed on the conveyor belt, the motor rotates faster and generates higher torque. This compensation ensures that the conveyor belt continues to move at a constant linear speed regardless of load changes, maintaining stable object positioning and uniform painting quality.

3.3 Specification table

To define mathematical model in the future and clarify the controllable quantities of the system, it is necessary to obtain what is input or output value, whether they produce discrete or analog signal. Table 3.1 contains details about the parameters.

Table 3.1 – Description of the parameters shown in black box

Parameter	Unit	Description	Input or Output signal	Discrete or Analog
U	V	Applied armature voltage	I	A
W	Rad/s	Rotor angular speed	Q	A
Z(t)	-	Disturbances	I	A
G(t)	-	Regulator	I	A

The system has analog control inputs, primarily the motor voltage or speed reference, and analog outputs, such as motor speed and conveyor belt velocity. Disturbances represent uncontrolled external effects acting on the system. Proper control of conveyor speed ensures smooth transportation, reliable synchronization with robotic manipulators, and consistent painting quality across the production line.

3.4 Conclusion

This section described the chosen control object of the multirobot painting system and justified the selection of the conveyor belt BLDC motor as the primary control plant. The nonlinear nature of robotic manipulators and the conveyor belt was explained, highlighting the influence of kinematic, dynamic, frictional, and load-dependent effects on system behavior. The structural scheme and specification of inputs, outputs, and disturbances clarified how the conveyor speed is controlled under varying operating conditions. Maintaining a constant conveyor belt speed despite load changes is essential for synchronized operation with robotic manipulators and for ensuring uniform painting quality, making the BLDC motor-driven conveyor a suitable and effective control object for further modeling and controller design.

4 MATHEMATICAL MODEL OF CONTROL OBJECT

A mathematical model of a control object is a simplified representation of a real physical system using mathematical equations. Such models are necessary for analyzing system behavior and are also needed to check whether the system is linear or nonlinear, stable or unstable. Developing an accurate mathematical model is a key step in modern control systems. The control object of project is a conveyor belt. However, the transfer function of the conveyor belt is nonlinear, which is discussed in more detail in section 4.1. For this reason, the motor that drives conveyor belt will be selected as the basis for the transfer functions. The motor is a 57BLF series brushless DC motor. All motor data will be selected from the 57BFL01 series [17]. The conveyor belt system in the automated painting line operates under variable load conditions that make its dynamics nonlinear.

4.1 Transfer function of a DC motor instead of the TF of a conveyor belt

The conveyor belt system in an automated painting line operates under variable load conditions, which makes its dynamics nonlinear. Changes in the transported mass, uneven weight distribution, and varying friction change the torque requirement, belt tension, and system inertia. All this leads to changes in parameters over time, i.e., to nonlinear effects such as stretching and slippage of the belt, which complicates the derivation of a single linear transfer function for the entire conveyor. Most of the reasons why this system is nonlinear are as follows:

The first reason is uneven load distribution. As mentioned above, if the load is distributed unevenly, this creates a system with distributed parameters that is difficult to model using a single linear transfer function. If the load is concentrated on one side or in certain areas of the conveyor, the tension and friction forces will vary spatially along the belt.

The second reason is belt slippage. When using variable conveyor belts, uneven forces or excessive loads can cause the belt to slip on the rollers or drive pulley, resulting in nonlinear discontinuity in the dynamics of the system.

The third reason is elastic effects. This term refers to the fact that the properties of the belt material, such as elasticity or viscoelasticity, in most cases lead to its stretching or deformation under load. This results in nonlinear relationships between force and displacement.

The last reason is the interaction of forces. The interaction between tension, friction, and gravity forces is interrelated. That is, increasing tension due to a large load can reduce slippage. But this can increase friction losses, creating a nonlinear feedback loop.

Load variations and force distribution across a conveyor belt introduce significant nonlinearities that complicate the derivation of a linear transfer function for the entire system. Variations in load mass, distribution, and frictional forces, coupled with belt tension and elastic effects, create a complex, time-varying system. By focusing on the motor's transfer function, engineers can isolate a component with linear dynamics, treating the conveyor's nonlinearities as external disturbances. This

approach simplifies modeling, enables linear control design, and ensures robust performance in the presence of load variations. For practical applications, combining a linear motor model with appropriate compensation for load effects offers a balanced solution for designing efficient and controllable conveyor belt systems.

4.2 Differential equation

The BLDC motor employed in this study is modeled with one electrical state (armature current) and one mechanical state (rotor angular speed). The electrical dynamics link the applied voltage, winding inductance and resistance, and the back-EMF that is proportional to the speed. The mechanical dynamics link the electromagnetic torque (proportional to current) with the rotor inertia and viscous friction. This compact electromechanical model is standard for BLDC drives with trapezoidal back-EMF operated under averaged conditions [18]. This can also be understood by looking at Figure 4.1.

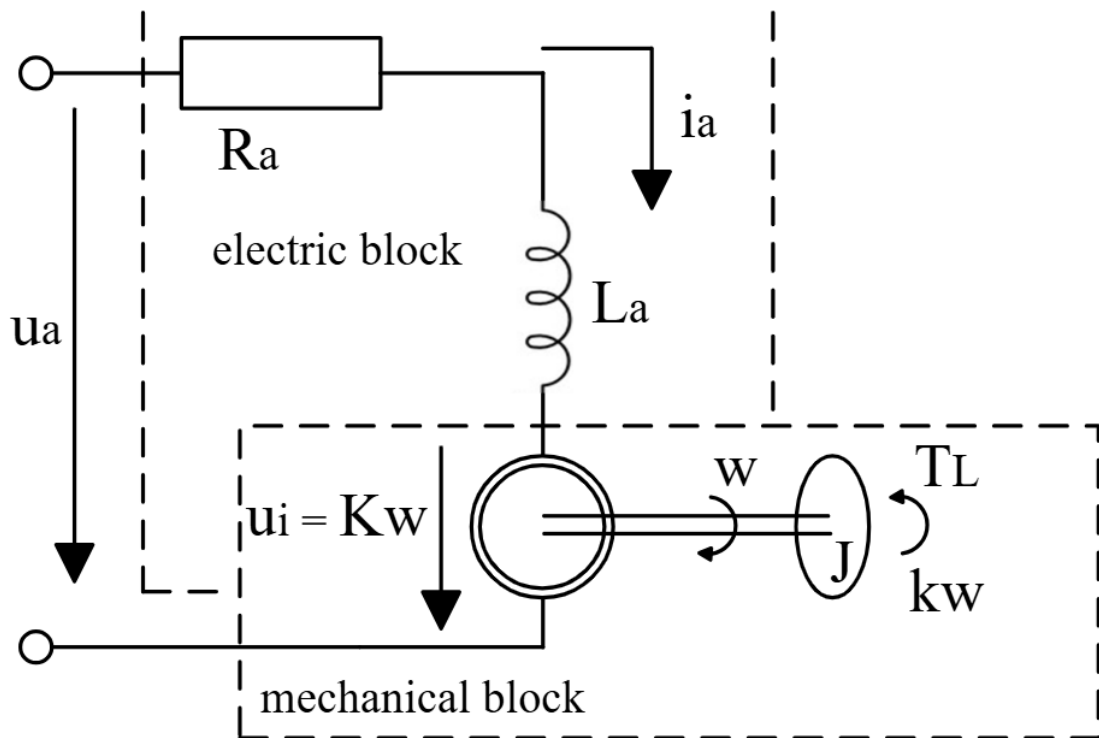


Figure 4.1 – Circuit model of the DC motor

Each of the parameters in this model is necessary to accurately describe the physical behavior and interaction of the electrical and mechanical parts of the motor.

R_a (winding resistance) is necessary to account for energy losses. This affects the current in the circuit and, in turn, the mechanical power that the motor can develop. L_a (winding inductance) affects the dynamic behavior of the system and the motor's response time to changes in input voltage. K (coupling coefficient between the electrical and mechanical parts) is necessary to model how the electric current in the motor windings affects mechanical rotation. J (rotor moment of inertia) is

important for modeling how the mass and mass distribution of the rotor affect its ability to accelerate and decelerate. T_l (mechanical load or load torque) is an external force acting on the rotor that resists its rotation. k (friction coefficient) determines the friction force between two contacting surfaces. In this work, K_e is set equal to K_t in SI units to preserve electromechanical consistency.

The meaning of all symbols and variables is shown in Table 4.1. The values of these variables from future calculations are also provided here.

Table 4.1 - Variables of the BLDC motor

	Variable designation	Variable description	Variable value	Unit of measurement
1	2	3	4	5
1	J	Moment of Inertia	1.2e-5	kg · m ²
2	b	Viscous friction coefficient	1e-4	H · m · s/rad
3	w(t)	Angular velocity	-	rad/s
4	R	Armature Resistance	1.3275	Ω
5	L_a	Armature Inductance	0.00075	H
6	$i_a(t)$	Armature Current	-	A
7	u(t)	Applied Armature Voltage	-	V
8	K_e	Back-EMF Constant	0.065	V · s/rad
9	K_t	Torque Constant	0.065	N · m / A
10	T_L	Load (disturbance) torque	-	N · m

Kirchhoff's voltage law is used to find the differential equations of the electrical part. His voltage law states that for a closed circuit of series connections, the algebraic sum of all voltages around any closed circuit in the circuit is zero. Looking at Figure 4.1, the following equation for electrical part can be written as:

$$-L_a \cdot \dot{i}_a - R_a \cdot i_a - K_e \cdot \omega + u_a = 0, \quad (4.1)$$

For the mechanical part of the system, an analogue of Newton's second law for rotational motion is used, which describes the dynamics of a rotating body. Looking at Figure 4.1, the following equation for mechanical part can be written as:

$$J \cdot \dot{\omega} = K_t \cdot i_a - k \cdot \omega - T_L \quad (4.2)$$

T_L is a disturbance signal, so it will not be taken into account when determining the transfer function which mean $T_L = 0$. Also, k - friction coefficient, will be written as b so as not to confuse it with K . Thus, this differential equation can be written as:

$$u_a(t) = L_a \cdot \frac{di_a(t)}{dt} + R_a \cdot i_a(t) + K_e \cdot \omega(t), \quad (4.3)$$

$$K_t \cdot i_a(t) = J \cdot \frac{d\omega(t)}{dt} + b \cdot \omega(t) \quad (4.4)$$

In BLDC motor models, K usually stands for the electromechanical coupling constants. There are actually two closely related constants: the back-EMF constant K_e and the torque constant K_t . The back-EMF constant K_e converts angular speed into induced voltage, while the torque constant K_t converts armature current into electromagnetic torque. In consistent SI units energy/power balance implies they are numerically equal which mean

$$K_e = K_t, \quad (4.5)$$

even though their units differ: K_e is in V·s/rad, and K_t is in N·m/A.

4.3 Transfer Function

In various control applications, the motor is examined through a transfer function that maps the Laplace-domain output to the input under zero initial conditions [19],[20]. To find the transfer function, it is first necessary to convert the differential equations of the system into Laplace form. This is how differential equation of the electrical part in equation (4.3) is look like into Laplace form:

$$V(s) = L_a s \cdot I(s) + R_a \cdot I(s) + K_e \cdot \Omega(s), \quad (4.6.1)$$

$$V(s) = (L_a s + R_a) \cdot I(s) + K_e \cdot \Omega(s), \quad (4.6.2)$$

where,

$$V(s) = \mathcal{L}[u_a(t)]; \quad s \cdot I(s) = \mathcal{L}\left[\frac{di_a(t)}{dt}\right]; \quad (4.6.3)$$

$$I(s) = \mathcal{L}[i_a(t)]; \quad \Omega(s) = \mathcal{L}[\omega(t)] \quad (4.6.4)$$

The Laplace form for mechanical part of the differential equation in (4.4) look like:

$$K_t \cdot I(s) = Js \cdot \Omega(s) + b \cdot \Omega(s), \quad (4.7.1)$$

$$K_t \cdot I(s) = (Js + b) \cdot \Omega(s) \quad (4.7.2)$$

For the solving $I(s)$ from mechanical differential equation in (4.7.2) into Laplace form is look like:

$$I(s) = \frac{(Js + b)}{K_t} \cdot \Omega(s) \quad (4.8)$$

After subtracting the current from equation (4.8) into electrical equation (4.6.2) it will be look like

$$V(s) = (L_a s + R_a) \cdot \frac{(Js + b)}{K_t} \cdot \Omega(s) + K_e \cdot \Omega(s), \quad (4.9.1)$$

$$V(s) = \left(\frac{(Js + b)(L_a s + R_a) + K_e K_t}{K_t} \right) \cdot \Omega(s), \quad (4.9.2)$$

Selecting the applied armature voltage u as the input and the rotor angular speed ω as the output, the transfer function for Brushless DC Motor is

$$G(s) = \frac{K_t}{(R + L \cdot s)(J \cdot s + b) + K_e \cdot K_t}, \quad (4.10.1)$$

$$G(s) = \frac{K_t}{JLs^2 + (b \cdot L + J \cdot R)s + (bR + K_e \cdot K_t)} \quad (4.10.2)$$

The variable values will be taken from the Table 4.1. However, due to the lack of information about b (coefficient of viscous friction), it is necessary to test this motor to determine it. Since this is currently impossible, the “ b ” data will be assumed to be equal to $1 \cdot 10^{-4}$ or 0. For b is $1 \cdot 10^{-4}$, the transfer function from equation in (4.10.2) takes the following form:

$$G(s) = \frac{0.065}{9 \cdot 10^{-9}s^2 + 1.6005 \cdot 10^{-5}s + 0.00435775}, \quad (4.11.1)$$

When b is 0 transfer function of motor takes the following form:

$$G(s) = \frac{0.065}{9 \cdot 10^{-9}s^2 + 1.5930 \cdot 10^{-5}s + 0.004225}, \quad (4.11.2)$$

In this study the viscous-friction coefficient b (denoted as k in the cited source) is neglected in the analytical transfer-function derivation in order to obtain a compact second-order model dominated by the electrical and inertial dynamics.[7] For 57BLF01 data the contribution of the b -terms is demonstrably small near the rated point: in the denominator

$$JLs^2 + (bL + JR)s + (bR + K_e \cdot K_t), \quad (4.12.1)$$

the mixed term $b \cdot L$ constitutes only $\approx 0.47\%$ of

$$bL + JR, \quad (4.12.2)$$

since $b \cdot L = 7.5 \times 10^{-8}$ and $J \cdot R = 1.593 \times 10^{-5}$, while the constant part $b \cdot R$ is $\approx 3.05\%$ of $\ddot{\theta}$

$$b \cdot R + K_e \cdot K_t, \quad (4.12.3)$$

because $b \cdot R = 1.3275 \times 10^{-4}$ versus $K_e \cdot K_t = 4.225 \times 10^{-3}$. Thus, setting $b = 0$ changes the static gain only from 14.92 to 15.38 (rad/s)/V, leaving stability and dominant time constants virtually unchanged. This approximation is standard for averaged BLDC/DC models at medium and high speeds, where viscous losses are small compared to copper and electromechanical coupling losses. However, for low-speed operation, during transient startup processes, or when bearing resistance/air resistance losses are not negligible, b should be retained. Even if $b = 0$ in a verified source, it will be set to $1e-4$ in this system, since in real tests the coefficient of viscous friction is never equal to 0[7]. Therefore, the first transfer function will be used in the following simulations.

4.4 State Space

The state space is necessary because it provides a compact and universal model of the system dynamics in the form of

$$\dot{x} = Ax + Bu, \quad y = Cx + Du, \quad (4.13)$$

where x is state vector, u is input voltage, and y is output angular velocity. In this representation, it is easy to take into account multiple inputs and outputs. For motor, this is natural two states

$$x = [i_a \omega]^T, \quad (4.14.1)$$

where

$$i_a = x_1, \quad \omega = x_2. \quad (4.14.2)$$

The following system of linear ordinary differential equations describes the state space model:

For Electrical equation

$$L_a \cdot \dot{i}_a = -R_a \cdot i_a - K_e \cdot \omega + u_a, \quad (4.15.1)$$

$$\dot{x}_1 = -\frac{R_a}{L_a} \cdot x_1 - \frac{K_e}{L_a} \cdot x_2 + \frac{1}{L_a} u_a \quad (4.15.2)$$

For Mechanical equation

$$J \cdot \dot{\omega} = K_t \cdot i_a - b \cdot \omega - T_L, \quad (4.16.1)$$

$$\dot{x}_2 = \frac{K_t}{J} \cdot x_1 - \frac{b}{J} \cdot x_2 - \frac{T_L}{J}. \quad (4.16.2)$$

The unknown load torque T_L can be considered as a disturbance signal, so the input of the system is the armature voltage u_a and the output is the angular velocity ω . State space matrix is

$$\dot{x} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -\frac{R_a}{L_a} & -\frac{K_e}{L_a} \\ \frac{K_t}{J} & -\frac{b}{J} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} \frac{1}{L_a} \\ 0 \end{bmatrix} u_a, \quad (4.17.1)$$

$$y = [0 \quad 1] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + 0u_a, \quad (4.17.2)$$

Elements of the general description of the linear state space modeling:

$$A = \begin{bmatrix} -\frac{R_a}{L_a} & -\frac{K_e}{L_a} \\ \frac{K_t}{J} & -\frac{b}{J} \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{L_a} \\ 0 \end{bmatrix}, \quad C = [0 \quad 1], \quad D = 0, \quad (4.18)$$

State space modeling allows analyzing controllability, observability, and internal stability by analyzing eigenvalues. Meanwhile, TF simplifies the analysis of system behavior in the frequency domain.

4.5 Stability

The stability of the linearized 57BLF01 motor was evaluated via the eigenvalues of the state matrix A constructed at the rated operating point. In accordance with linear systems theory, internal (asymptotic) stability holds if and only if all eigenvalues of A have strictly negative real parts. For the present parameter set (positive R , L , J , b and nonzero electromechanical constants K_e , K_t) the open-loop poles are approximately -1442.7 and -335.6, which are both located in the left half-plane. Therefore, the open-loop motor plant is stable. Physically, the model contains two dominant modes: a fast electrical mode associated with the coil's small time constant, and a slower mechanical mode governed by rotor inertia. Both modes are damped. Electrical damping arises from copper losses and back-EMF: as speed

increases, the induced back-EMF opposes the applied voltage, returning mechanical energy to the electrical domain where it is dissipated by the stator resistance. When viscous friction is included, it provides additional mechanical damping; when it is neglected ($b=0$), the back-EMF–resistance pathway still supplies sufficient dissipation to prevent sustained oscillations or growth [20]. Consequently, the linearized system is stable both mathematically (left-half-plane eigenvalues) and physically (all stored energy has a path to dissipation, with no internal sources that inject net energy).

4.5.1 Step Response

The step response was evaluated for the closed-loop unity-feedback system with a reference value of 16.80955 rad/s. The response is very fast, with a rise time of 0.000487 s and a settling time of 0.004022 s. However, unlike the previously considered open-loop case, the closed-loop response is no longer monotonic. It exhibits an overshoot of 34.57%, which indicates an underdamped transient behavior. The steady-state value reaches 15.7534 rad/s, resulting in a steady-state error of 1.0561 rad/s or 6.28%. This behavior indicates that the closed-loop system is stable but underdamped. The feedback loop greatly accelerates the response, yet it also introduces oscillatory transient behavior, which is consistent with the complex-conjugate closed-loop poles. Therefore, the present closed-loop system is much faster than the open-loop plant, but it is less smooth and exhibits a noticeable overshoot. The corresponding transient behavior is shown in Figure 4.1 below.

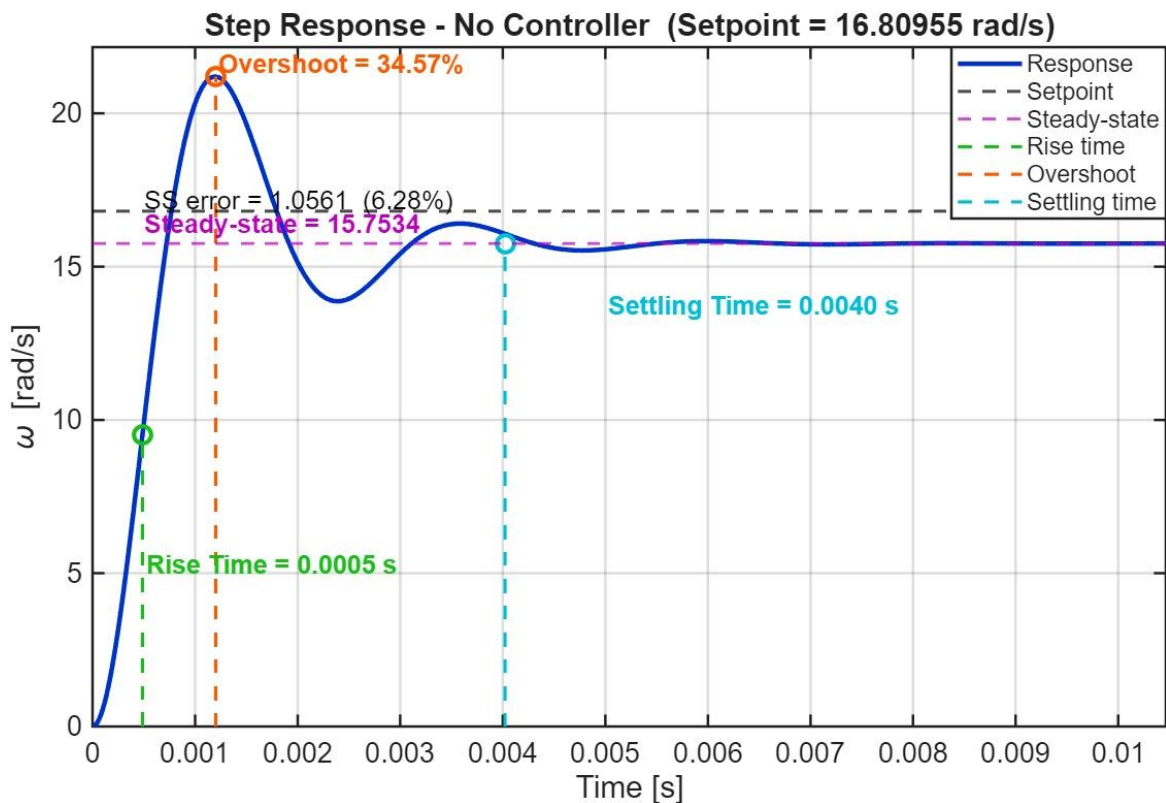


Figure 4.1 – Step response of the closed-loop system

Overall, the model demonstrates a rapid but underdamped transition with a finite steady-state error, accurately reflecting the current closed-loop speed-control dynamics of the motor, as shown in the figure below. The values of dynamic performance is more simply showed in Table 4.2.

4.5.2 Impulse Response

The impulse response was also evaluated for the closed-loop system. In contrast to the earlier open-loop interpretation, the waveform is no longer monotonic. Instead, the response exhibits a sharp initial peak followed by decaying oscillations. This confirms that the dominant closed-loop modes are oscillatory, although the oscillations gradually vanish with time.

From a physical point of view, the impulse input excites the closed-loop dynamics immediately, revealing both the fast response and the limited damping of the loop. The absence of sustained oscillations confirms stability, while the ringing indicates that the system is underdamped rather than overdamped. The corresponding waveform is shown in Figure 4.2 below.

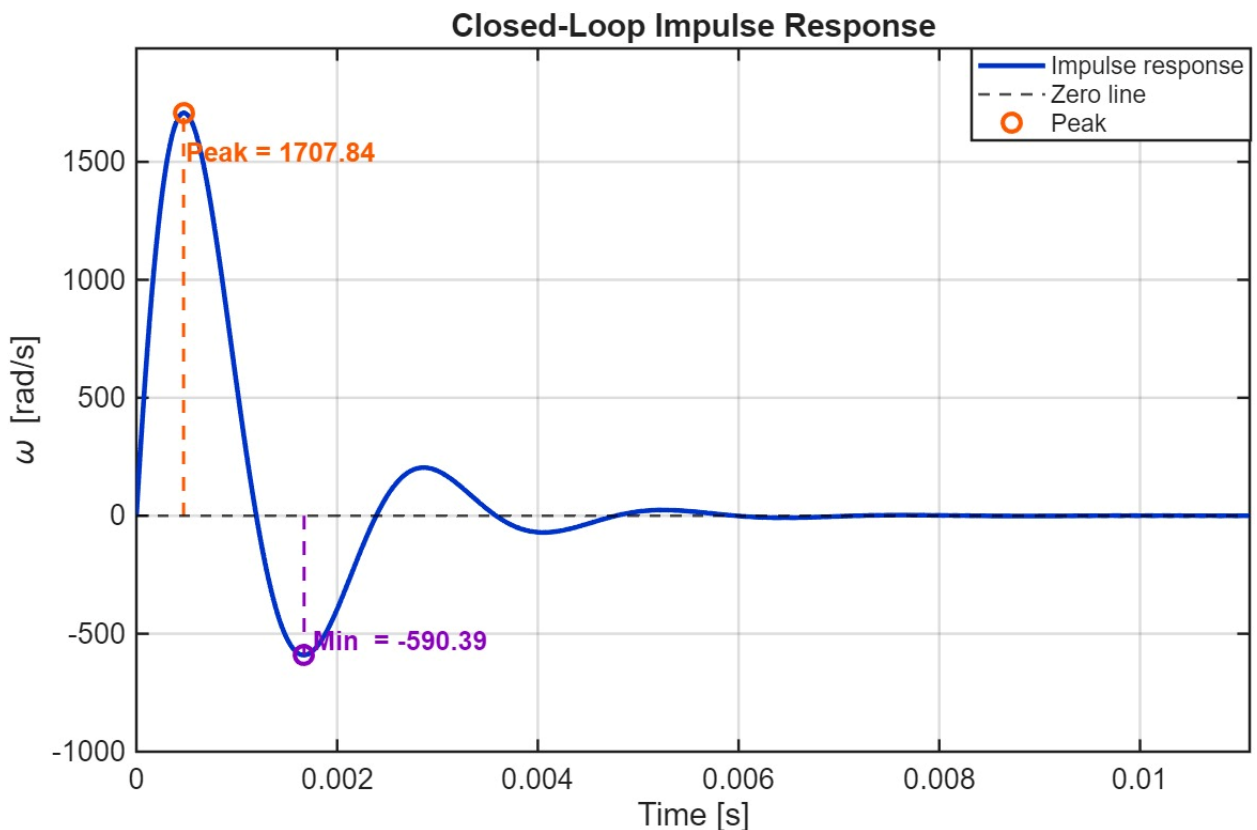


Figure 4.2 – Impulse response of the closed-loop system

Thus, the impulse response supports the same conclusion as the step response: the closed-loop system is stable and very fast, but its transient dynamics remain oscillatory.

4.5.3 Frequency Response

The Bode analysis was carried out for the open-loop transfer function. The gain margin is infinite because no finite phase-crossover point is reached for the ideal second-order motor model. The phase margin is 37.79 degrees at the gain-crossover frequency of 2488.36 rad/s.

This means that the open-loop plant is stable, but the available robustness is not especially large. The finite phase margin is consistent with the oscillatory transient response observed later in the closed-loop system. The corresponding frequency response is shown in Figure 4.3 below.

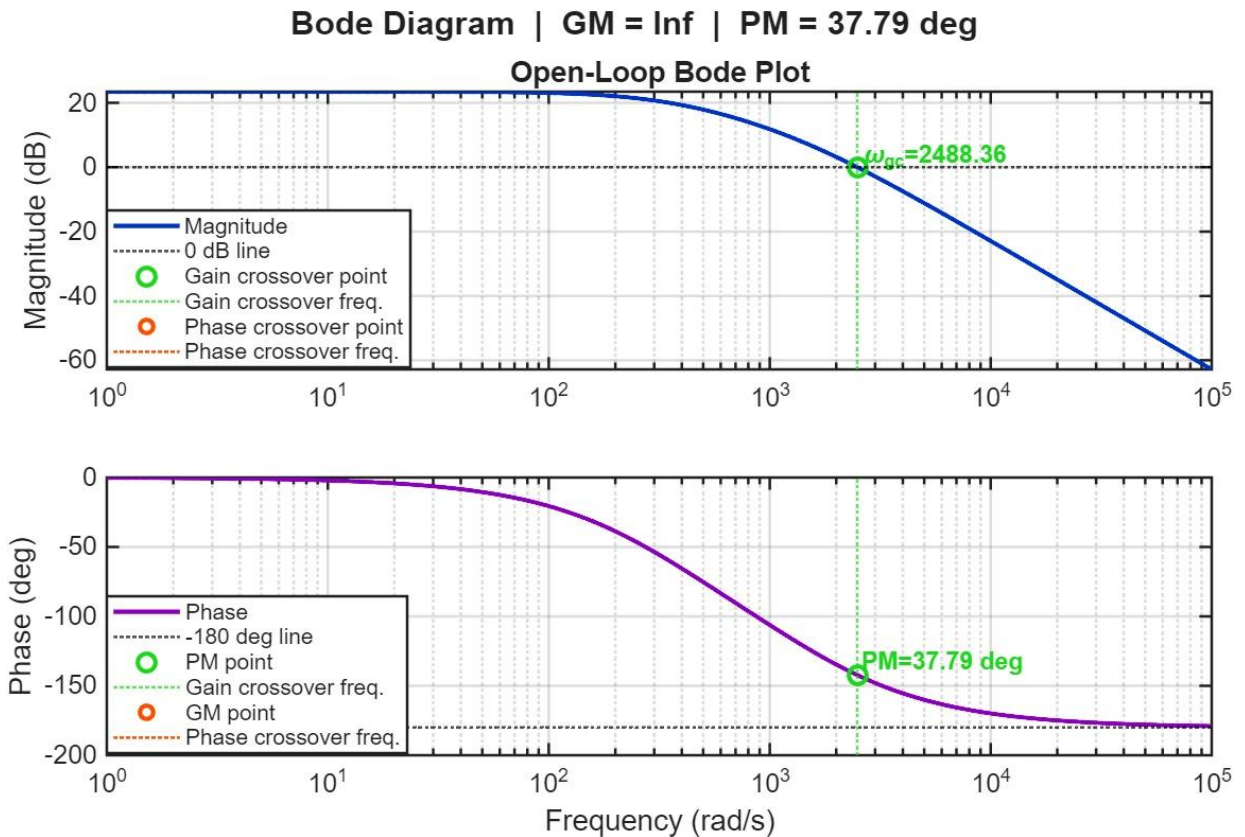


Figure 4.3 – Bode diagram of the open-loop system

In a real drive system, sampling, PWM update, computation time, and actuator delay introduce additional phase lag. Therefore, a small transport delay is later introduced during the Ziegler-Nichols tuning procedure to obtain a finite ultimate gain and ultimate period.

4.5.4 Nyquist

The Nyquist plot is a graph of the frequency response of an open-loop system constructed in a complex plane. It is used to evaluate the stability of a closed-loop system based on the position of this curve relative to the critical point at -1. The main types of Nyquist plots are stable, unstable, and marginally stable.

The stability of the Nyquist plot is determined by the direction of the arrow and its proximity to the critical point [20]. Circling, the critical point clockwise, indicates instability, while the absence of circling indicates stability. The closer the curve approaches the critical point, the lower the stability limits and the higher the risk that a small increase or phase change will destabilize the circuit.

According to the figure below (Figure 4.5.1), the Nyquist plot is stable. Because the locus does not pass through the critical point and does not form a clockwise encirclement around it.

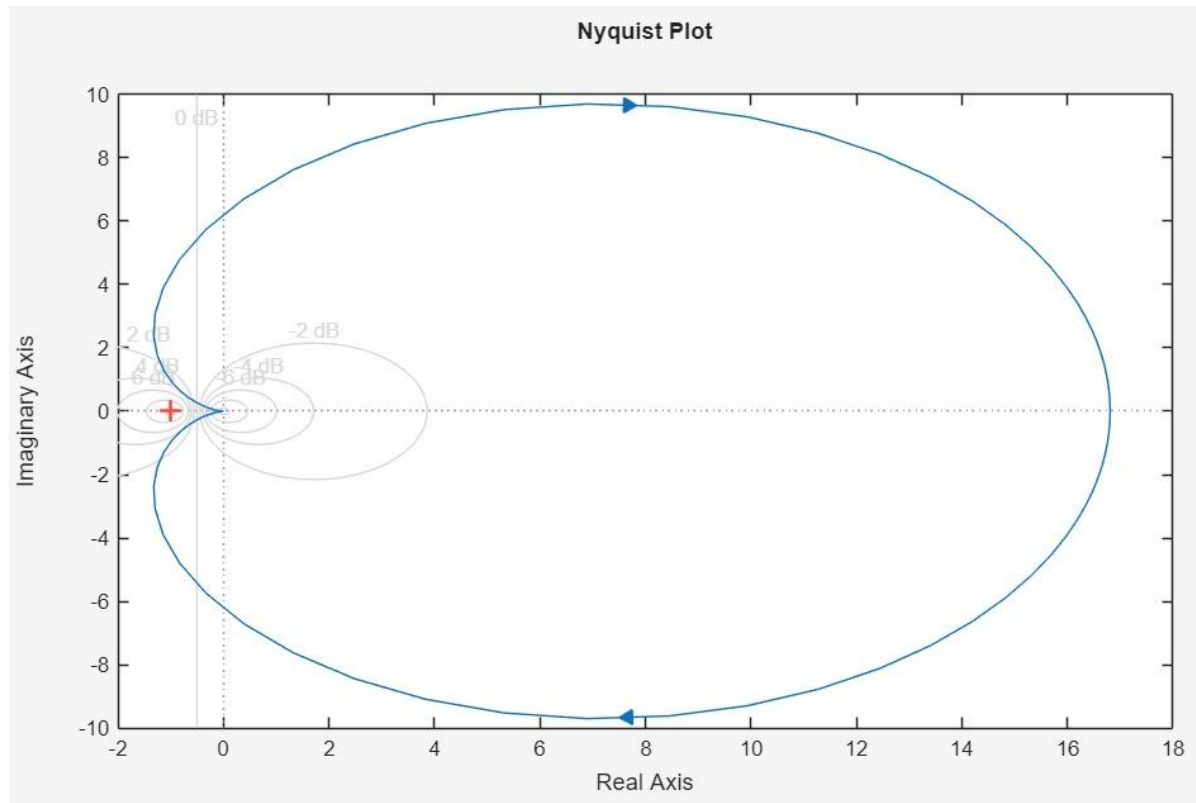


Figure 4.5.1 – Nyquist plot of the open-loop system

The enlarged view of the Nyquist diagram is shown in Figure 4.5.2 below in order to illustrate the local behavior of the curve near the critical region more clearly. In this zoomed representation, the phase margin, gain margin, and stability margin are indicated directly on the plot. Such a close-up view makes it easier to see how far the frequency-response locus remains from the critical point $(-1,0)$, and therefore provides a more detailed interpretation of the stability reserve of the system. In particular, the phase margin is shown by the angle between the negative real axis and the radius drawn to the phase-margin point, while the stability margin is represented by the shortest distance from the Nyquist curve to the critical point. The gain-margin direction is also indicated, showing how the system would approach instability if the loop gain were increased. Thus, Figure 4.5.2 complements the general Nyquist diagram by presenting the main robustness indicators in a clearer and more visually interpretable form.

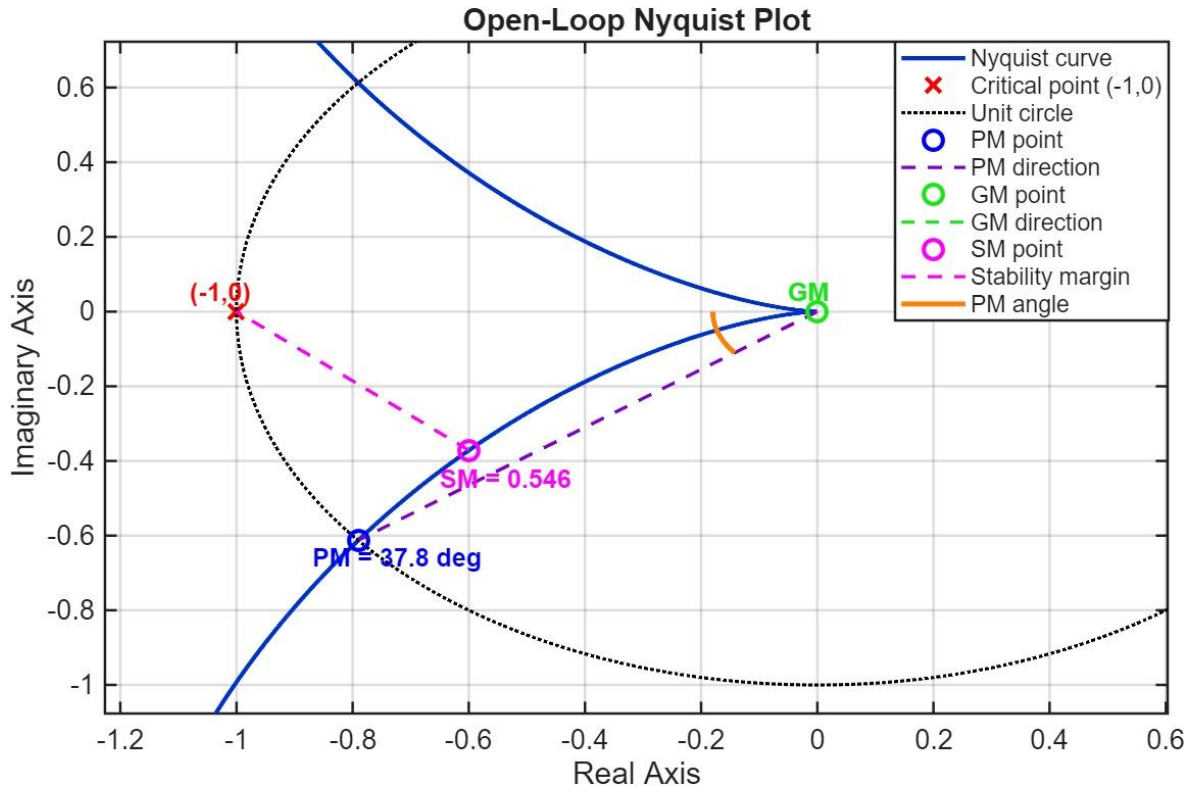


Figure 4.5.2 – Nyquist plot with phase margin, gain margin, and stability margin

This indicates that the system has no unstable poles in the right half of the complex plane. In a Nyquist plot, stability is determined by the number of encirclements of the critical point $(-1, 0)$ in the complex plane, and since there are no encirclements in this case, the system remains stable.

4.5.5 Hurwitz stability criterion

The stability of the 57BLF01 installation was tested using the Hurwitz criterion, which establishes internal stability without explicitly finding eigenvalues. The idea is to confirm that a characteristic polynomial has all its roots in the left half-plane by checking the signs of its coefficients and the positivity of the principal minors of the Hurwitz matrix [20].

The denominator of this model is the second order. The Routh-Hurwitz stability criterion states that for a system having a characteristic equation

$$P(s) = s^2 \cdot a_0 + s \cdot a_1 + a_2 = 0, \quad (4.19)$$

to be asymptotically stable, all the principal minors of the matrix

$$H = \begin{bmatrix} a_1 & 0 \\ a_0 & a_2 \end{bmatrix}, \quad (4.20)$$

must be positive, nonzero. So minors of the second order model is

$$\Delta_1 = a_1 > 0, \quad \Delta_2 = a_1 \cdot a_2 - a_0 \cdot 0 > 0. \quad (4.21.1)$$

The Hurwitz criterion requires that all two minors should be positive. These conditions are fulfilled for the 57BLF01 parameters, and the calculation shows

$$P(s) = s^2 + 1778s + 10^6 = 0 \quad 4.21.2$$

All coefficients are positive, so the transfer function is stable. This result is consistent with the zero-pole map, the eigenvalue analysis, and the observed monotonic step and impulse responses.

4.5.6 Pole-zero map

Pole-zero map of the 57BLF01 voltage to speed plant. In this representation, poles correspond to the system's internal modes, while zeros shape input-output interactions and can complicate control [20]. This view is useful for judging stability and for spotting zero locations that might degrade phase or gain characteristics during controller design.

The map in the Figure 4.6 shows two real poles in the left half-plane and no finite zeros, i.e., a strictly proper, minimum-phase plant. The closed-loop system has a pair of complex-conjugate poles located at approximately $-889.2 \pm 2629.8i$. Since the real parts of both poles are negative, the system is stable. However, the imaginary parts explain the oscillatory and underdamped transient response observed in the step and impulse plots.

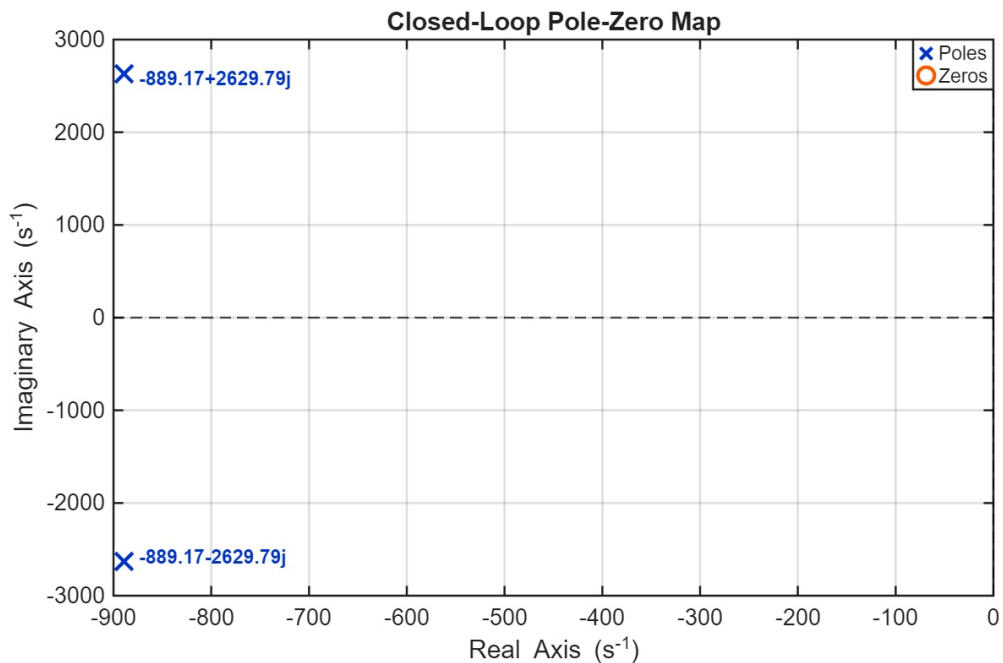


Figure 4.6 – Pole-zero map

The absence of troublesome zeros means that the system does not exhibit nonminimum-phase behavior or right-half-plane zeros that would further restrict the achievable bandwidth. Therefore, the main limitation in the present closed-loop

response is not caused by adverse zero placement, but rather by the location of the complex poles, which provide stability together with a moderate oscillatory character.

4.5.7 Controllability and Observability

The controllability and observability of the linearized 57BLF01 model were verified at the rated operating point. In line with linear systems theory, a model is fully controllable if every internal state can be influenced by the external input, and fully observable if the internal states can be reconstructed from the measured output over a finite time interval. Using the state-space form established earlier (with armature voltage as input and angular speed as output), it was computed the Kalman controllability and observability matrices and evaluated their ranks. For the selected two-state model, both ranks are equal to 2, which matches the dimension of the state vector. Therefore, the system is fully controllable and fully observable[20].

The controllability matrix is

$$\mathcal{C} = [B \ AB \ A^2B \ \dots \ A^{n-1}B] \in \mathbb{R}^{n \times nm}, \quad (3.22.1)$$

$$\text{rank}(\mathcal{C}) = n. \quad (3.22.2)$$

The Observability matrix is

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \end{bmatrix} \in \mathbb{R}^{np \times n} \quad (3.23.1)$$

$$\text{rank}(\mathcal{O}) = n \quad (3.23.2)$$

Physically, the input voltage directly drives the armature current and, through the nonzero torque constant, generates electromagnetic torque that affects rotor speed, so both electrical and mechanical states are reachable. Conversely, measuring speed provides sufficient information together with the known dynamics linking speed and current to reconstruct the unmeasured electrical state, so no internal mode remains hidden. These conclusions hold whether viscous friction is neglected or included, because electrical dissipation and electromechanical coupling still provide the necessary pathways for actuation and state reconstruction.

4.6 Desired parameters

For the conveyor belt, the 57BLF01 drive must deliver smooth, steady motion with minimal product disturbance and robust tolerance to delay and load changes. Table 4.2 below provides ranges for desired parameters. Accordingly, the targets emphasize zero overshoot, fast but non-aggressive settling, and comfortable stability margins. The ranges below are consistent with the identified plant, the measured open-loop dynamics, and standard motion-drive practice.

Table 4.2 – Desired parameters of the step response

N	The desired specifications	Range of parameters
1	2	3
1	Rise time	$< 0.0005s$
2	Settling time	$< 0.005s$
3	Setpoint	16.80955 rad/s
4	Steady State	15.7534 rad/s
5	Steady State error	1.0561 rad/s or 6.28%
6	Overshoot	34.57%
7	Phase margin	37.79°
8	Gain margin	inf

In this table, rise time shows how quickly speed moves from its initial value to the target band near the final value. Transient time describes how long the main part of the response lasts before only a small tail remains. Settling time marks the moment the speed enters a tight tolerance around the final value such as ± 2 percent and stays there. Overshoot is how much the speed exceeds the final value during the transient, and for a conveyor the target is zero to avoid slip or product disturbance.

Phase margin tells how much extra phase lag the loop can tolerate at unity gain before instability, so larger values mean stronger robustness to delay and modeling error. Gain margin shows how much the loop gain can increase before instability. Together these margins ensure quick response while keeping motion smooth and stable for conveyor transport.

To sum up this table describes the desired parameters of this plane and specifies the target dynamic and robustness indices for the conveyor application.

4.7 Conclusion

In this chapter, the mathematical model of the control object was developed and analyzed. Since the conveyor belt has nonlinear behavior due to variable load, uneven weight distribution, friction, belt slippage, and elastic effects, the BLDC motor was selected as the main object for transfer function development. The motor model was described using differential equations, Laplace transformation, transfer function, and state-space representation. This made it possible to study the dynamic behavior of the system in a simplified but technically correct form.

The stability of the model was verified using several methods, including eigenvalue analysis, step response, impulse response, Bode diagram, Nyquist plot, Hurwitz criterion, pole-zero map, controllability, and observability analysis. The results showed that the motor model is stable, controllable, and observable. Therefore, the developed mathematical model can be used as a reliable basis for further controller design and for improving the performance of the conveyor system in the distributed multirobot painting line.

5 REGULATOR SYNTESIS

This chapter provides a clear and practical overview of designing a controller for a BLDC-driven conveyor system used in a multi-robot painting line. It also explains why feedback control is necessary for stable and repeatable operation, and how multiple controllers will be designed, tuned, and compared in a single MATLAB/Simulink test bench according to the system architecture described earlier.

Classic single-loop controllers such as P, PI, PD, PID, as well as the state-space method (LQR) will be considered. For PID/PI and others, two practical tuning approaches widely used in engineering practice will be applied: the auto-tuning procedure and the Ziegler-Nichols method. Each controller is presented in simple terms: how it works, where it is typically used, and what advantages or limitations it may have for installation.

All controllers are tested on the same installation model using identical scenarios to ensure an objective comparison. How quickly and smoothly the system follows the setpoint and how confidently it maintains the required speed when minor disturbances occur in the process will also be evaluated. The comparison is based on standard quality criteria such as response speed, settling behavior, tendency to overshoot, steady-state accuracy, control effort, and overall line stability. The results are summarized in a summary diagram and tables for the three main methods, after which the most suitable controller for the painting line will be selected and briefly justified for implementation in a distributed control system.

5.1 Modelling of the mathematical model and control system (PID)

A PID controller converts the speed error into a drive command via three coordinated actions. Let y represent the measured motor speed, r represent the desired speed and e represent the control error, which is defined as $e = r - y$ [21].

The textbook law is

$$u(t) = K \left[e(t) + \frac{1}{T_i} \int_0^t e(\tau) d\tau + T_d \frac{de(t)}{dt} \right] \quad (5.1)$$

Here, K is the proportional gain is T_i the integral time T_d is the derivative time [21]. The proportional part provides an immediate correction based on the current error. The integral part accumulates past errors and removes steady-state offsets. The derivative part reacts to the rate of change of the error, adding predictive damping to shorten settling time and reduce overshoot when set to a moderate level.

Real controllers extend this basic law to ensure reliable operation in noisy and constrained environments. The derivative channel is filtered to limit high-frequency noise gain, which is often implemented as a first-order roll-off on the derivative term. The proportional path uses setpoint weighting, enabling the controller to react strongly to disturbances while avoiding aggressive jumps in response to command

steps. The integrator is protected by anti-windup tracking, ensuring that, when the actuator saturates, the integrator does not drift, allowing for quick and smooth recovery. In digital form, the algorithm is discretized with a chosen sampling period and implemented in an incremental update that supports clean saturation handling and bumpless transfer. Thanks to these practical features, a well-tuned PID delivers a fast rise, minimal or zero steady-state error, and stable behavior under load and transport delay for the conveyor motor. In practical implementation, derivative filtering and anti-windup protection are normally required to avoid noise amplification and integrator saturation.

5.1.1 PID controller with Ziegler- Nichols Method

The Ziegler–Nichols closed-loop method provides rapid initial PID settings based on a straightforward experiment involving the real loop or a high-fidelity simulation. Set the controller to proportional only by setting T_i to infinity and T_d to zero. Then, raise the proportional gain until sustained, constant-amplitude oscillations appear after a small step. Record the ultimate gain K_u and the oscillation period P_u , then read the initial P, PI or PID parameters from the classic ZN table [21]. This produces a lively quarter decay response. Keep the practical features used throughout this project after that. Apply a small first-order filter to the derivative measurement, apply set-point weighting to the proportional path and protect the integrator with anti-windup to ensure clean recovery after saturation [21].

In formula (4.10), the transfer function is second order, which causes the gain and phase margin coefficients to tend to infinity, making the loop unrealistically stable, which is why a 1ms delay was intentionally included. The object was represented as a first-order delay with transport delay

$$G(s) = Ae^{-sL} / (Ts + 1) \cdot \quad (5.2)$$

K_u and P_u in a closed loop still need to be found, but they immediately become unstable when converted to controller parameters. Reduce K by about ten to twenty percent, lengthen T_i slightly to calm the integral, and keep T_d moderate and filtered. If the step shows command overshoot, reduce the proportional setpoint weight b to maintain fast noise suppression while keeping the delay-dominated loop stable and smooth for the conveyor motor [22], [23].

From the ultimate test in the Figure 5.1,

$$K_u = 0.342642, \quad (5.3)$$

was obtained. The relay plot shows clean sustained oscillations with about sixteen cycles over 0.1 s giving

$$P_u \approx 0.005021 \text{ s}, \quad (5.4)$$

by visual count, which matches

$$P_u = \frac{2\pi}{\omega_{gm}}, \quad (5.5)$$

computed from the margin of the delayed model. These (K_u, P_u) values feed the ZN table, after which the planned detuning and the same practical features will be applied, see the figure with sustained oscillations.

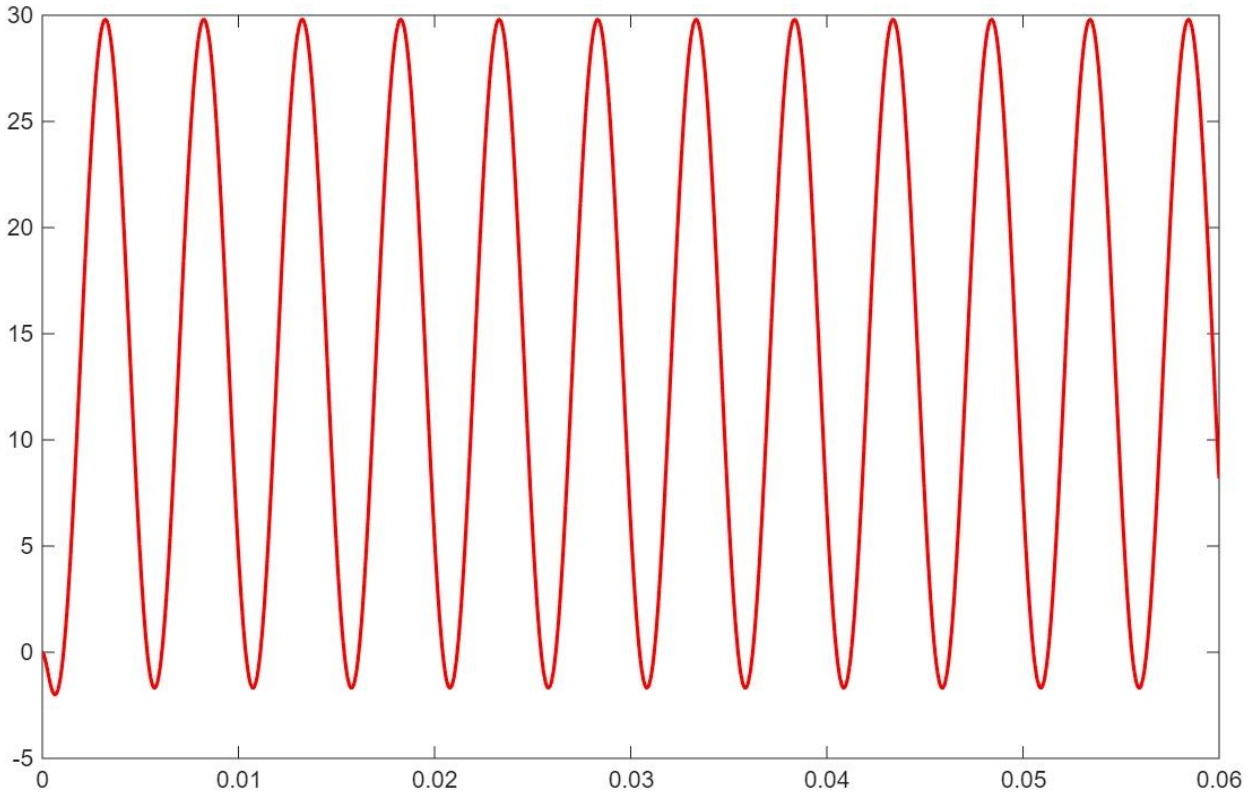


Figure 5.1 – Ultimate oscillation for Ziegler–Nichols method

Using the classical Ziegler-Nichols formulas, the following controller gains were obtained:

$$\text{PID: } K_p = 0.205585, K_i = 81.896753, K_d = 1.290201 \times 10^{-4} \quad (4.6.1)$$

$$\text{PI: } K_p = 0.154189, K_i = 36.853539 \quad (4.6.2)$$

$$\text{P: } K_p = 0.171321; \quad \text{PD: } K_p = 0.274114, K_d = 1.720268 \times 10^{-4} \quad (4.6.3)$$

Figure 4.2 shows the step responses of the Ziegler–Nichols controllers, compared with open-loop and unity-feedback responses without a controller at the setpoint of 16.8096 rad/s. Traces with integral action (PI and PID) converge to the setpoint line because the K_i/s . The S term removes steady-state error, with PI settling smoothly and PID rising slightly faster, reaching a small peak that matches the

quarter-decay target. P and PD respond quickly, but both finish below the setpoint, revealing a constant bias since there is no integral term [22], [23].

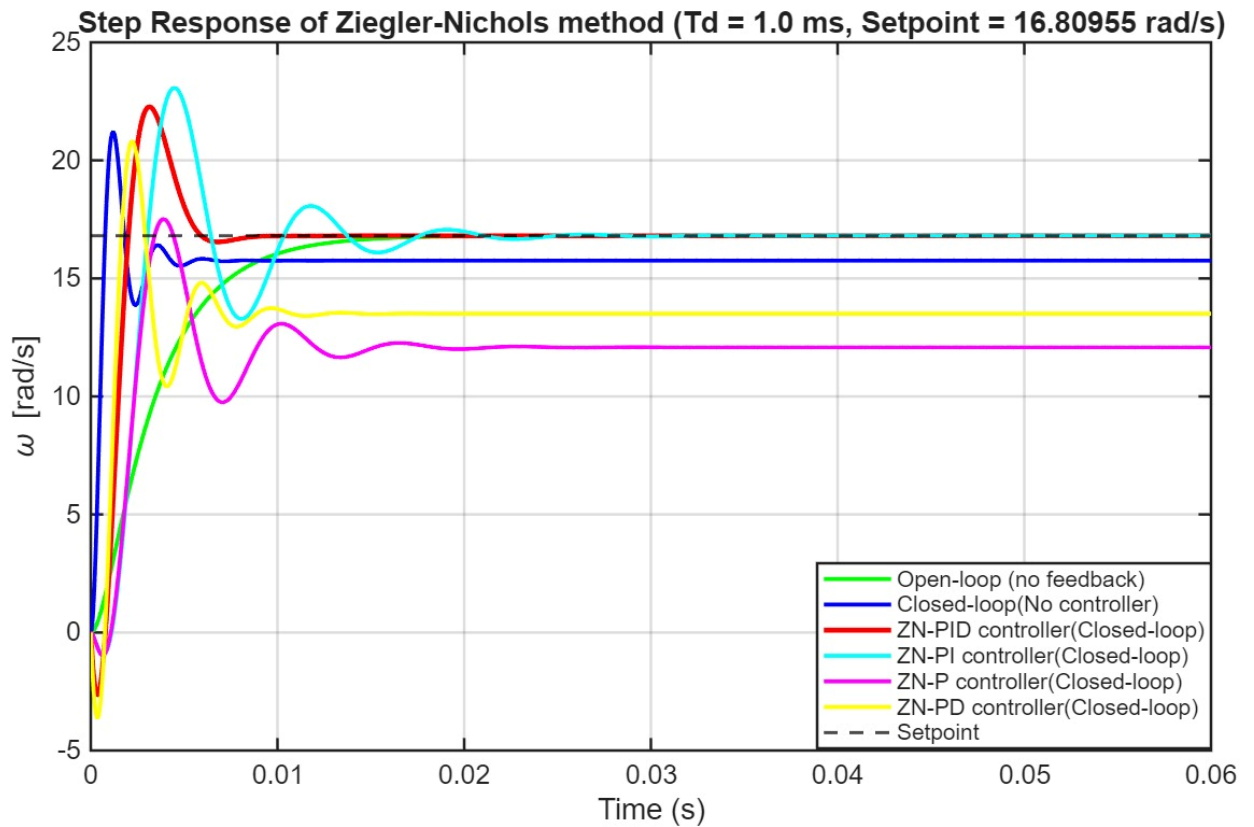


Figure 4.2 – PID controller with Ziegler Nicholas method

The table below confirms this with numbers. The simulation results show that the PI and PID controllers eliminate steady-state error because they contain an integral term. However, the Ziegler-Nichols settings are aggressive and produce large overshoot. For example, the ZN-PID controller has 32.47% overshoot, while the ZN-PI controller has 37.26% overshoot. Therefore, Ziegler-Nichols is useful as a preliminary tuning method, but the gains should be detuned before practical implementation.

Table 4.1 – PID controller with Ziegler Nicholas Method

№	Configuration	Rise time (s)	Settling time (s)	Overshoot (%)	Final value (rad/s)	SSE (rad/s)
1	2	3	4	5	6	7
1	Without regulator	0.0005	0.0040	34.57	15.7534	1.0561
2	PID regulator	0.0009	0.0056	32.47	16.8096	0.0000
3	PI regulator	0.0014	0.0168	37.26	16.8096	0.0000
4	P regulator	0.0011	0.0144	44.96	12.0817	4.7279
5	PD regulator	0.0005	0.0085	54.00	13.5062	3.3033

Overall this table shows us that PID delivers the cleanest tracking for this loop with less overshoot zero steady state error and a little bit long settling time.

5.1.2 PID controller with auto-tuning

A PID controller blends proportional, integral, and derivative actions to turn the speed error into a smooth command for the drive. K_p shapes the immediate response, K_i removes steady state offset by accumulating past error, and K_d adds predictive damping by reacting to the rate of change of the error. With the usual practical structure used in this project, derivative acts on the measurement and is filtered, the proportional path uses set point weighting, and the integrator has anti windup so recovery after saturation is clean and repeatable. Autotune provides the initial gains with a short data driven experiment near the operating point. The tool excites the loop with a safe signal, fits a simple low order model, and then computes K_p , K_i , and K_d to meet targets for rise time, settling, overshoot, and robustness while respecting actuator effort. Formula from the model is

$$C(s)=Kp+Kis+Kds \quad (5.7)$$

where K_p is the proportional gain coefficient, K_i is the integral gain coefficient, and K_d is the derivative gain coefficient. For the auto-tuned PID controller, the obtained gains were

$$K_p = 0.126990, \quad K_i = 31.540119, \quad K_d = 7.428132 \times 10^{-5} \quad (5.8)$$

This Figure 5.3 illustrates these concepts in the context of linearised motor model.

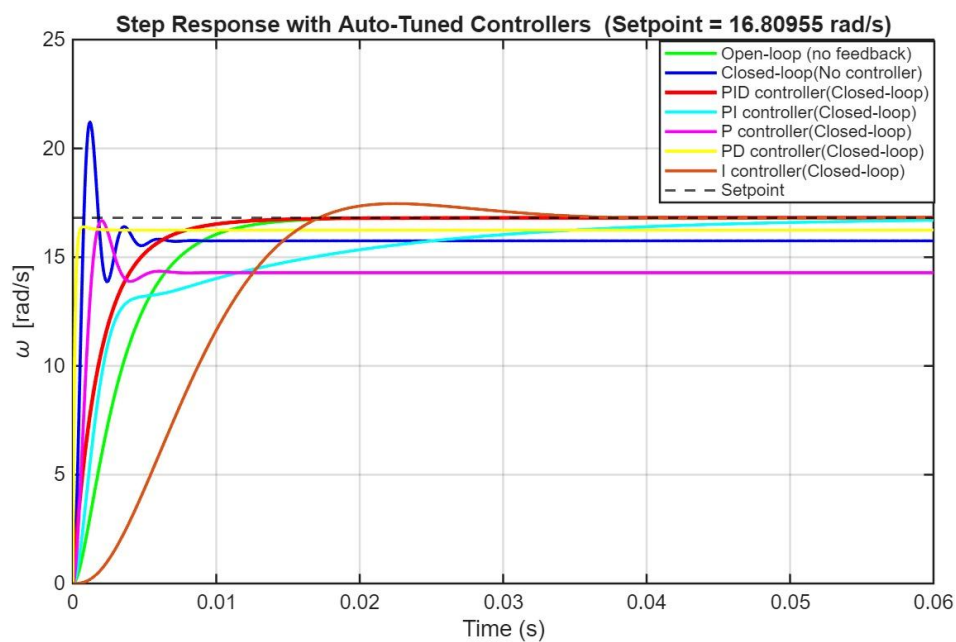


Figure 5.3 – PID controller with autotune

The closed-loop PID system's transfer function is

$$T(s) = \frac{C(s)G(s)}{1 + C(s)G(s)}. \quad (5.9)$$

The transfer function of unity feedback without PID is

$$T_0(s) = \frac{G(s)}{1 + G(s)C(s)} \quad (5.10)$$

Traces that include the integral term reach the setpoint line because a first-order loop drives the step error to zero. Adding the derivative term improves damping, meaning that PD and, in particular, PID settle faster and with less peaking than P or PI. Pure I removes bias, but responds slowly. Unity feedback with no controller results in larger peaking and a visible offset, as there is no integral action to eliminate steady-state error.

Table 5.2 presents data on the basis of which the following conclusions can be drawn. Among the zero bias options, PID achieves the shortest settling and virtually zero overshoot, which makes it the most balanced choice for the speed loop. PI also removes offset but settles slower, which may be acceptable when simplicity and low noise are priorities. PD gives the sharpest initial rise yet cannot remove bias, so it is unsuitable on its own for precise tracking. P control is quick but underdamped and leaves the largest steady state error. Pure I eliminate offset but is the slowest and shows noticeable peaking. The case without any regulator confirms that feedback alone is not enough for the required specifications, and a tuned PID is justified for the conveyor motor.

Table 5.2 – PID controller with autotune

Nº	Configuration	Rise time (s)	Settling time (s)	Overshoot (%)	Final value (rad/s)	SSE (rad/s)
1	2	3	4	5	6	7
1	Without regulator	0.0005	0.0040	34.57	15.7534	1.0561
2	PID	0.0047	0.0095	0.00	16.8096	0.0000
3	PI	0.0173	0.0430	0.00	16.8096	0.0000
4	P	0.0009	0.0045	16.70	14.2877	2.5218
5	PD	0.0002	0.0004	0.90	16.2470	0.5625
6	I	0.0106	0.0293	3.88	16.8096	0.0000

To sum up the table shows that PID provides the best overall results, with a fast settling time, minimal overshoot and zero steady-state error. PI also removes bias, but is slower. P and PD remain biased; I is slow; and the case without a regulator results in overshooting and inaccuracy.

This is the coefficients for PID controller:

$$K_p = 0.126987459178308, \quad (5.11.1)$$

$$K_i = 31.5387669386669, \quad (5.11.2)$$

$$K_d = 7.42817361586824^{-5}. \quad (5.11.3)$$

The table confirms that the auto-tuned PID controller provides the best overall compromise among the classical controllers. It eliminates steady-state error, avoids overshoot, and settles within 0.0095 s.

5.2 Linear-Quadratic Regulator

The Linear-Quadratic Regulator is a state-feedback method for linear systems. It uses the full state vector to calculate the control input and minimize a quadratic performance index that balances state deviation and control effort.

$$u = -kx + Nbar \cdot r \quad (5.12)$$

For the standard LQR design, the feedback gain and reference scaling were

$$K = [2.9162 \quad 0.0303], \quad Nbar = 0.101856, \quad (5.13)$$

The standard LQR uses reference scaling to remove the steady-state error for the nominal motor model. However, this approach does not include an integral state, so disturbance rejection depends on the accuracy of the model and the absence of unmodelled load torque.

An integral-augmented LQR was also tested to improve tracking robustness and disturbance rejection. In this case, an additional integral state was added to accumulate the speed error. The obtained gains were:

$$K_{state} = [3.7056 \quad 0.9317], \quad K_i = -536.6563. \quad (5.14)$$

The negative sign of K_i follows from the sign convention used in the augmented state-space model. With reasonable diagonal weights Q and R , LQR provides a transparent way to tune the balance between response speed and control effort. This property is important for the painting line because the conveyor must move steadily under small delays and load variations.

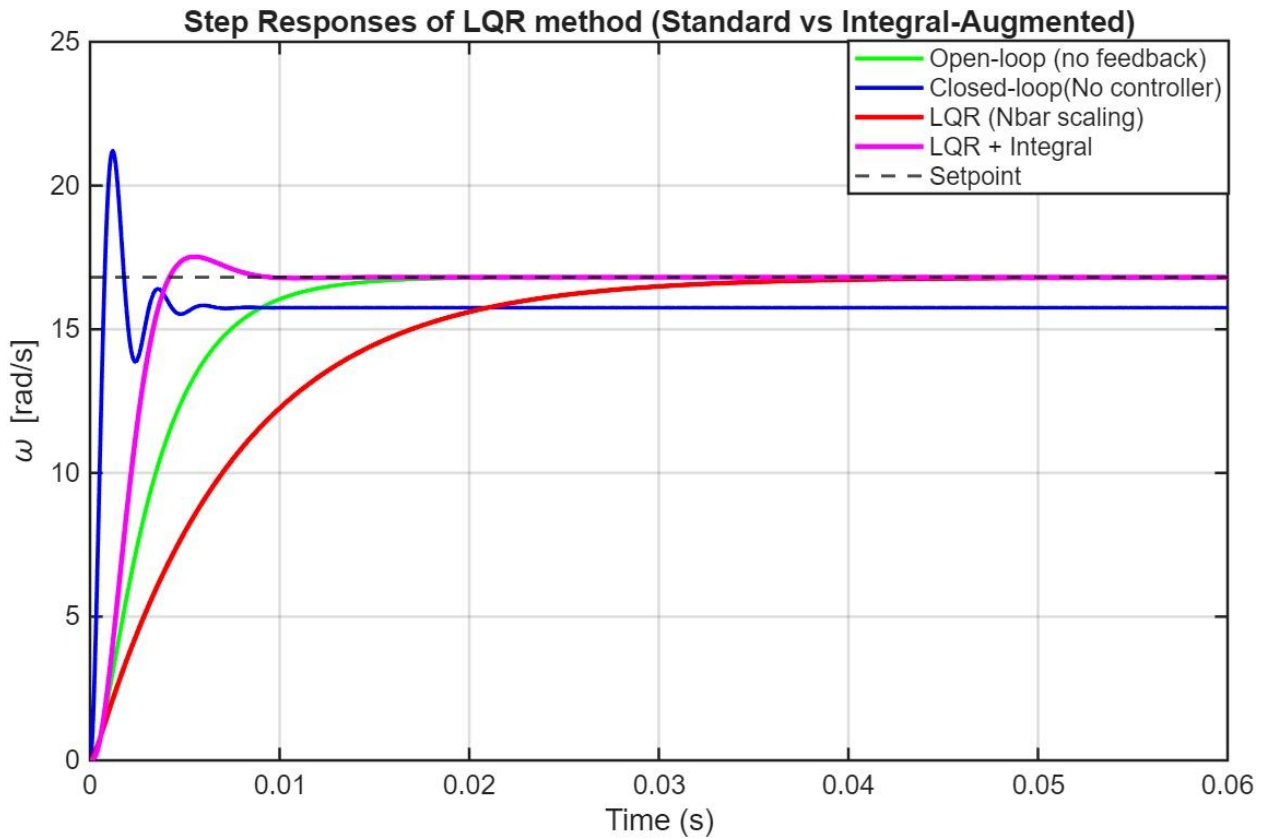


Figure 5.4 - LQR controller

Figure 5.4 shows that the standard LQR response is smooth, monotonic, and reaches the setpoint with zero steady-state error in the nominal model. However, it is slower than the auto-tuned PID controller. The integral-augmented LQR is faster and also removes steady-state error, but it introduces a small overshoot of 4.25%. Therefore, LQR is a strong option when full state feedback is available and smooth motion is prioritized.

Table 5.3 – LQR controller

N	Configuration	Rise time (s)	Settling time (s)	Overshoot (%)	Final value (rad/s)	SSE (rad/s)
1	2	3	4	5	6	7
1	Without regulator	0.0005	0.0040	34.57	15.7534	1.0561
2	Standard LQR	0.0165	0.0296	0.00	16.8096	0.0000
3	LQR with integral action	0.0026	0.0073	4.25	16.8096	0.0000

In the step tests, the baseline closed-loop system reacts very quickly, but it overshoots the setpoint by 34.57% and settles with a steady-state error of 1.0561 rad/s. Such behavior is risky for the painting line because short speed peaks and the remaining offset can reduce coating quality. The standard LQR response is slower

at the start, with a rise time of 0.0165 s and a settling time of 0.0296 s, yet it reaches the target without overshoot and with zero steady-state error in the nominal model. The transient is smooth and non-oscillatory, which lowers mechanical stress and keeps the conveyor speed stable. The integral-augmented LQR is faster, with a rise time of 0.0026 s and a settling time of 0.0073 s, but it produces a small overshoot of 4.25%. Therefore, LQR improves smoothness and accuracy compared with the baseline loop, while the integral-augmented version provides a faster alternative with a small overshoot.

5.3 Comparison of methods: results and choice

This subsection compares the main tested control methods in order to determine which one is most suitable for BLDC motor speed control in the conveyor system. The comparison is based on the same plant model, the same setpoint of 16.80955 rad/s, and the same evaluation criteria for each method. The main indicators are rise time, settling time, overshoot, final value, steady-state error, and the overall smoothness of the transient response. The auto-tuned PID controller provides the best practical balance between speed, accuracy, and smoothness. It reaches the setpoint with a rise time of 0.0047 s and a settling time of 0.0095 s, without overshoot and with zero steady-state error. Visually, the response approaches the setpoint smoothly and remains stable, which satisfies the requirements of the conveyor speed loop.

Table 5.4 – 3 methods analysis table

N	Configuration	Rise time (s)	Settling time (s)	Overshoot (%)	Final value (rad/s)	SSE (rad/s)
1	2	3	4	5	6	7
1	Without regulator	0.0005	0.0040	34.57	15.7534	6.28
2	Auto PID regulator	0.0047	0.0095	0.00	16.8096	0.00
3	ZN–PID regulator	0.0009	0.0056	32.47	16.8096	0.00
4	LQR	0.0165	0.0296	0.00	16.8096	0.00
5	Integral + LQR	0.0026	0.0073	4.25	16.8096	0.00

The Ziegler-Nichols PID controller also eliminates steady-state error and has a very short rise time of 0.0009 s. However, its transient response is aggressive and produces a high overshoot of 32.47%. Therefore, the Ziegler-Nichols method is useful as an initial tuning approach, but the obtained gains should be detuned before practical implementation in the conveyor drive.

The standard LQR controller has no overshoot and no steady-state error, but it is slower than the auto-tuned PID controller. This response is smooth and reliable, making LQR suitable for cases where maximum smoothness is more important than

response speed. The integral-augmented LQR improves the speed of response and keeps zero steady-state error, but it introduces a small overshoot of 4.25%.

Considering both the numerical indicators and the visual behavior of the curves, the auto-tuned PID controller is selected as the primary controller for the conveyor motor speed loop. It provides the best overall balance of response speed, stability, and tracking accuracy while remaining simple to implement in the existing PLC-based control architecture. Ziegler-Nichols tuning can be used as a preliminary method or backup reference, while LQR remains a strong alternative when full state feedback and smoother motion are required.

5.3.1 Simulink Verification of the Auto-Tuned PID Controller

For the PID controller with Autotune method, there was made Simulink model. From the Simulink model there was observed the step response for the closed loop system without feedback, open loop and PID controller with Autotune method.

The setpoint for the process was established, which equal to 16.2516 RPM (radians per second).

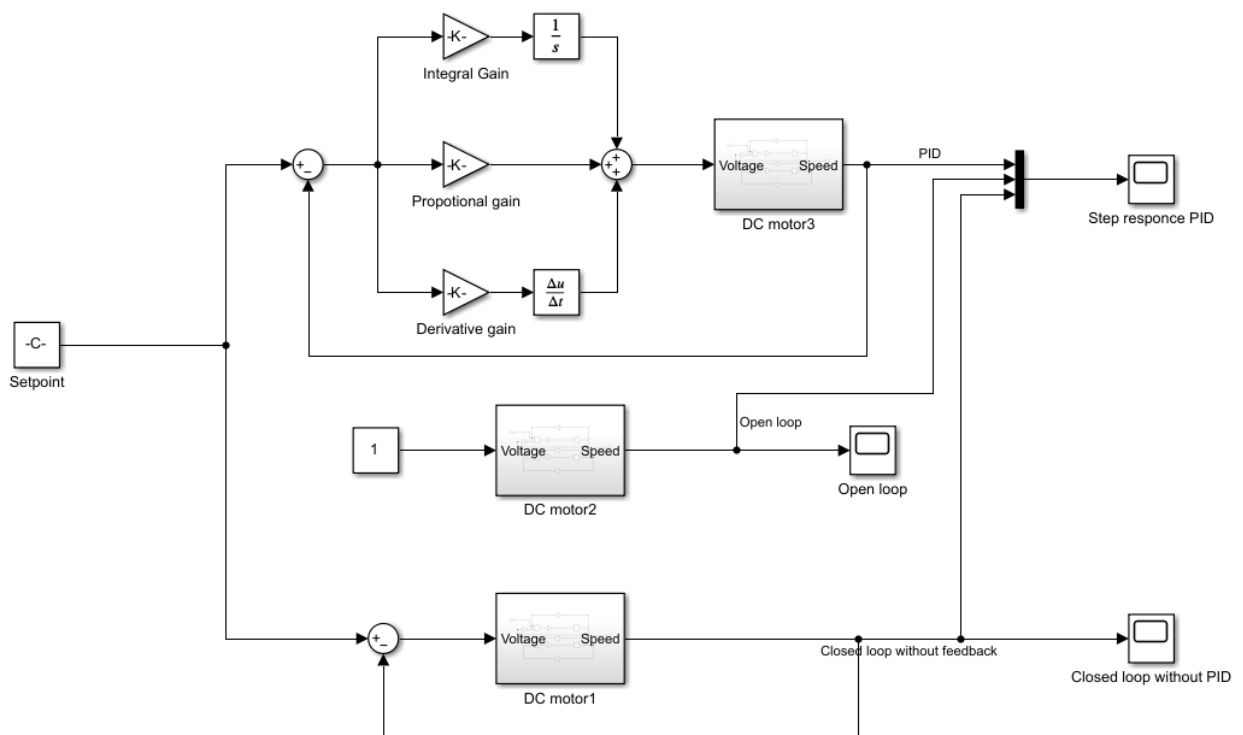


Figure 5.6 – Simulink model for the Autotune PID regulator

Figure 5.7 shows the DC motor transfer-function blocks used in the Simulink model. These blocks represent the same motor transfer function derived in Chapter 3 and used in the MATLAB controller analysis. The Simulink verification confirms that the auto-tuned PID controller provides the most suitable response for the conveyor speed-control process

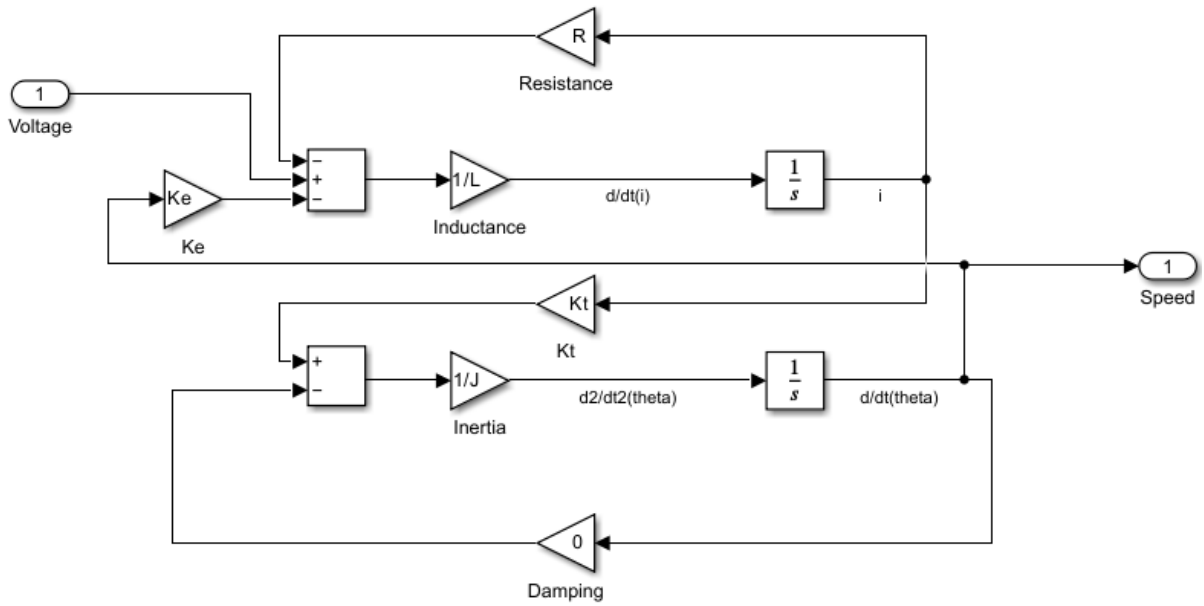


Figure 5.7 – DC Motor from Simulink model

From all the observations which was made this Simulink model is the most suitable for the whole process, as proved before.

5.4 Conclusion

In this chapter, several controllers for the conveyor motor speed loop were designed, simulated, and compared. The Ziegler-Nichols method produced usable initial gains but resulted in large overshoot, which makes it unsuitable for direct implementation without detuning. The auto-tuned PID controller provided the best practical balance between speed, accuracy, and smoothness. It reached the required setpoint with zero steady-state error and no overshoot. The standard LQR controller produced a very smooth response with zero steady-state error, but it was slower than the auto-tuned PID controller. The integral-augmented LQR improved the response speed while introducing a small overshoot.

Based on the simulation evidence, the auto-tuned PID controller is selected as the primary solution for the conveyor motor speed loop. Ziegler-Nichols tuning is kept as a preliminary tuning reference, while LQR is reserved for cases where maximum smoothness and state-feedback control are preferred. The chapter therefore provides a clear controller choice and a reproducible workflow that can be applied to future control loops in the system.

6 COMPLEX OF THE TECHICAL DEVICES

The automated paint line is a system that combines industrial controllers, actuators, sensors, and tools to analysis data. All devices are integrated into a single information and control field using industrial PROFINET and OPC UA protocols, which ensures real-time operation as well as the possibility of further scaling. On the figure 6.1 presented Purdue model of this system.

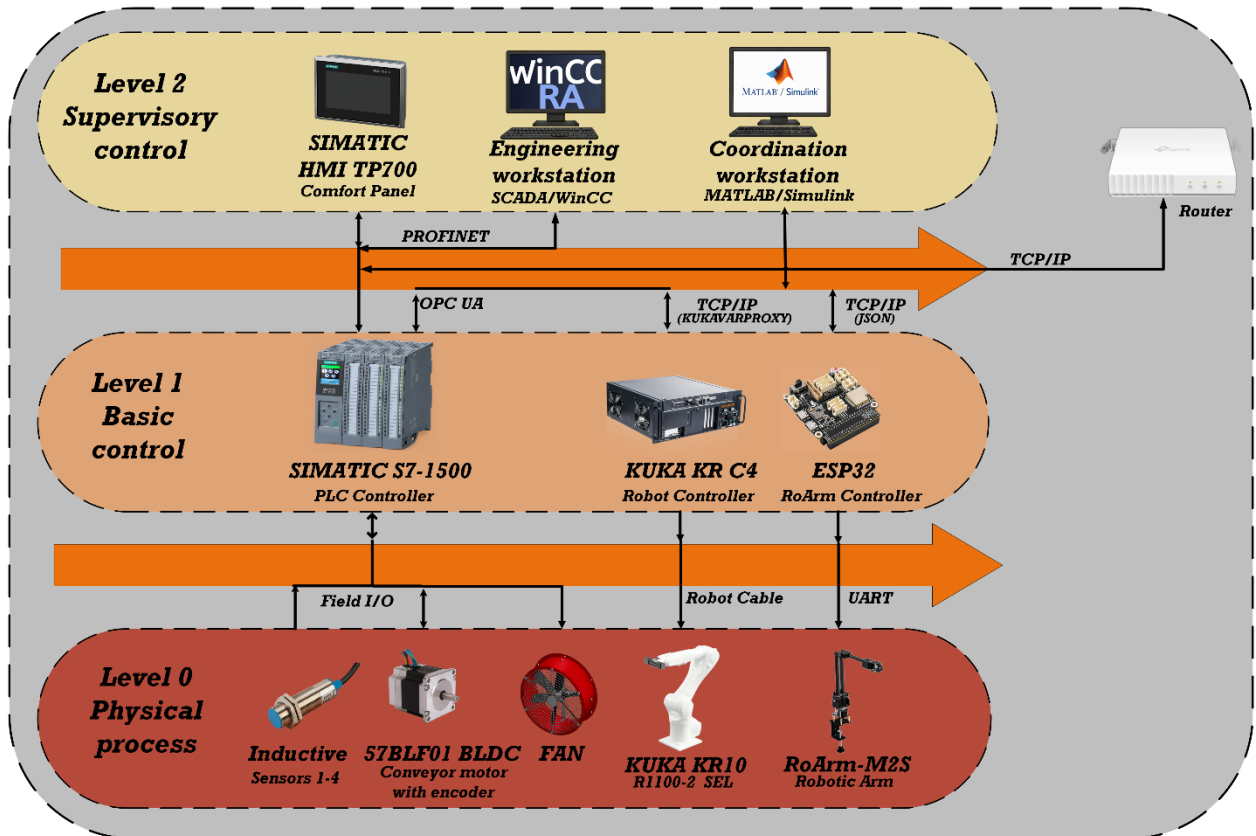


Figure 6.1 – Purdue model of the technological process

Level 0 – Physical Process. This level contains all devices that work directly with the physical process. Four inductive sensors detect workpieces on the conveyor. The 57BLF01 BLDC motor with encoder drives the conveyor belt. A fan is used for drying. The KUKA KR10 R1100-2 SEL performs handling tasks and the RoArm-M2S executes painting operations. These devices connect to Level 1 controllers through Field I/O, robot cable, and UART.

Level 1 – Basic Control. This level includes controllers that run the control programs in real time. The SIMATIC S7-1500 PLC is the main controller — it reads sensor data and sends commands to the motor and fan. Here the KUKA KR C4 controls the KUKA KR10 manipulator through an internal cable. ESP32 controls the RoArm-M2S via UART. All controllers connect to Level 2 through a TCP/IP network using a common router.

Level 2 – Supervisory Control. This level provides process monitoring and coordination. The SIMATIC HMI TP700 Comfort Panel connects to the PLC via

PROFINET and shows the process status to the operator. The Engineering Workstation runs WinCC SCADA, which collects data from the PLC through OPC UA. The Coordination Workstation with MATLAB/Simulink plans trajectories and communicates with the KUKA KR C4 via TCP/IP (KUKAVARPROXY) and with the ESP32 via TCP/IP (JSON). MATLAB also receives real-time feedback from both robots for monitoring.

6.1 Information about the controller type

The SIMATIC S7 - 1500 is the brand new series of controllers from Siemens . This series of controllers designed to automate industrial processes . S7 - 1500 series of controllers support Step 7 (Tia Portal). Also it supports IEC 61131-3, which is international standard for PLC languages. IEC 61131-3 contains several programming languages like as LAD, FBD, SCL, Graph STL . And S7 - 1500 supports communication protocols like as PROFINET, Ethernet, and OPC UA. Which allows direct integration with SCADA, HMI, drives and robotic systems .



Figure 6.2 – Training case S7-1500

For this project, there was used SIMATIC S7-1500 Training Case (6ZB2310-0DK00). This training case contains fully functional PLC (Figure 6.2). Which has in it a CPU module, power supply, configurable Discrete and Analog I/O modules. It can directly connect to the sensors, actuators, security devices and has network communication. The CPU series of the training case is S7 1513F-1 PN . Here on this training case Digital Input modules.



Figure 6.3 – S7-1513F-1 with I/O modules

The CPU 1513F-1 PN is a failsafe controller, meaning it supports both conventional automation functions and functional safety tasks according to SIL 3

(Safety Integrity Level 3) and PL e(Performance Level e) standards. For this type of PLCs where it has solid processing power and has memory for middle sized systems, it f production cells, robotic workstations, drive control, and training rigs. Its built-in diagnostics and high reliability help ensure efficient operation and safe equipment behavior.

For communications, it includes a PROFINET interface with an integrated 2-port switch, enabling real-time links to distributed I/O and other devices. It also has an onboard OPC UA server for direct, secure data exchange with SCADA systems, HMIs, and robot controllers no external gateway required streamlining integration and interoperability.

The selection of the main controller was based on a comparative analysis of the Siemens S7-1500 and the Modicon M241 models. The analysis focuses on key technical characteristics such as memory capacity, programming tools, communication interfaces, and protocol support. The results of this comparison are presented in Table 6.1, which highlights the main similarities and differences between the two controllers.

Table 6.1 Comparison of Controllers

Parameter	SIMATIC S7 - 1500	Modicon M241 (TM241CE24R)
1	2	3
Number of Points	1024	Logic Controller 1024
Memory	1.5 Mb for data	64 MB for data
1	2	3
Memory	0.450 Kb for program	8 MB for program
Memory Card	yes	yes 24 (14 inputs, 10 outputs)
Programming Language	LAD, FBD, SCL, STL, Graph	LD, FBD, SFC, ST, IL 0.3 ms for event task, 0.7 ms for other instructions
Programming Tools	TIA Portal	EcoStruxure Machine Expert Small automation systems with moderate I/O needs
Network	PROFINET, Modbus TCP, OPC UA	Modbus, Ethernet/IP, FTP, OPC UA, Web Server
OPC UA	supported	supported -10 to 55°C

The Modicon M241 physically starts and can stop the conveyor via PROFINET, but cannot ensure stable simultaneous operation with MATLAB and two manipulators through OPC UA - the server freezes every 10–12 minutes, and the painting cycle increases from 5.2 s to 14.7 s. Table 6.2 provides key specifications about SIMATIC S7-1513F-1 PN central processing unit.

Table 6.2 – Specifications of 6ES7 513-1FL02-0AB0

Parameter	Characteristics
1	2
Range of product	SIMATIC S7-1500 automation system
Product type	Fail-safe central processing unit
Concept	PROFINET IRT
Applications	Standard and fail-safe applications
Memory	900 KB program memory; 2.5 MB data memory
Rack configuration	1 rack; up to 32 modules per rack
I/O capacity	Local and distributed I/O modules
Communication	1× PROFINET IRT; 2-port switch; 2× RJ45

Continuation of table 6.1

1	2
Execution time	6 ns bit; 7 ns word; 9 ns fixed-point; 37 ns floating-point
Power / dimensions	24 V DC; 35 × 147 × 129 mm; 336 g

The Siemens S7-1500 is the only controller that guarantees 100 % repetition of the declared cycle of 4.8–6.2 seconds 24/7, zero additional licenses and full compatibility with the existing network Figure 6.1. Therefore, the SIMATIC S7-1513F-1 PN remains the recommended and actually verified solution for this automated painting cell.

6.2 Description of controller modules

SIMATIC S7-1500 controller for this project contains the power supply, central processing unit where it is the head of the entire PLC. The system contains digital input , digital output and analog input modules.

The digital input is the SIMATIC S7-1500 DI module (6ES7521-1BL00-0AB0) with 32 channels at 24 V DC (High-Feature class). It captures signals from switches, proximity sensors, photoelectric sensors and other field devices. Offering fast input response, configurable debounce/filter times, per-channel status LEDs and integrated diagnostics, it speeds up troubleshooting. Designed using TIA Portal, it is ideal for medium-sized cells and robot workstations where reliable, high-density input acquisition is required.



Figure 6.4 – Digital input module

Within the system, the module's signals are processed by the CPU and then shared with distributed devices in real time via PROFINET. For higher-level integration, the CPU's built-in OPC UA server directly exposes those data points to

SCADA, HMIs, or robot controllers, eliminating the need for extra gateways and streamlining interoperability while keeping configuration centralized.

Table 6.3 – Specifications of 6ES7521-1BL00-0AB0

Parameter	Characteristics
1	2
Product Type	Discrete Input module
Input Type	Current sink
Discrete Input Number	32
Discrete Input Voltage	24V DC Positive
Discrete Input Current	2.5 mA
Operating Temperature	0°C ... +60°C
Storage Temperature	–40°C ... +70°C
IP Degree of Protection	IP20
Mass	0.260 kg
Dimensions	35 mm × 147 mm × 129 mm

The Digital Output 6ES7522-1BL01-0AB0 is a SIMATIC S7-1500 DO module featuring 32 24 V DC channels, each of which can deliver up to 0.5 A. The module is designed to drive various loads, including pneumatic valve manifolds, motor contactors, grippers, signal lamps and buzzers. Built-in short-circuit and overload protection ensures stable and safe operation, and the High-Feature (HF) class enables per-channel diagnostics, speeding up troubleshooting and reducing downtime.



Figure 6.5 – Digital output module

In this project, the module provides the necessary number of discrete outputs for external actuators (see Figure 6.5), with the relevant specifications provided in Table 6.4. The CPU executes commands and distributes them in real time over PROFINET to field devices. At the supervisory level, the integrated OPC UA server

in the CPU exposes these output states and alarms directly to SCADA, HMIs, or robot controllers, eliminating the need for additional gateways.

Table 6.4 – Specifications of 6ES7522-1BL01-0AB0f

Parameter	Characteristics
1	2
Product Type	Discrete Output module
Output Type	Transistor
Discrete Output Number	32
Discrete Output Voltage	24V DC Positive
Discrete Output Current	0.5 mA
Operating Temperature	0°C ... +60°C
Storage Temperature	–40°C ... +70°C
IP Degree of Protection	IP20
Certificates	PL d, SIL 2,
Dimensions	0.260 kg 35 mm × 147 mm × 129 mm

Analog Input 6ES7531-7KF00-0AB0 is a SIMATIC S7-1500 AI module (Figure 6.6) with eight universal channels for voltage, current, thermocouples, and RTDs at 16-bit resolution . It has wire-break monitoring, alarms, configurable integration times and smoothing. Also it can calibrate in RUN time for fast commissioning. An onboard 24 V encoder supply (short-circuit protected) powers field transmitters; module supply is 24 V DC .



Figure 6.6 – Analog input module

In this project, the module provides the required analog measurement capacity for key process variables, with the main specifications summarized in Table 6.5.

Table 6.5 – Specifications of 6ES7522-1BL01-0AB0f

Parameter	Characteristics
1	2
Product Type	Discrete Input module
Input Type	P-reading
Discrete Input Number	32
Discrete Input Voltage	24V DC Positive
Discrete Output Current	0.5 mA
Operating Temperature	0°C ... +60°C
Storage Temperature	–40°C ... +70°C
IP Degree of Protection	IP20
Certificates	PL d, SIL 2,
Dimensions	0.260 kg 35 mm × 147 mm × 129 mm

The power supply module monitors the status of each input and passes this information to the CPU via the system backplane bus. In this project, the 6EP1332-4BA00 provides the necessary power for the PLC CPU and I/O modules (see Figure 6.7).



Figure 6.7 – Power supply module

The characteristics of the Power supply module listed below in a Table 6.6. All the necessary data is located here.

Table 6.6 – Specifications of 6EP1332-4BA00

Parameter	Characteristics
1	2
Product Type	Power Supply Module
Input Type	P-reading

Continuation of table 6.6

1	2
Discrete Input Number	32
Discrete Input Voltage	24V DC Positive
Discrete Output Current	0.5 mA
Operating Temperature	0°C ... +60°C
Storage Temperature	−40°C ... +70°C
IP Degree of Protection	IP20
Certificates	PL d, SIL 2,
Dimensions	0.260 kg 35 mm × 147 mm × 129 mm
Supply Circuit Type	AC
Supply Voltage	120V/230V
Input Current	1.4 A(120V) 0.8 A(230V)
Protection Types	Overvoltage, short-circuit protection
Status LEDs	LED green for 24 V OK; LED red for error; LED yellow for stand-by
Net Weight	0.45 kg
Operating Temperature	0°C ... +60°C
Storage Temperature	−40°C ... +85°C
IP Degree of Protection	IP20

It supports diagnostic functions such as wire-break, signal fault, and short-circuit detection to enhance system reliability and simplify maintenance. Configuration options, including input filtering and behavior, can be adjusted in the TIA Portal engineering environment.

6.3 Information about data networks

The data transmission network for the paint line consists of two standard industrial protocols and fully complies with the scheme shown in Figure 6.1. PROFINET helps to exchange data within the line quickly and in real time. Four inductive sensors, digital I/O modules, a DC electric drive and a TP700 Comfort HMI control panel are connected to the S7-1500 via a single PROFINET line. When an object appears on the conveyor, it is understood by the system as an inductive sensor, the sensor signal is sent to the controller in less than 1 ms. The PLC starts the engine instantly, and further on the situation can also turn it off. The HMI panel updates the screen every 500 ms to display the current value.

OPC UA (Open Platform Communications Unified Architecture) is universal communicational industrial standard for data exchange between various devices and programmable systems by different productions. The main basic architecture logic are server, client and services. The OPC UA connects the controller to external

systems, acting as a second layer. The built-in OPC UA server of the CPU 1513F-1 PN processor continuously provides access to all process tags. MATLAB communicates with them with a refresh rate of 50 ms, receives information about the installed area and sends commands to both manipulators via the OPC UA channel. At the end of their program, the manipulators send an end-of-action signal, and the PLC receives the signal and starts or stops the motor.

6.4 Operator interface

The operator interface of the automated painting line is realized on the SIMATIC HMI TP700 Comfort panel 6AV2124-0GC01-0AX0, which is produced by Siemens. This panel belongs to the Comfort Panel series and is designed for machine-level visualization and monitoring tasks in industrial environments. TP700 Comfort provides the operator with real-time access to the process status, alarm management and manual control of the system. The panel is shown in Figure 6.8.



Figure 6.8 – SIMATIC HMI TP700 Comfort Panel

The TP700 Comfort features a 7-inch widescreen TFT touch display with a resolution of 800×480 pixels and 16 million colors, which provides clear and detailed visualization of the process screens. The panel operates on Windows CE 6.0 and has 12 MB of configuration memory, which allows to store up to 500 configurable pages with 400 objects per page. Additionally, it has 2 MB of internal memory for recipes. The memory can be expanded using USB or SD card slots.

For communication, the TP700 Comfort is equipped with a PROFINET interface with an integrated 2-port switch and an MPI/PROFIBUS DP interface. In this project, the panel is connected to the SIMATIC S7-1500 PLC via PROFINET, which provides real-time data exchange between the controller and the operator screen. The panel updates the displayed values every 500 ms, allowing the operator

to monitor conveyor status, sensor states and manipulator activity. The panel also has one RS 422/485 interface, two USB 2.0 ports and two SD card slots for additional data transfer and archiving.

The TP700 Comfort has an IP65 protection rating on the front side and IP20 on the rear side, which makes it resistant to dust and low-pressure water jets. The operating temperature range is from 0°C to 50°C. The panel is designed for vertical mounting and operates safely in humidity conditions up to 90% without condensation. All the necessary technical characteristics of the TP700 Comfort panel are provided in Table 6.7.

Table 6.7 – Specifications of 6AV2124-0GC01-0AX0

Specification	Value
1	2
Model	6AV2124-0GC01-0AX0
Type	HMI Comfort Panel
Display	7-inch widescreen TFT
Resolution	800 × 480 pixels
Colors	16 million
Configuration memory	12 MB
Recipe memory	2 MB
Operating system	Windows CE 6.0
Supply voltage	24 V DC
Current consumption	0.5 A
Power consumption	12 W
Communication interfaces	1 × PROFINET (2-port switch), 1 × MPI/PROFIBUS DP
Additional interfaces	1 × RS 422/485, 2 × USB 2.0, 2 × SD card slots

The main user interface of the painting line in this project is the SIMATIC HMI TP700 Comfort panel. It provides status information for the conveyor, sensors, motors and manipulator. The PROFINET interface to the S7-1500 PLC enables high-speed communication and the IP65 front protection provides a safe operation in harsh environments. All alarm and screen design is carried out in WinCC within the TIA Portal, making programming easier and maintaining all system settings within one programming environment

6.4 Specification of sensors with basic characteristics

Contact-free sensors are used for contactless detection of metal objects in production, which are the most cost-effective solution. During operation, when an electric or magnetic wire is approached or removed from the sensor, the signal automatically changes. At the moment when the object is in range, playful currents arise, which directly affect the oscillation amplitude of the generator. The oscillation

amplitude is analyzed through the built-in electronic circuit, so when it passes the sensitivity threshold, the output of the NO/NC sensor changes. Such a simple operation scheme leads to the fact that they are not subject to wear and are correspondingly insensitive to temperature, light, water and noise, which makes them durable. The figure 6.9 inductive sensor, which is produced by Siemens company.



Figure 6.9 - SIMATIC PXI200 M12 NOT flush

Inductive sensors have a wide range of types and different types. These reliable devices are popular in the industry with constant feedback. They are also often used in industries where metal components are the main control object. For example, machine manufacturing, robotics, conveyor systems, as well as the automotive industry.

This type sensor has several specification in construction and protection degree. Furthermore, the all additional technical characteristics are given in the table 6.8.

Table 6.8 - Technical characteristics of inductive sensor

Specification	Value
1	2
Model	3RG40 22-3JB00
Type	Inductive proximity sensor (2-wire, PLC type)
Operating voltage (DC)	15 ... 34 V DC
Rated operating distance (Sn)	4 mm (not flush)
Output type	2-wire, NO (normally open)
Protection degree	IP67
Connection type	M12 connector / 2 × 0.25 mm ² cable
Housing dimensions	M12 × 40 mm
Mounting distance	≥ 15 mm between sensors
Power-up delay (tv)	40 ms

Continuation of table 6.8

1	2
Enclosure material	Brass, nickel-plated
Switching indicator	Yellow LED
Switching frequency	300 Hz

Siemens has long proven the reliability of its devices. The German company has created a series of contactless sensors with different specifications and features for specific tasks. It can be noted that these devices have the international quality standard IEC 60947-5-2, EN 60947-5-2 (VDE 0660). Standards evaluate the quality of materials, durability, and have essential protective properties such as moisture and dust resistance.

6.5 Specifications of actuators

After receiving data from the sensors, as well as after implementing logic in the controller, the PLC sends data to the actuators that execute the program. An actuator is a device used to convert control signals into physical motion. They play an important role in automation systems. The actuating devices are mainly valves, pumps, lamps, clamping, pressing and packaging machines, etc. Since the control object is the 57BLF01 BLDC conveyor motor, as well as the KUKA KR10 R1100-2 SEL and RoArm-M2S manipulators, they will be discussed in the future.

6.5.1 Specification Brushless DC Motor

The BLDC 57BLF01 motor was selected to rotate the conveyor belt at a certain angular velocity depending on the conveyor speed. This engine differs from others in that it is more powerful than conventional brush engines and is also the most optimal depending on the power and price ratio. The figure of the motor is shown in Figure 6.10.



Figure 6.10 - 57BLF01 BLDC motor

The drives in this series are highly efficient with significant torque and moderate rotational speed, which is applicable to this system. All technical characteristics of motor is shown in the table 6.9.

Table 6.9 - Technical characteristics of brushless motor

Specification	Value
1	2
Rated Voltage	24 V DC
Rated Current	4 A
Rated Speed	3000 RPM
Number of Poles	8
Number of Phases	3
Output Power	63 W
Rated Torque	0.2 Nm
Winding Type	Star
Mass	0.65 kg
Body Length	59 mm
Ambient Temperature Range	-20°C to 50°C

It is mainly used in CNC machines, 3D printers and also in medical equipment. Since the selected 57BLF01 is a three-phase brushless DC motor, it cannot be connected directly to the PLC output module or to a simple DC power supply. For stable operation, a special BLDC motor driver is required. In this project, the 57BLF01 motor is used together with the BLDC-8015A driver, which is supplied as a compatible set with the motor. The driver which is shown in Figure 6.11 converts the DC supply voltage into controlled three-phase output signals for the U, V and W motor windings and also processes Hall sensor feedback from the motor.

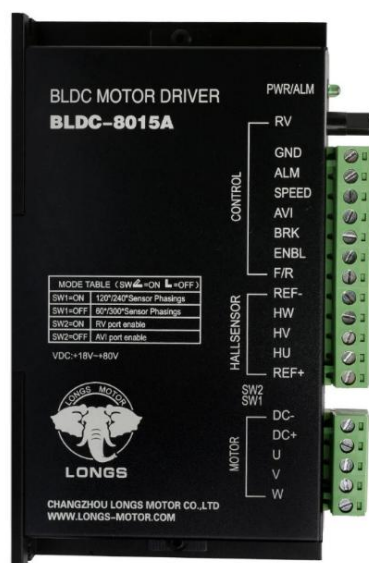


Figure 6.11 – BLDC-8015A driver with 57BLF01 brushless DC motor

The BLDC-8015A driver provides speed and direction control of the conveyor motor. The speed can be adjusted using a potentiometer or an analog input signal, while the direction of rotation can be changed using the CW/CCW control input. The driver also supports quick braking, run/stop control, speed output and alarm output. These functions make it suitable for conveyor applications where smooth rotation, reliable stopping and protection against electrical faults are required.

Table 6.10 – Technical characteristics of BLDC-8015A driver

Specification	Value
1	2
Product type	Brushless DC motor driver
Model	BLDC-8015A
Compatible motor	57BLF01 BLDC motor
Supply voltage	24–50 V DC
Rated output current	15 A
Peak output current	45 A, up to 3 s
Driving mode	SPWM
Speed control	Potentiometer / analog input
Direction control	CW/CCW rotation input
Additional functions	Run/stop, quick brake, speed output, alarm output
Protection functions	Overcurrent, overvoltage, motor stall, missing speed alarm
Cooling method	Self-cooling
Weight	About 300 g
Operating temperature	0°C to +50°C

In the automated painting line, the PLC sends control commands to the drive system, while the BLDC-8015A driver performs the electrical control of the conveyor motor. This separation is important because the PLC is responsible for logic and sequence control, whereas the driver is responsible for power conversion, phase commutation, speed regulation and motor protection. Therefore, the BLDC motor and its driver should be considered as one complete conveyor drive unit.

6.5.2 Specifications of manipulators

Manipulators are multi-axis electromechanical systems that consist of several manipulable joints. They are used to execute programmed motions and handling, positioning, painting, dosing and other tasks in automation systems. In this project, the RoArm-M2-S robotic arm is used to demonstrate the movement and spot-painting operations of the compact manipulator in the conveyor painting system. The RoArm-M2-S is a smart 4-DOF robotic arm shown in figure 6.12.



Figure 6.12 - RoArm-M2S

It features a lightweight design (carbon fiber and 5052 aluminum alloy), a 360° spin base, and three flexible joints, resulting in a workspace of about 1 meter diameter. The manipulator features an ESP32 control board and can communicate via Wi-Fi, ESP-NOW, UART, USB, HTTP requests and JSON commands. This enables its use in laboratory automation, robotic education, and for secondary development. The robot can be programmed via a multi-platform web user interface, JSON commands, or a ROS2-based software. In this project the RoArm-M2-S is used as a cheap and flexible robotic actuator for coordinated motion testing with the conveyor and the PLC control system. The key technical specifications of the RoArm-M2-S are given in Table 6.11.

Table 6.11 - Technical characteristics of RoArm-M2S

Specification	Value
1	2
Power Input	7–12.6 V DC
Recommended Power Supply	12 V, 5 A power supply
Weight	Less than 900 g
Payload	0.5 kg at 0.5 m
Work Space	Approximately 1 m diameter, 360° omnidirectional
Degrees of Freedom	4 DOF
Controller	ESP32-WROOM-32
Range of Motion of Base	360°
Range of Motion of Shoulder	180°
Range of Motion of Elbow	225°
Range of Motion of End Joint	135° in gripper mode / 270° in wrist mode

Continuation of table 6.11

1	2
Feedback System	12-bit magnetic encoder
Communication Interfaces	Wi-Fi, ESP-NOW, UART, USB, HTTP/JSON

The RoArm-M2-S manipulator is not used as a large industrial robot in the automated painting line, but as a small robotic actuator for demonstration of motion in the laboratory environment and testing of painting trajectories. The open-source software, web control and ROS2 support make it easy to integrate with MATLAB/Simulink and other higher-level control software. The manipulator motion is controlled by the control system and the PLC is used for sequence control, interlocks and communication with other devices.

The robot can be controlled manually using a training console or automatically in KUKA software. The KUKA KR10 R1100-2 SEL manipulator with an implemented KR C4 compact controller was selected. The manipulator is shown in Figure 6.13.



Figure 6.13 - KUKA KR10 R1100-2 SEL

Manipulators are required to capture and place an object on the conveyor. The device provides positioning on 6 axes, performing actions with minimal error. Technical characteristics of KUKA KR10 R1100-2 SEL is shown in the table 6.12.

Table 6.12 - Technical characteristics of KUKA KR10 R1100-2 SEL

Specification	Value
1	2
Power Input	220-240 V
Weight	59 kg

Continuation of table 6.12

1	2
Payload	10 kg
Maximum Payload	11 kg
Maximum range	1101 mm
Accuracy	±0,02 mm
Degree of freedom	6
Temperature range	0°C-45°C
Controller	KR C4 compact
Work Space	208×208 mm
Range of motion along the axes 1	±170°
Range of motion along the axes 2	-190°/45°
Range of motion along the axes 3	-120°/156°
Range of motion along the axes 4	±185°
Range of motion along the axes 5	±120°
Range of motion along the axes 6	±350°

Manipulators are mainly used in automated painting, welding, dosing and assembly lines in the automotive, aerospace and electronic industries, and they are used instead of pneumatic and hydraulic cylinders, increasing accuracy and energy efficiency by 40-60 percent.

6.6 Economic analyzes and cost estimation

The economic part is one of the pillars of the project implementation. In this section, the cost of the main components that will be used in the development of the management system is described in detail. Prices are based on publicly available catalogs from distributors such as Siemens, KUKA and WaveShare, as well as generally accepted market prices for sensors and motors. All prices were reflected relative to educational institutions or small industrial enterprises, respectively oriented and reflect the main retail prices for 2025. In the following table 6.13 the cost of components in USA dollars is shown.

Table 6.13 – The economic analyze

№	Name	Model/Specification	Quantity	Price (USA)	Total (USA)
1	2	3	4	5	6
1	KUKA Manipulator	KR10 R1100-2/SEL	1	42 000	42 000
2	RoArm Manipulator	RoArm M2s	2	307	614
3	BLDC motor	57BLF01 motor + BLDC-8015A motor driver	1	65	65
4	Inductive sensors	SIMATIC PXI200M12	4	155	620

Continuation of table 6.13

1	2	3	4	5	6
5	PLC CPU	6ES7521-1BL10-0AA0	1	1 289	1 510
6	Analog input module	6ES7531-7KF00-0AB0	1	409	458
7	Digital input module	6ES7521-1BL00-0AB0	1	413	413
8	Digital output module	6ES7522-1BL01-0AB0f	1	653	653
9	Power module	6EP1332-4BA00	1	212	212
10	Operator panel	SIMATIC HMI TP700 Comfort, 6AV2124-0GC01-0AX0	1	900	900
11	Wireless router	TP-LINK TL-WR844N	1	15	15
12	Engineering software	TIA Portal V19	1	350	350
13	SCADA software	WinCC Runtime Advanced	1	2 500	2 500
14	MATLAB/Simulink	Student Suite license	1	99	99
15	End effector	Electromagnetic lock	1	25	25
16	Conveyor belt	Conveyor belt	1	400	400
17	Spray gun	Spray gun	2	20	40
	Overall				50 874

Looking at details, the most cost effective components are the KUKA industrial robot manipulator and Siemens PLC system, that overall cost more than 82% of the total project price. However, the RoArm manipulator from waveshare company provide an flexible and affordable solution for complex painting system. The total cost of whole project is approximately 50 874 dollars (USA).

6.7 Conclusion

The industrial automated paint line is designed on the basis of SIMATIC S7-1513F-1 PN controller which offers opportunities to apply real-time functioning protocols like PROFINET, as well as standardized data exchange OPC UA. The chosen digital and analog I/O modules are able to provide good signal capture and safe actuation with inbuilt diagnostics and the power source maintains the stability of all the components. On the whole, the system meets the required accuracy, dependability, and capability to scale to automated industrial painting tasks, and can have new stations or more sophisticated control capabilities added without requiring a redesign of the fundamental architecture.

7 PROGRAM REALIZATION

This section presents the software implementation of a multirobot painting control system integrating a KUKA KR 10 R1100-2 industrial manipulator, two Waveshare RoArm-M2-S lightweight robotic arms, and a Siemens S7-series Programmable Logic Controller (PLC). All high-level control and trajectory generation logic is hosted in MATLAB R2025b, while the PLC is responsible for supervisory interlock, safety enforcement, and process sequencing. The software architecture is designed around a network-transparent communication paradigm: the KUKA robot is driven through the KukaVarProxy middleware over a TCP/IP socket, and each RoArm M2s is driven through HTTP/JSON commands delivered over a shared Wi-Fi network.

The PLC control program, developed in Siemens TIA Portal V19, implements the full technological sequence of the painting line, including conveyor indexing, drying zone management, and robot handshake signals. The operator interface is realized through a WinCC-based HMI that provides mode selection, process visualization, and alarm handling. Conveyor motor speed is regulated by a dedicated PID controller implemented directly within the S7-1500 PLC.

The following subsections detail each software layer, the communication protocols, the motion-planning algorithms, and the graphical user interfaces developed to coordinate the painting workflow.

7.1 PLC–MATLAB integration and supervisory control architecture

This section describes the role of the PLC and MATLAB in the developed control system, as well as the communication between them. The PLC is responsible for process sequencing, safety interlocks, and direct interaction with industrial devices, while MATLAB is used for high-level control, trajectory generation, data processing, and visualization. Together, these two parts form a coordinated control architecture for the multirobot painting system.

7.1.1 Software architecture overview

The control software was divided into three functional levels to make the system easier to control, test, and maintain. Each level has its own role in the operation of the multirobot painting system. The general software structure is shown in Figure 7.1.

1. Supervisory / HMI tier -MATLAB scripts and GUIs running on the operator PC.
2. Motion tier - MATLAB functions that compute trajectories and dispatch commands to physical hardware.
3. Device tier - firmware and interpreter software resident on each robotic controller (KRC4 for KUKA, ESP32 firmware for RoArm M2s, and TIA Portal project for the PLC).

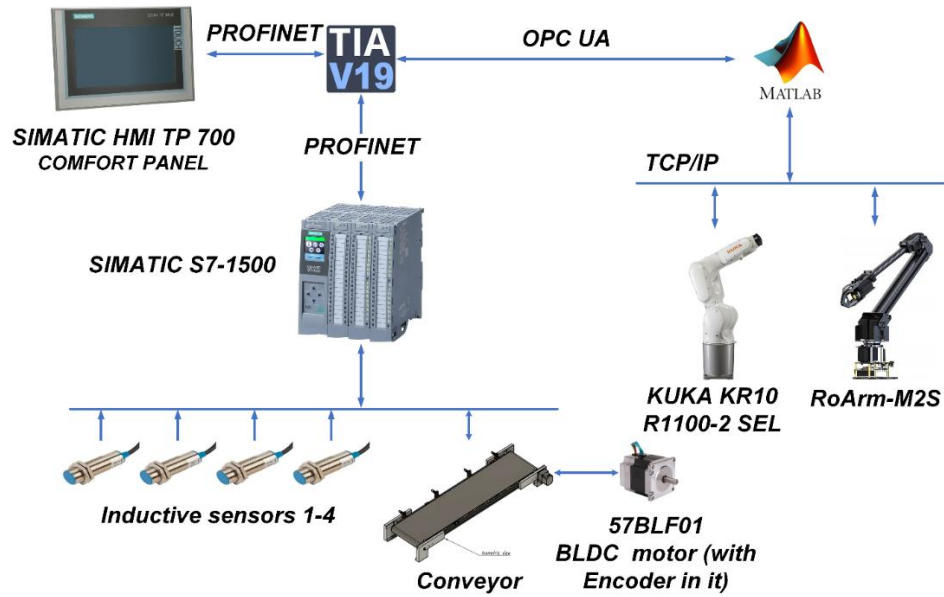


Figure 7.1 - Software Architecture

Communication between tiers is entirely network-based, which decouples the MATLAB host from the individual device implementations and allows hardware to be reconfigured without rewriting the control logic.

7.1.2 PLC-MATLAB communication and safety handshake

The PLC exposes a set of binary output signals (digital I/O) that serve as safe-to-proceed handshakes to the MATLAB supervisory layer. Before each painting stroke, MATLAB reads the PLC's "zone-clear" flag. The flag is set by the PLC only when all physical safety zones are unoccupied, all E-stop circuits are healthy, and the paint supply pressure is within tolerance. MATLAB holds the robot at the current waypoint until the flag is asserted, then releases the streaming loop. If an E-stop is triggered mid-stroke, the PLC drives the robot controllers into a hold state via their digital inputs, and a fault code is latched in the HMI alarm log. From table 7.1 can be seen environment development for each software and hardware.

Table 7.1 - Development Environment

Component	Tool / Version
1	2
Host control software	MATLAB R2025b
KUKA middleware	KukaVarProxy(open-source, port 7000)
KUKA programming	KRL on KRC4 Compact
RoArm firmware	Waveshare ESP32 HTTP server
PLC programming	Siemens TIA Portal
MATLAB toolboxes	Instrument Control Toolbox, Optimization Toolbox

After defining the development environment, the next step is to describe how these tools interact during the operation of the system. Each software component has a specific function, but all of them work together to provide synchronized control of the manipulators, conveyor, PLC and HMI.

7.1.3 System workflow and execution sequence

The complete painting cycle is organized as a step-by-step sequence. This sequence defines how the devices are initialized, how the trajectories are loaded, and how the robots, PLC, and auxiliary equipment work together during the process:

1. Initialization. The operator starts the MATLAB supervisory script, which checks network reachability of all devices (KUKA at 172.31.17.101, Arm 1 at 172.31.17.193, Arm 2 at 172.31.17.102, PLC via its PROFINET/Ethernet interface). All devices are commanded to their home positions.

2. Trajectory loading. The operator loads or generates a pick-and-place trajectory (.mat file) for the KUKA and stroke waypoint lists for each RoArm through their respective GUIs.

3. PLC handshake. MATLAB polls the PLC zone-clear flag. On assertion, it enables the robot control loops.

4. Concurrent execution. The KUKA streams its dense waypoints at 83 Hz over TCP/KVP. Each RoArm receives sequential HTTP commands at ≈ 20 Hz. The PLC continuously monitors safety zones and process variables, raising a halt if any constraint is violated.

5. Completion and logging. Upon completion of all strokes, each robot is returned to home. Execution logs are saved to .mat files and the PLC alarm log is archived via TIA Portal diagnostics.

This sequence provides a clear and repeatable workflow for the automated painting process. It also ensures that all robots operate in a coordinated way, while the PLC continuously supervises safety and process conditions.

7.2 MATLAB-based pick and place process using KUKA KR10 R1100-2

This section describes the software implementation of the pick-and-place process using the industrial KUKA KR 10 R1100-2 manipulator. The focus is on the robot connection, trajectory generation, motion execution, and data exchange between MATLAB and the KUKA controller.

7.2.1 Network and middleware setup

The KUKA KR 10 R1100-2 (Agilus series, KRC4 Compact controller) resides on the local LAN. The controller's KLI interface (Virtual Network 5) is configured either via DHCP or with a static IPv4 address, for example 172.31.17.101, with subnet mask 255.255.255.0. The open-source KukaVarProxy (KVP) application is installed on the controller's embedded Windows environment and listens on TCP

port 7000. KVP exposes the robot's global KRL variable table over a simple binary protocol, allowing an external client to read and write any declared global variable.

Two KRL files reside in the robot controller such as MatLabControl.dat and MatLabControl.src. MatLabControl.dat declares the global variables used as the communication interface.

Following this is Pseudo Code:

```
DECL GLOBAL POS target_pos = {X 509.876, Y 339.917, Z 552.366, A 0.0,
B 90.0, C 0.0}
```

```
DECL GLOBAL BOOL move_trigger = TRUE
```

MatLabControl.src - an infinite polling loop that monitors move_trigger. When the flag transitions to TRUE, the robot executes a linear motion (LIN) to target_pos at the programmed velocity, then resets the flag.

This handshaking scheme write target, pulse trigger, robot moves and gives MATLAB synchronous flow control at the cost of one round-trip latency per waypoint, which is acceptable for painting-stroke motion where inter-waypoint intervals are 12 ms or longer.

7.2.2 TCP/IP Communication protocol implementation

To send motion commands from MATLAB to the KUKA controller, a TCP/IP connection is established through KukaVarProxy. MATLAB connects to KVP using the built-in tcpclient object from the Instrument Control Toolbox:

```
MATLAB tcp = tcpclient(ROBOT_IP, 7000, 'Timeout', 10);
```

This connection allows MATLAB to read and write global KRL variables during the execution of the program. In this table 7.2, can be shown the KVP binary frame format, that have been used.

Table 7.2 – the KVP binary frame format

Field	Width	Description
1	2	3
Message ID	2 B (big-endian)	Arbitrary per-call identifier
Content length	2 B (big-endian)	Byte count of the remaining content
Mode	1 B	0x00 = read, 0x01 = write
Variable name length	2 B (big-endian)	Length of the name string
Variable name	N B	ASCII name of the KRL global
Variable value length	2 B (big-endian)	Length of the value string (write only)
Variable value	M B	ASCII representation of the value

The helper functions writeVar and readVar encapsulate frame construction, big-endian byte swapping (swapbytes/typecast), and non-blocking response parsing with a 50 ms spin-wait timeout. All byte arrays are assembled as row vectors of uint8 and dispatched in a single write(tcp, msg) call to minimize TCP fragmentation.

A reply frame carries a three-byte trailer [0 1 1] to indicate a successful write. If the trailer is absent or the socket times out, the helper returns false and the calling loop logs a warning without aborting execution, providing graceful degradation during transient network interruptions.

7.2.3 Pick and place trajectory generation

Trajectory creation is handled by `create_pick_and_place_trajectory.m`. The operator selects a robot model (KR6 R900, KR10 R1100, or KR16 R1610), which establishes the maximum workspace radius. Key Cartesian poses home, approach-pick, pick, lift-pick, intermediate, transition, approach-place, place, lift-place, and return-home are parameterized analytically to avoid known singularity configurations.

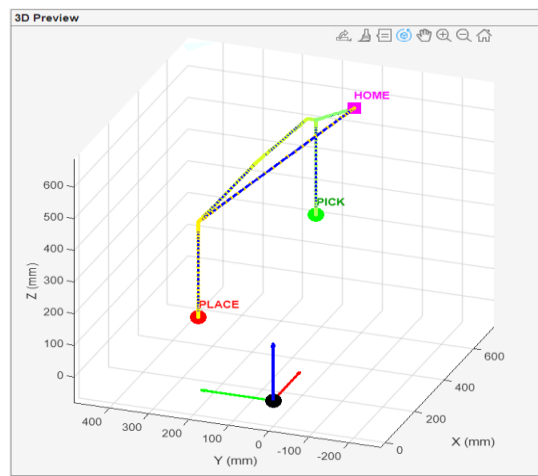


Figure 7.2 – 3D Preview Pick and Process

Singularity avoidance. The KUKA KR 10 R1100-2 exhibits a wrist singularity (A4/A5 coupling) when the tool orientation expressed in the robot-native ZYZ Euler convention has a large B angle near $\pm 180^\circ$. To prevent this, the convention adopted throughout the software Pseudo Code is:

$$A = -180^\circ, B = 2^\circ, C = -180^\circ \text{ (small } B, \text{ tool pointing downward)}$$

An intermediate waypoint displaced in Y between the pick and transition locations was introduced to avoid the A4 axis approaching its $\pm 170^\circ$ software limit during the lateral transfer motion.

Dense waypoint interpolation. Between successive key poses, dense waypoints are generated using a quintic smoothstep (C2 smooth) blending function:

$$s_5(\lambda) = 6\lambda^5 - 15\lambda^4 + 10\lambda^3, \lambda \in [0,1] \quad (7.1)$$

This ensures zero velocity and zero acceleration at both ends of each segment while maintaining position continuity across the full trajectory. With 40 points per segment and 9 segments, a full pick-and-place cycle produces approximately 351

waypoints. Workspace validation checks that no waypoint exceeds 90% of the robot's maximum reach, and an approximate joint limit check flags any waypoints where A1 approaches $\pm 170^\circ$ or a wrist-singularity B angle is detected.

7.2.4 Real time trajectory execution and monitoring

The file `execute_pick_and_place_trajectory.m` implements a real-time trajectory execution loop. During this process, MATLAB sends one trajectory waypoint to the KUKA controller every 12 ms, which is approximately 83 Hz. This update rate allows the manipulator to move smoothly between points and perform the pick-and-place operation without sharp motion changes:

1. A safety checklist is printed and the operator must type YES to proceed.
2. The TCP connection is established and an initial S/T configuration read is performed by parsing the `$POS_ACT` variable.
3. Each waypoint is encoded as a KRL POS string (e.g., `{X 600.000,Y 100.000,Z 400.000,A -180.000,B 2.000,C -180.000}`) and dispatched via `writeVar(tcp, 'target_pos', posStr)`, immediately followed by a `move_trigger pulse`.
4. A4 and A5 joint values are polled every 0.3 s by reading `$AXIS_ACT`. If either axis exceeds its guard threshold ($|A4| > 170^\circ$, $|A5| > 110^\circ$), execution is suspended, a second reading is taken 80 ms later, and if the violation persists the loop aborts and the robot is returned to the home pose.
5. Timing is enforced by a spin-wait on tic/toc rather than `pause()`, providing sub-millisecond jitter.
6. A live MATLAB figure updates every 8 steps, showing the executed 3D path, Cartesian position and orientation signals, and A4/A5 guard status.

On completion, the execution log (executed positions, joint guard readings, total time, and success count) is saved to a *timestamped .mat* file for post-process analysis.

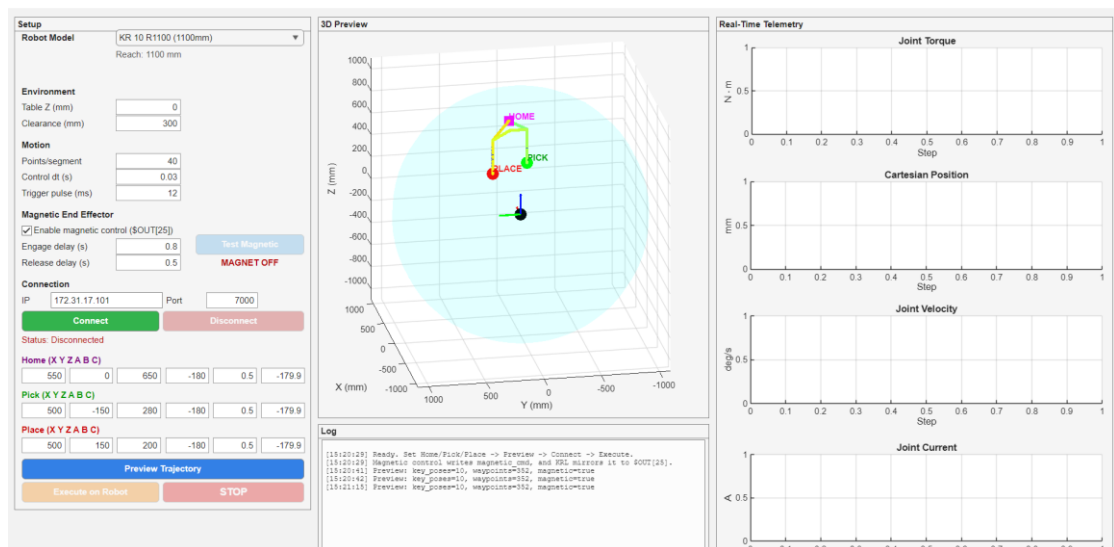


Figure 7.3 – GUI Pick and Place interface

kuka_pick_and_place_gui.m provides an App-Designer–style graphical front-end that encapsulates the trajectory creation and execution workflow. Operators specify pick and place coordinates via sliders and numeric fields, select the robot model from a drop-down list, preview the trajectory in a 3D axes object, and start or abort execution without interacting with the command line.

7.2.5 Real time telemetry and end-effector feedback

Real-time telemetry data can be received from the KUKA manipulator during operation. This data includes joint torque, Cartesian position, joint velocity, and joint current for each robot axis. These parameters are used for operator monitoring and for additional safety control. For example, the PLC can use this information to determine whether the robot is moving normally, stopped, or in an abnormal state.

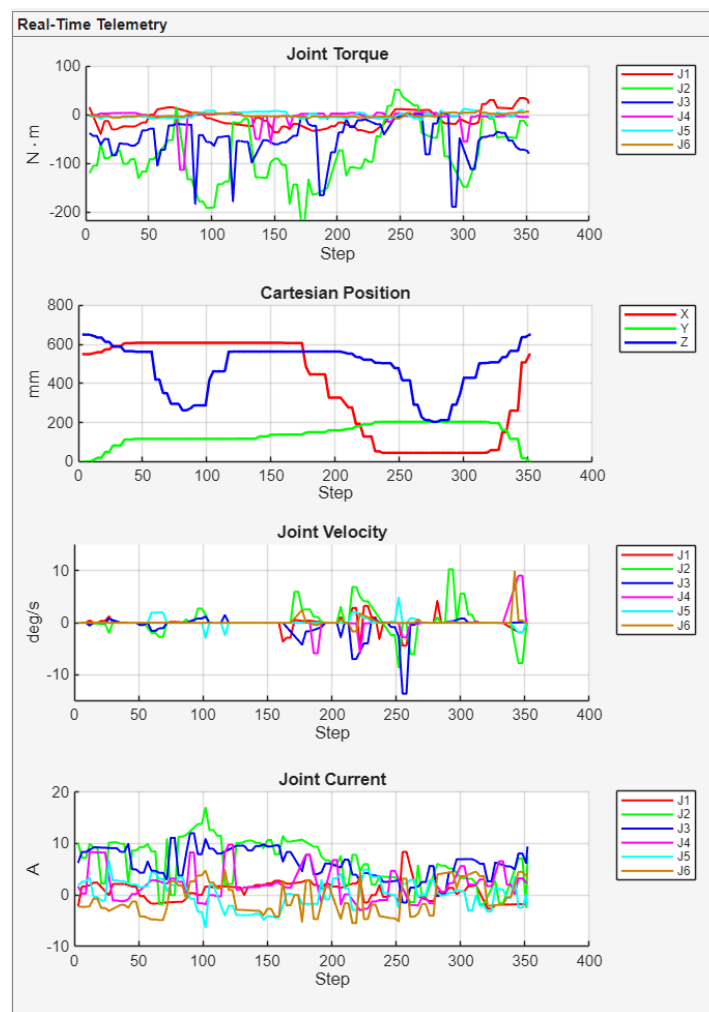


Figure 7.4 – Real Time Telemetry data from Kuka manipulator

According to standard coordinate visualization practice, the X, Y, and Z axes are marked in red, green, and blue, respectively. This makes the robot position easier to understand during monitoring and debugging.

For the transportation process, a magnetic end effector is used. When the manipulator reaches the pick position, MATLAB sends a command to activate the magnetic gripper. This command is written to the global KUKA variable *magnetic_cmd*. After that, the KUKA program assigns this value to digital output *\$OUT[25]*, which is connected to the magnetic gripper:

```
DECL magnetic_cmd = TRUE
```

```
$OUT[25] = magnetic_cmd
```

In this way, MATLAB controls the gripper through the KUKA controller, while the PLC and operator can monitor the process through telemetry and digital signals.

7.3. MATLAB-based painting process using RoArm M2s manipulators

This section describes the software implementation of the painting process in MATLAB. In this process, two RoArm-M2-S manipulators are used as collaborative robotic arms for applying the painting motion. The main focus is on their network configuration, command structure, trajectory generation, and coordinated operation during the painting cycle.

7.3.1 Hardware and network configuration

The Waveshare RoArm-M2-S is a 4-DOF robotic arm (Base, Shoulder, Elbow, End Effector) with an ESP32 microcontroller running a built-in HTTP server. Its Wi-Fi can operate in two modes:

1. AP mode (factory default): each arm creates a standalone access point (*SSID RoArm-M2, password 12345678, IP 192.168.4.1*).
2. STA mode (recommended for multi-arm deployments): the arm connects to an existing router and receives a DHCP-assigned IP address displayed on its OLED panel.

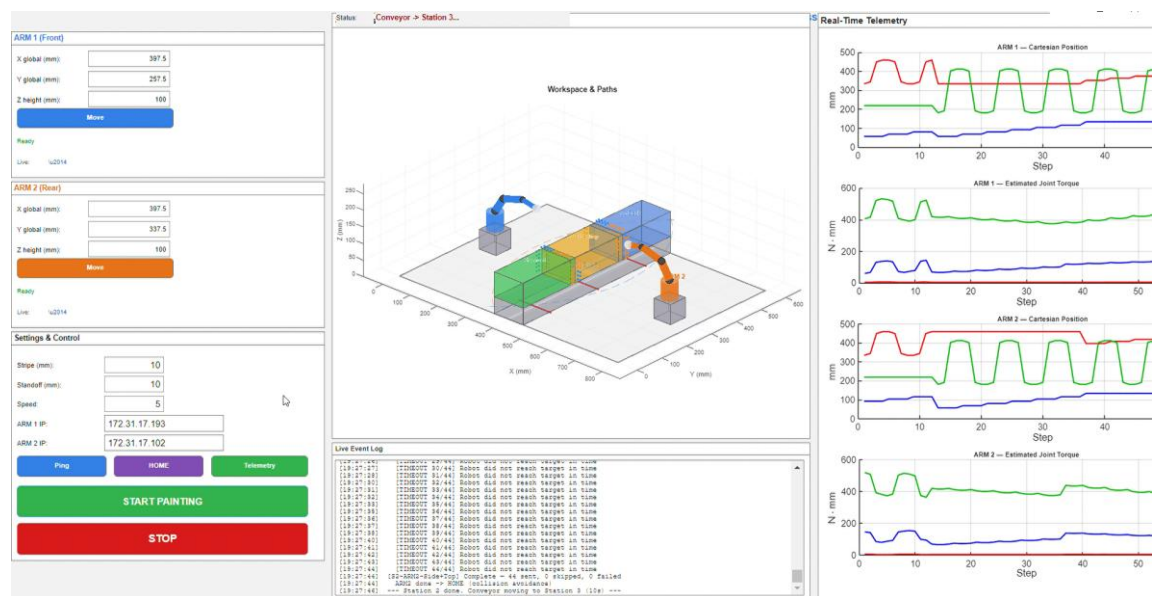


Figure 7.5 – GUI painting process interface

For the painting system, STA mode is mandatory so that both arms and the MATLAB host share a single subnet (e.g., 172.31.17.x), allowing simultaneous independent control. Arm 1 is assigned 172.31.17.193 and Arm 2 172.31.17.102.

7.3.2 HTTP/JSON communication protocol

RoArm manipulators are controlled through HTTP commands sent from MATLAB over the Wi-Fi network. Each command is sent as an HTTP GET request to the following endpoint:

http://<ARM_IP>/js?json=<URL-encoded JSON>

Here, <ARM_IP> represents the IP address of the selected RoArm manipulator. The command itself is written in JSON format. The main field in this JSON structure is the numeric identifier T, which defines the type of command executed by the robot firmware. Additional parameters depend on the selected command type, such as joint angle, Cartesian position, speed, or acceleration.

Table 7.3 – the key commands used in the painting system

T value	Command	Parameters
100	CMD_MOVE_INIT	(none) go to factory home
121	CMD_SINGLE_JOINT_ANGLE	joint (1- 4), angle (°), spd, acc
122	CMD_JOINTS_ANGLE_CTRL	b, s, e, h (°), spd, acc
104	CMD_XYZT_CTRL	x, y, z (mm), t (°), spd
1041	CMD_XYZT_DIRECT_CTRL	x, y, z (mm), t (rad) - non-blocking
105	CMD_SERVO_RAD_FEEDBACK	(none) - returns full state JSON
106	CMD_EOAT_HAND_CTRL	cmd (rad), spd, acc
210	CMD_TORQUE_CTRL	cmd (0=release, 1=lock)

MATLAB sends commands using webread with a *weboptions('Timeout', 2.5)* object. A retry wrapper (send_json_command) attempts up to two transmissions separated by 200 ms before issuing a non-fatal warning, preserving system continuity if a packet is lost.

7.3.3 Object oriented MATLAB control classes

The control of the RoArm M2s manipulators in MATLAB is implemented using object-oriented programming. Two MATLAB classes are developed to encapsulate the communication and motion control logic of the robot.

The first class, RoArm M2s_MotionControl, is a serial communication variant that is used as a fallback or legacy solution. It operates through the MATLAB serial object with a baud rate of 115200 and sends control commands in JSON format via the UART interface.

In this implementation, joint limits are checked during each method call to ensure safe operation of the manipulator. Forward kinematics are calculated using

the Denavit–Hartenberg (DH) parameter model, where joint angles are defined in radians. Inverse kinematics are solved using MATLAB’s `fmincon` optimization function, with joint limits applied as constraints to guarantee physically valid solutions. RoarmM2_MotionControl_WiFi primary variant. Communicates via webread over HTTP. Methods include:

moveHome() sends $T=100$ and resets the internal joint state to $[0, 0, 90, 180]^\circ$.

moveToJointAngles(angles) validates limits and dispatches $T=122$.

moveSingleJoint(joint, angle) validates single-axis limits and dispatches $T=121$.

moveToCartesian(position, handAngle) converts the hand angle to radians and dispatches $T=1041$ (direct, non-blocking).

controlEndEffector(angle) converts degrees to radians and dispatches $T=106$.

getFeedback() dispatches $T=105$ and prints the JSON state response.

setTorque(state) locks or releases joint servos via $T=210$.

Joint limits enforced throughout: Base $\in [-180^\circ, 180^\circ]$, Shoulder $\in [-90^\circ, 90^\circ]$, Elbow $\in [-45^\circ, 180^\circ]$, End Effector $\in [45^\circ, 315^\circ]$.

7.3.4 Painting trajectory generation and execution

For the painting application, each RoArm executes a series of Cartesian strokes. The *go_to_point.m* utility provides interactive terminal control: the operator types $[x\ y\ z]$ coordinates in millimetres, and the script constructs a $T=104$ JSON command and delivers it via *send_json_command*. This mode is used for operator-guided commissioning and for debugging new painting patterns.

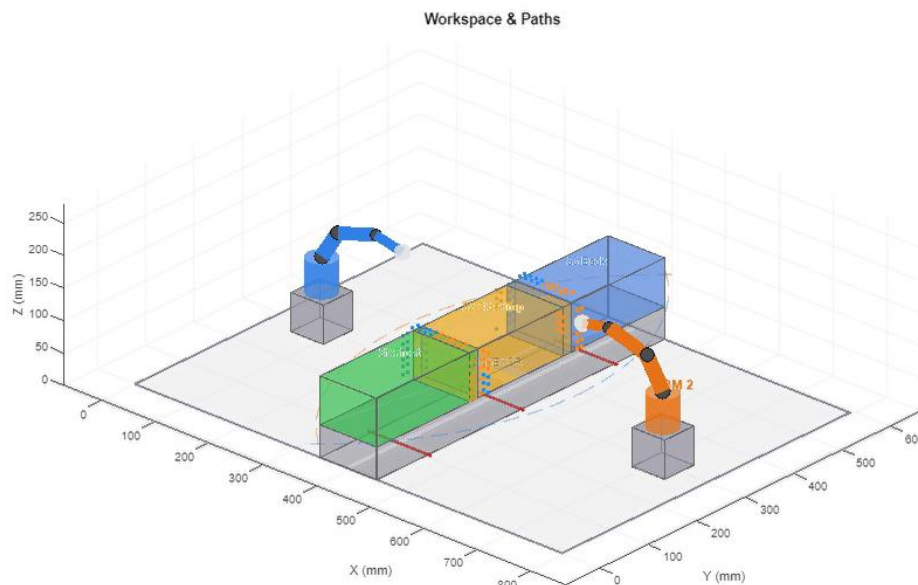


Figure 6.6 – 3D preview

For autonomous painting operation, the file *roarms_painting_process_gui.m* implements a MATLAB graphical application. This interface allows the operator to

define painting stroke waypoints, preview the planned trajectory, and send the motion sequence to one or both RoArm manipulators.

To provide smooth motion between waypoints, an S-curve trajectory profile based on a quintic polynomial is used:

$$q(t) = q_i + (q_f - q_i) \left[10 \left(\frac{t}{T} \right)^3 - 15 \left(\frac{t}{T} \right)^4 + 6 \left(\frac{t}{T} \right)^5 \right] \quad (6.2)$$

This interpolation method reduces sharp changes in motion and limits jerk during transitions between painting strokes. A 50 ms sampling interval is used between waypoints, which provides a balance between HTTP communication delay and smooth robot movement.

7.4 Industrial configuration of the controller

This subsection describes the integration architecture used to connect all the main components of an automated paint line. The Siemens S7-1500 will act as a PLC with DI (6ES7521-1BL00-0AB0) DQ (6ES7522-1BL01-0AB0), AI (6ES7531-7KF00-0AB0) modules and a PSU(6EP1332-4BA00) providing the necessary power for the PLC. The interaction between the PLC, control software, manipulators, conveyor system and auxiliary equipment is carried out using standardized industrial protocols and deterministic input/output channels. The goal of this level of integration is to ensure the synchronized, secure, and logically consistent operation of the entire production cycle.

7.4.1 Integration of conveyor and fan

The conveyor system is directly connected to the Siemens S7-1500 PLC through standard digital inputs, digital outputs and encoder feedback channels. The proximity sensors installed along the conveyor send digital signals to the PLC, allowing the controller to determine when a workpiece arrives at critical positions such as the painting area, the drying zone or the inspection point. At the same time, the motor encoder provides continuous feedback on belt movement, including speed and indexing distance. This feedback enables the PLC to stop the conveyor with high positional accuracy and ensures that both the RoArm M2S painting manipulator and the KUKA pick-and-place robot receive the workpiece at the correct moment. Through the combination of digital sensor signals and encoder data, the PLC supervises all conveyor movements, synchronizes them with robot operations, and guarantees stable and reliable material flow throughout the production cycle.

The drying fan unit is controlled by the PLC through a dedicated digital output channel. This output activates an external relay, which in turn supplies power to the fan motor. The PLC starts the drying fan when the painted part enters the drying zone and maintains airflow for the required duration of the technological process. Once drying is complete, the PLC deactivates the relay, cutting power to the fan

before the conveyor resumes movement. This method provides a simple and robust interface, ensures electrical isolation between the control logic and the power circuit, and guarantees that drying is consistently executed in coordination with all other subsystems of the automated line.

7.5 Control system development in TIA Portal

The automated painting line control program is developed in Siemens TIA Portal V19 and executed on the central Siemens S7-1500 PLC. This part of the software implementation transforms the industrial configuration described in the previous subsection into a complete PLC-based control system. The program coordinates all major subsystems of the automated line, including the conveyor motor, RoArm M2S painting manipulators, the drying zone fan and the KUKA pick-and-place robot.

The PLC operates as the main sequencer of the system. It generates start and stop commands, processes sensor feedback, monitors the current state of the production cycle and applies interlock conditions between different subsystems. The control logic ensures that the conveyor, manipulators, drying unit and pick-and-place robot operate in the correct technological order. For example, the conveyor is allowed to move only when the previous operation is completed, the painting manipulators are activated only when the workpiece reaches the painting position, and the KUKA robot receives a pick command only after the drying stage is completed.

Several operating modes are implemented in the PLC program. Automatic mode is used for continuous cyclic operation of the painting line. Manual mode allows the operator to test individual mechanisms, such as the conveyor, fan and robot interface signals, during commissioning or maintenance. Stop mode safely interrupts the current cycle, while Emergency Stop immediately disables all actuator commands and places the system into a safe state. The transition between these modes is controlled by the PLC logic to prevent unsafe or contradictory commands..

7.5.1 Program variables and tags

Control system variables form the interface between the physical process, the PLC logic and the HMI. In the TIA Portal project, all signals are organized in structured data blocks and symbolic tags that clearly separate conveyor, painting, drying and pick-and-place functions. Digital variables include start/stop commands, mode selection bits, limit switch states, proximity sensor inputs and interlock flags for safe motion. Analog and integer values represent conveyor speed setpoints, encoder-derived position or speed feedback, timing parameters and counters for cycle statistics. The primary variable groups are listed in the tag table shown in Figure 7.7.

Name ▲	Data type	Address	...	...	...	Supervision	Comment
ALM_XA101_CV101	Bool	%M24.0	☐	☒	☒		ConveyorFault
ALM_XA102_FN101	Bool	%M24.3	☐	☒	☒		FanFault
ALM_XA103_RB101	Bool	%M24.1	☐	☒	☒		Spray ManipulatorFault
ALM_XA104_RB102	Bool	%M24.2	☐	☒	☒		Pick & Place ManipulatorFault
CV-101	Bool	%Q4.0	☐	☒	☒		Conveyor
ES101	Bool	%I0.4	☐	☒	☒		emergency button
FN-101	Bool	%M11.3	☐	☒	☒		Drying Fan
HMI_ES101	Bool	%M6.2	☐	☒	☒		emergency button in HMI
HMI_PS101	Bool	%M6.0	☐	☒	☒		Start process button in HMI
HMI_PS102	Bool	%M6.1	☐	☒	☒		Stop process button in HMI
HMI_PS103	Bool	%M6.3	☐	☒	☒		Automatic mode button in HMI
HMI_PS104	Bool	%M6.4	☐	☒	☒		Manual mode button in HMI
PS101	Bool	%I0.0	☐	☒	☒		Start process button
PS102	Bool	%I0.1	☐	☒	☒		Stop process button
PS103	Bool	%I0.2	☐	☒	☒		Automatic mode button
PS104	Bool	%I0.3	☐	☒	☒		Manual mode button
QI-101	Int	%MW1	☐	☒	☒		Package Counter
RB-101	Bool	%Q0.0	☐	☒	☒		Spray Manipulator
RB-102	Bool	%Q0.2	☐	☒	☒		Pick & Place Manipulator
SIM_CV101	Bool	%M2...	☐	☒	☒		ConveyorBroken
SIM_FN101	Bool	%M24.7	☐	☒	☒		FanBroken
SIM_RB101	Bool	%M24.5	☐	☒	☒		Spray ManipulatorBroken
SIM_RB102	Bool	%M24.6	☐	☒	☒		Pick & Place ManipulatorBroken
ZS-101	Bool	%I29.0	☐	☒	☒		Infeed Sensor
ZS-102	Bool	%I29.1	☐	☒	☒		Spray Position Sensor
ZS-103	Bool	%I29.2	☐	☒	☒		Drying/Fan Entry Sensor
ZS-104	Bool	%I29.3	☐	☒	☒		Pick&Place Position Sensor

Figure 7.7 Main PLC tag table for system I/O signals

Figure 7.7 presents the main tag table covering all primary I/O signals, including conveyor drive commands, proximity sensor inputs, robot enable flags and mode selection bits. Each tag follows a consistent naming convention that directly reflects the physical function it represents, making the program easier to read and commission. The second group of tags, covering internal logic variables, is shown in Figure 7.8.

additional_variables							
	Name	Data type	Address ▲	Retain	Acces...	Writa...	Visibl...
1	HMI_FN101_Visible1	Bool	%Q1.1	☐	☒	☒	☒
2	HMI_FN101_Visible2	Bool	%Q1.2	☐	☒	☒	☒
3	HMI_RB102_Visible	Bool	%Q1.4	☐	☒	☒	☒
4	HMI_WP101_Painted	Bool	%Q1.6	☐	☒	☒	☒
5	System_Byte	Byte	%MB0	☐	☒	☒	☒
6	AlwaysFALSE	Bool	%M0.3	☐	☒	☒	☒
7	tag3	Bool	%M4.0	☐	☒	☒	☒
8	HMI_WP101_Visible	Bool	%M7.0	☐	☒	☒	☒
9	HMI_WP102_Visible	Bool	%M7.1	☐	☒	☒	☒
10	HMI_WP103_Visible	Bool	%M7.2	☐	☒	☒	☒
11	HMI_WP104_Visible	Bool	%M7.3	☐	☒	☒	☒
12	HMI_Main	Bool	%M7.4	☐	☒	☒	☒
13	HMI_PID	Bool	%M7.5	☐	☒	☒	☒
14	HMI_Status	Bool	%M7.6	☐	☒	☒	☒
15	Get_value1_from_Matlab	Bool	%M11.0	☐	☒	☒	☒
16	Get_value2_from_Matlab	Bool	%M11.1	☐	☒	☒	☒
17	Send_value1_to_Matlab	Bool	%M11.2	☐	☒	☒	☒
18	Send_value2_to_Matlab	Bool	%M11.4	☐	☒	☒	☒
19	check	Bool	%I2.4	☐	☒	☒	☒
20	Alarm_Collection	Int	%MW21	☐	☒	☒	☒
21	HMI_PB_RB101	Bool	%M7.7	☐	☒	☒	☒
22	HMI_PB_RB102	Bool	%M8.0	☐	☒	☒	☒
23	HMI_PB_FN101	Bool	%M8.1	☐	☒	☒	☒
24	HMI_PB_CV101	Bool	%M8.2	☐	☒	☒	☒

Figure 7.8 Additional internal and HMI-related variables

Additional internal variables shown in Figure 7.8 are used to implement state machines, edge detection, one-shot triggers and safety conditions such as "robot area clear" or "drying complete". Many of these variables are mapped directly to HMI objects so that operators can monitor their values and adjust selected parameters without modifying the underlying program. The consistent naming and grouping of variables simplify debugging, commissioning and further expansion of the system.

7.5.2 Control logic implementation

The control logic implements the technological sequence of the painting line using a combination of cyclic tasks, interlock networks and state-based control structures. In Automatic mode, the PLC waits for a workpiece detection signal at the conveyor entry, indexes the conveyor to the painting position and then issues enable commands to the RoArm M2S manipulators once the part is correctly aligned. After painting, the logic advances the conveyor into the drying zone, starts the fan for a predefined time and only then releases the part to the pick-and-place area. The KUKA robot receives a "ready for pick" signal from the PLC, performs its removal cycle and reports completion back to the controller so that the next workpiece can be admitted. The first part of the Automatic mode ladder diagram is shown in Figure 7.9.

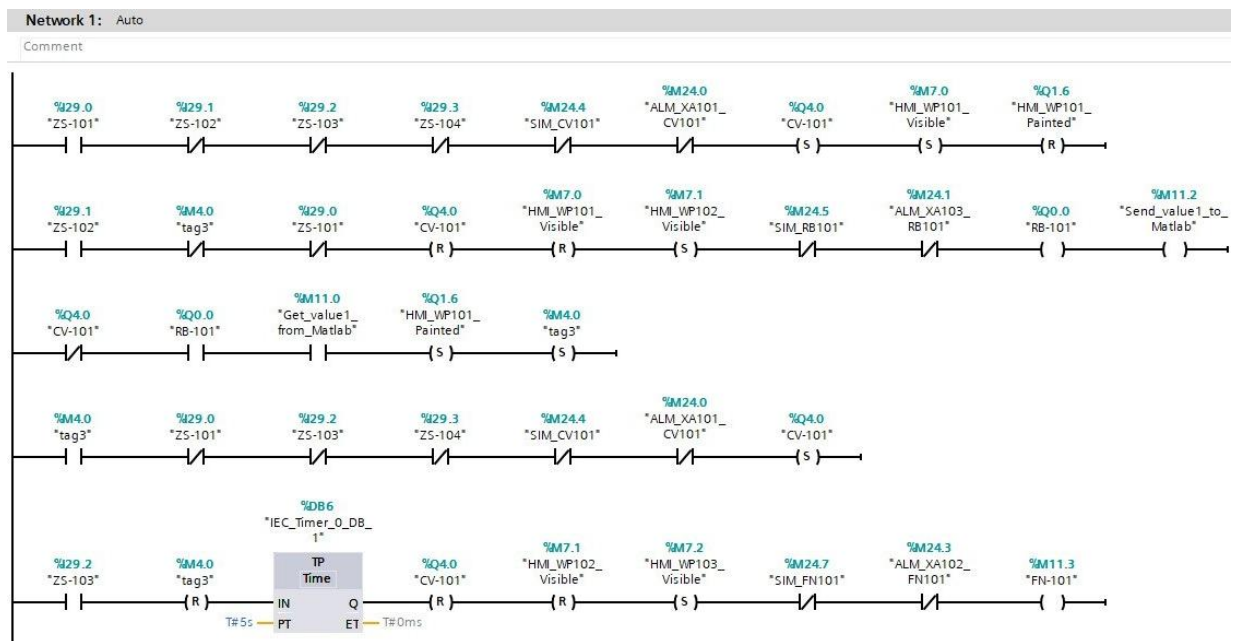


Figure 7.9 – Automatic mode logic, part 1

Figure 7.9 covers the initial networks of the Automatic mode: workpiece detection at the conveyor entry, conveyor indexing logic and the alignment check before the painting enable signal is issued. The sequence continues in Figure 6.10, which shows the second half of the Automatic mode program.

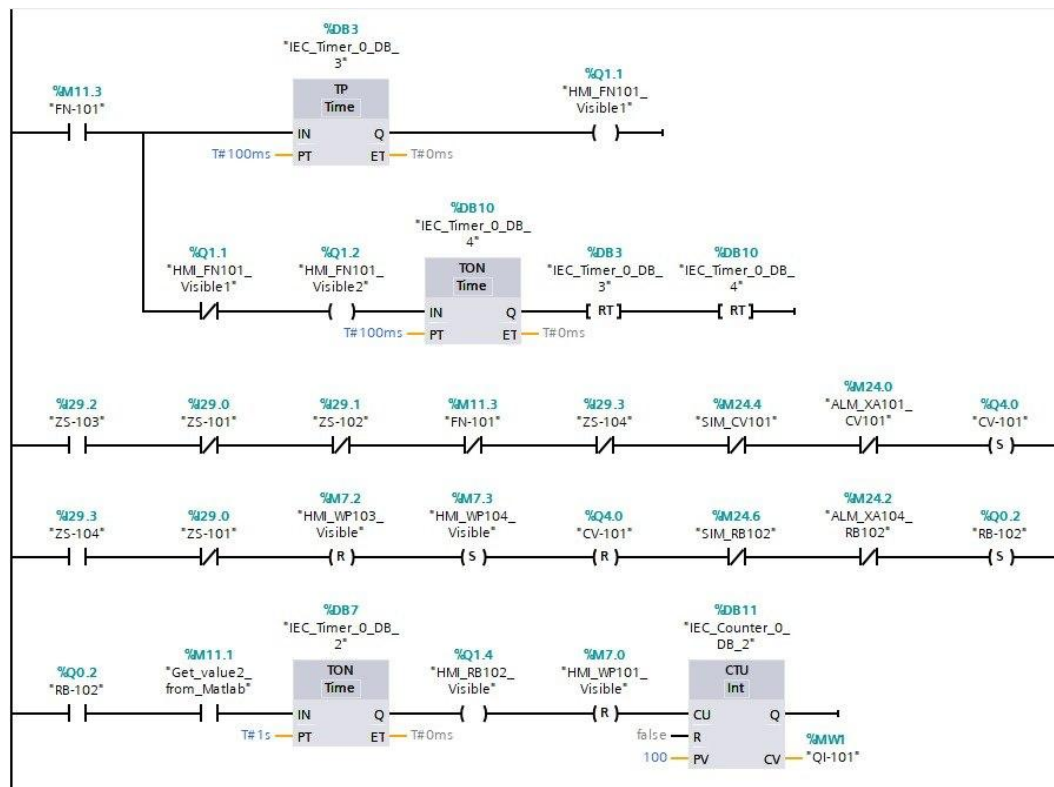


Figure 7.10 – Automatic mode logic, part 2

Figure 7.10 presents the drying zone activation, the fan timer network and the handshake logic with the KUKA pick-and-place robot. Once the robot reports cycle completion, the PLC resets the sequence and allows the next workpiece to enter the line. The Manual mode logic follows the same network structure but replaces the automatic sequencer conditions with individual operator commands. The first part of the Manual mode program is shown in Figure 7.11.

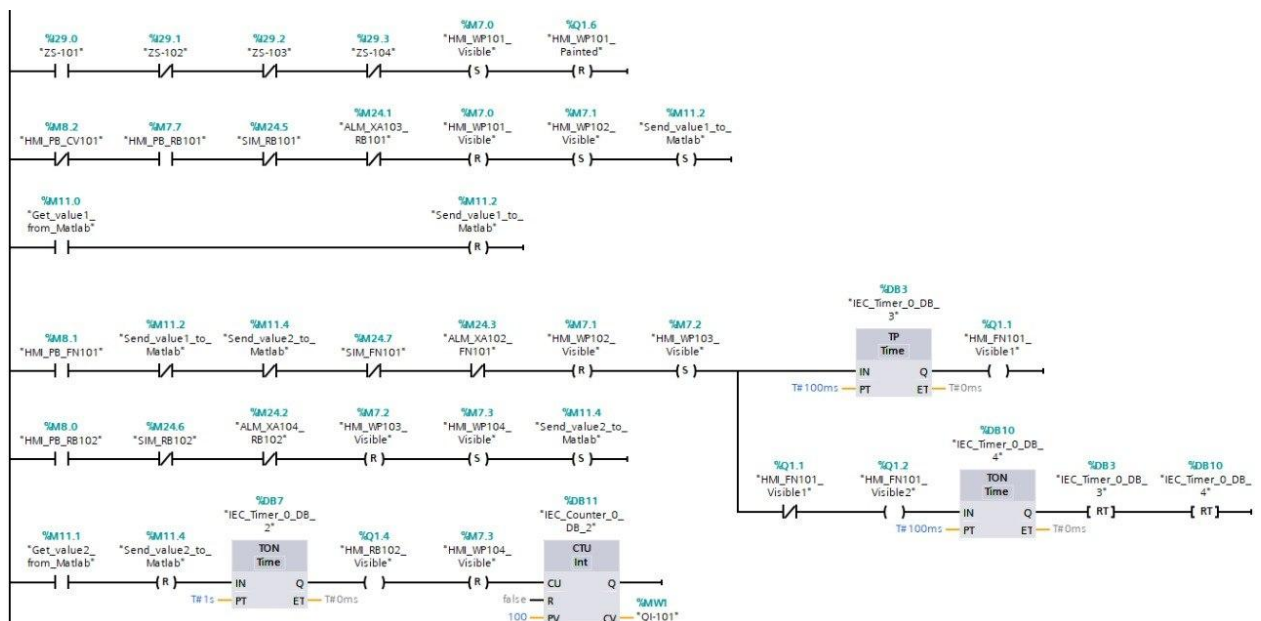


Figure 7.11 – Manual mode logic, part 1

Figure 7.11 shows the initial Manual mode networks, which enable individual device outputs when the corresponding HMI jog button is pressed and the relevant safety interlock is satisfied. Each output is protected by an additional permissive contact that prevents simultaneous activation of conflicting devices. The second part of the Manual mode program is presented in Figure 7.12.

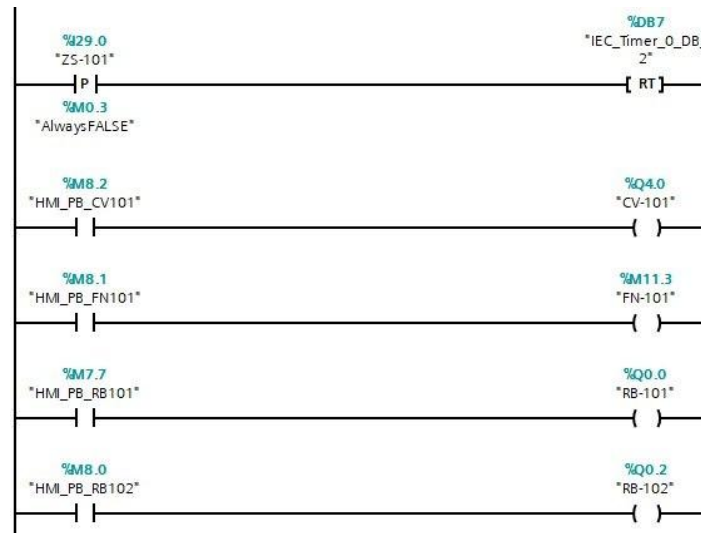


Figure 7.12 – Manual mode logic, part 2

Figure 7.12 covers the remaining Manual mode outputs, including the drying fan test command and the robot interface test signals. All Manual mode outputs are disabled as soon as the mode selector is moved to Automatic or Stop, ensuring a clean transition without residual activations. The overall main logic block, which selects the active mode and applies the global Emergency Stop override, is shown in Figure 7.13.

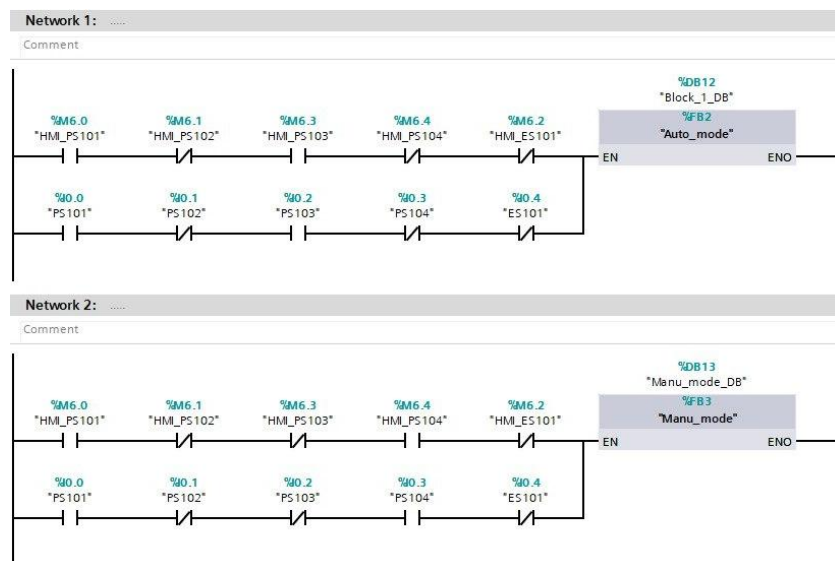


Figure 7.13 – Mode selection logic for automatic and manual operation

In Manual mode, the same logic blocks are reused but gated by manual commands and additional safety conditions, allowing operators to jog the conveyor, test robot interfaces and individually actuate devices. Stop and Emergency Stop signals override all other commands, forcing outputs to a safe state and preventing further motion until the fault is acknowledged and reset. This structured approach ensures deterministic, reproducible operation and provides a clear framework for future modifications of the technological cycle.

7.5.4 HMI visualization design

The Human Machine Interface for the control system is developed in WinCC and deployed on the operator panel installed near the cell. The main screen presents a top-level overview of the automated line, with graphical representations of the conveyor, painting area, drying zone and pick-and-place station, including color-coded status indicators for each subsystem. Dedicated mode-selection controls allow the operator to switch between Automatic, Manual and Stop states, while clearly displaying the active mode and any active safety interlocks or alarms. The overall system overview screen is shown in Figure 7.14.

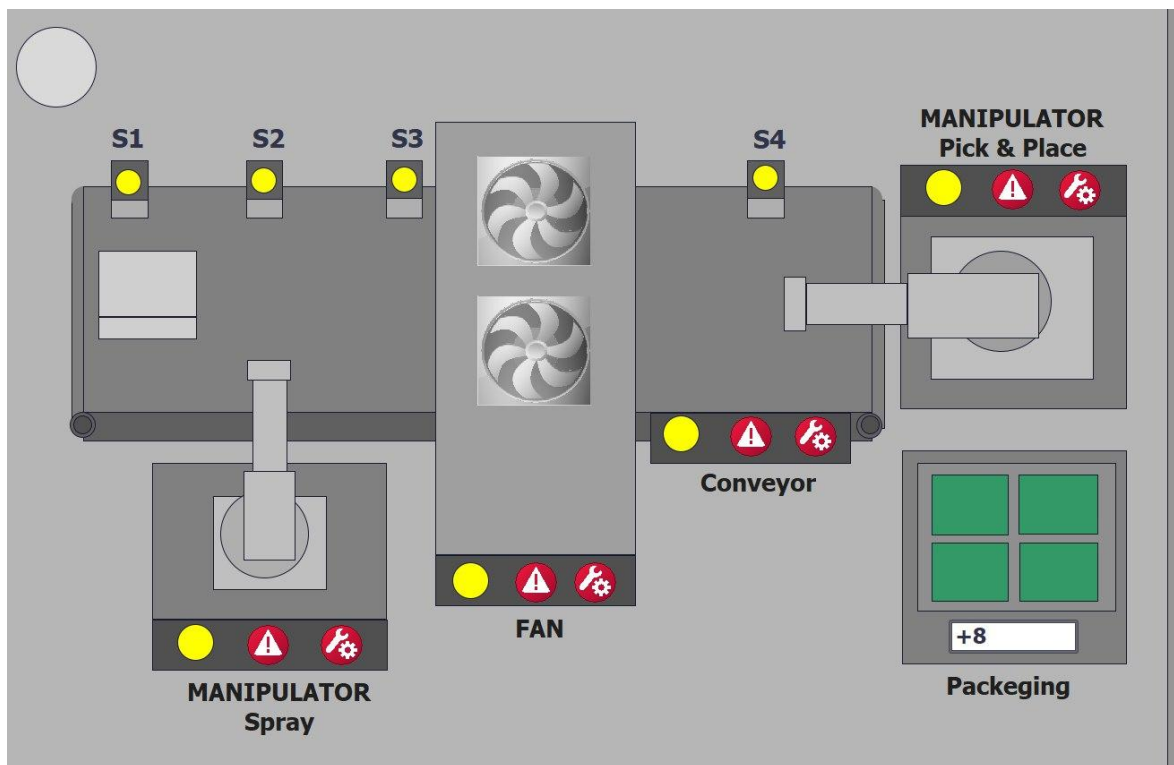


Figure 7.14 – Main HMI overview screen

Figure 7.14 shows the main HMI overview screen, where all subsystems are represented graphically and color-coded according to their current operational state: green for running and red for a stopped and fault condition. The operator control panel, which hosts the mode selector switch and the main status indicators, is shown in Figure 7.15.

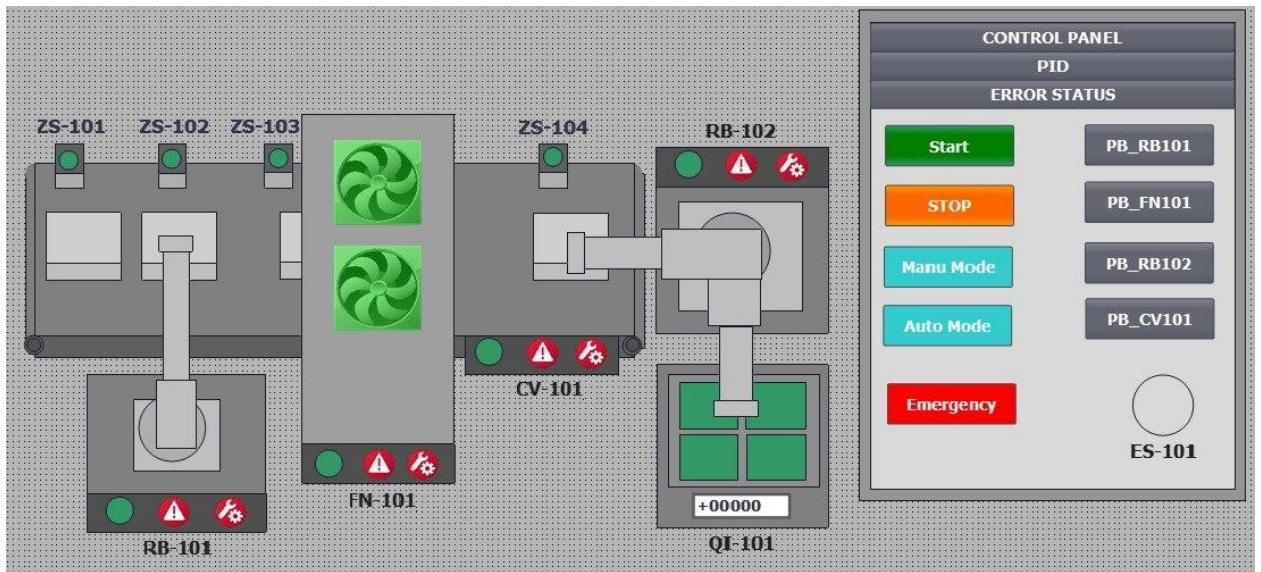


Figure 7.15 – HMI control panel for mode selection and system commands

Figure 7.15 presents the dedicated control panel area of the HMI, providing mode selection buttons, an emergency stop acknowledgement control and indicators for active safety interlocks. The full Automatic mode screen, which displays all running subsystem statuses simultaneously, is shown in Figure 7.16.

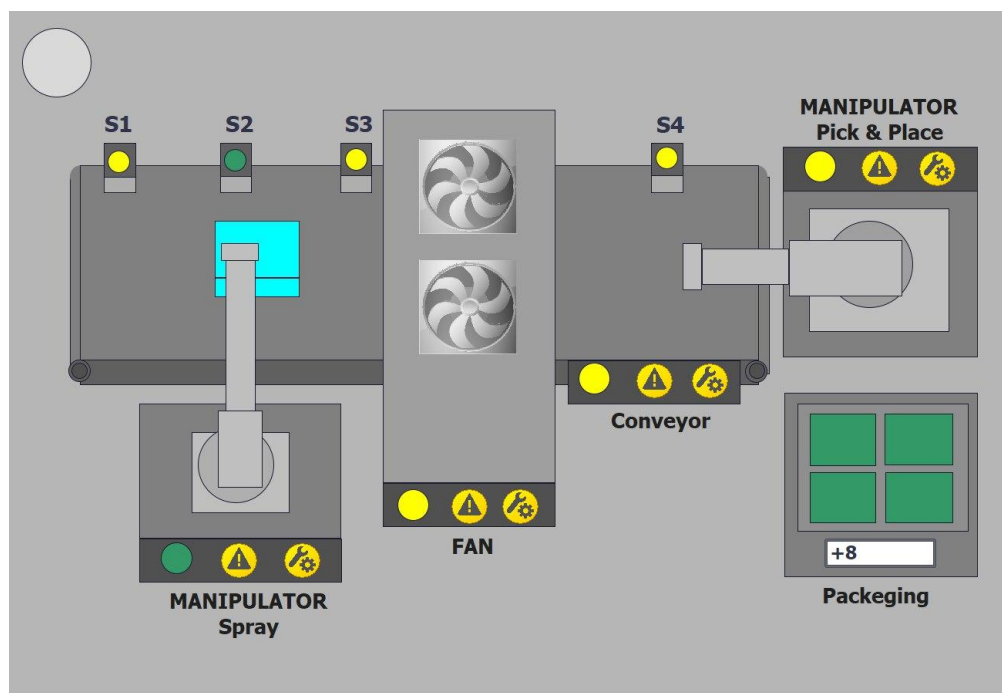


Figure 7.16 – Automatic mode HMI screen

Figure 7.16 provides a full-screen view of the Automatic mode, where the conveyor position, robot status indicators, drying timer and cycle counter are all

visible at the same time. The corresponding full view of the Manual mode screen is presented in Figure 7.17.

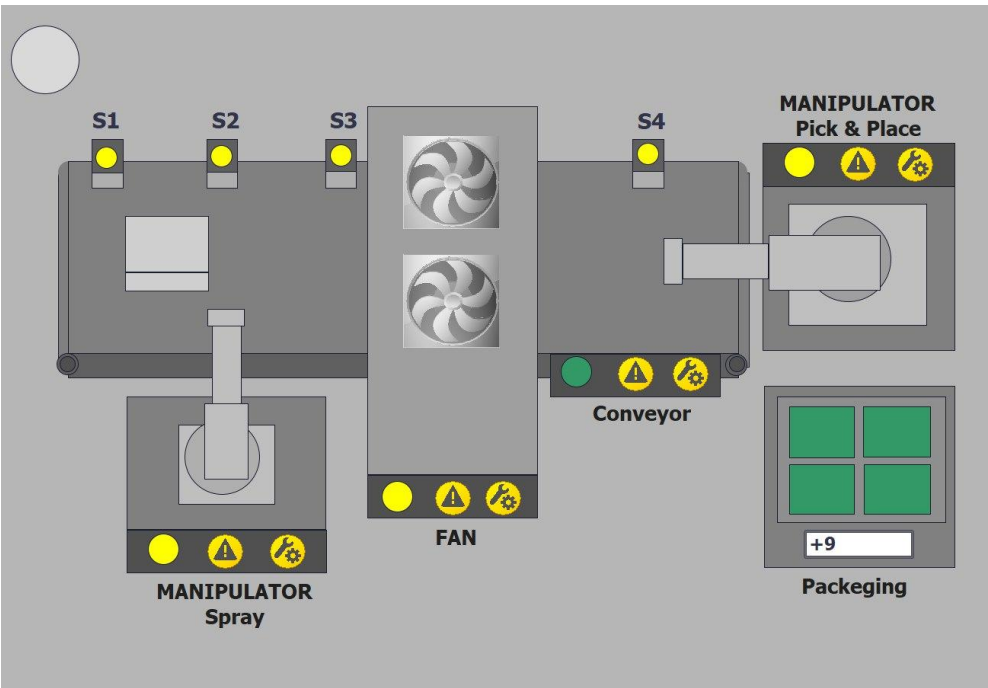


Figure 7.17 – Manual mode HMI screen

Figure 7.17 shows the Manual mode screen, where individual jog and test buttons are arranged by subsystem. Each control is visually locked when its safety interlock is not satisfied, preventing accidental activation. The alarm and fault status screen is shown in Figure 7.18.

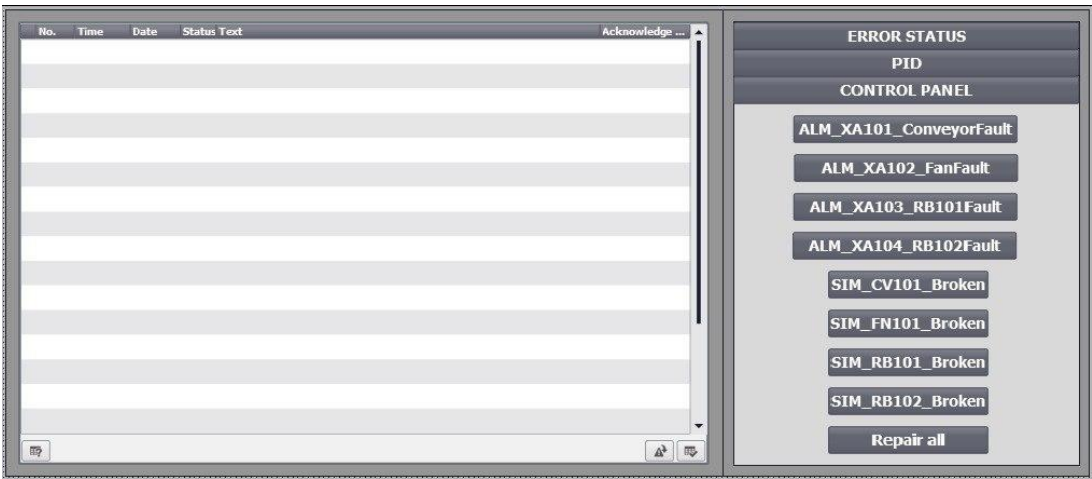


Figure 7.18 – Alarm and error status screen

Alarm and event pages record faults such as sensor errors, emergency stops or robot communication failures, helping the operator identify and resolve issues

quickly. This HMI structure ensures intuitive operation, transparent visualization of the process and safe interaction with the underlying control logic.

7.6 Motor PID programming

This subsection describes the implementation of the PID controller responsible for regulating the conveyor motor speed in the multirobot painting line. The PID algorithm was implemented inside the Siemens S7-1500 PLC using both a custom mathematical block (Logic_DB_3) and later the Siemens PID_Compact function block, which provides an integrated autotuned controller. The goal of this controller is to maintain a stable conveyor velocity corresponding to the selected setpoint, ensuring consistent paint quality and synchronized operation between the conveyor, manipulator, and inspection system.

7.6.1 PID variables

Two data blocks are created. The first block stores variables for the manual PID implementation. It contains the input value from the motor encoder, the output value forwarded to the next network, and several internal values that store previous samples of input and output. These values are used for the proportional, integral and derivative calculations. Temporary variables are also declared for intermediate arithmetic steps. The static variable structure of this data block is shown in Figure 7.19.

Logic_DB_3					
	Name	Data type	Start value	Retain	Accessible f...
1	▼ Input			☐	☐
2	■ u	Real	0.0	☐	☒
3	▼ Output			☐	☐
4	■ y	Real	0.0	☐	☒
5	■ InOut			☐	☐
6	▼ Static			☐	☐
7	■ u1	Real	0.0	☐	☒
8	■ u2	Real	0.0	☐	☒
9	■ y1	Real	0.0	☐	☒
10	■ y2	Real	0.0	☐	☒
11	■ Tmp_fb	Real	0.0	☐	☒
12	■ Tmp_u0	Real	0.0	☐	☒
13	■ Tmp_u1	Real	0.0	☐	☒
14	■ Tmp_u2	Real	0.0	☐	☒
15	■ Tmp_uSum	Real	0.0	☐	☒

Figure 7.19 – Static variables of the manual PID data block

Figure 7.19 shows the static variable declarations of the manual PID data block, including the previous error, previous output and accumulated integral sum. The second data block is created automatically by the PID_Compact function block and holds the tuning parameters and controller state. Its variable structure is shown in Figure 7.20.

PID_DB_2						
	Name	Data type	Start value	Retain	Accessible f...	Writa...
1	Input			☐	☐	☐
2	Output			☐	☐	☐
3	InOut			☐	☐	☐
4	Static			☐	☐	☐
5	SP_motor	Real	16.25163	☐	☒	☒
6	PV_motor	Real	0.0	☐	☒	☒
7	LMN_motor	Real	0.0	☐	☒	☒

Figure 7.20 – PID_Compact instance data block parameters

Figure 7.20 presents the PID_Compact instance data block, which stores the setpoint, the actual measured speed, the calculated output and internal controller state variables including error, tuning gains and operating mode flags. The Siemens PID_Compact block simplifies the control logic by performing all calculations internally. Instead of manually building the proportional, integral, and derivative stages, the PLC simply passes the setpoint and the process value into the PID_Compact block. The block calculates the output on its own, including anti-windup, output limiting, bumpless switching between modes, and tuning adjustments.

7.6.2 PID logic

The first version of the PID was written manually using standard ladder networks. In the first network the proportional and derivative parts are calculated using MUL and ADD function blocks. The second network computes the integral part and accumulates the running sum. The third network combines all three contributions to form the controller output. The fourth network shifts the stored input and output values so that the next execution cycle can calculate the derivative term correctly. The full manual PID ladder implementation is shown in Figure 7.21.

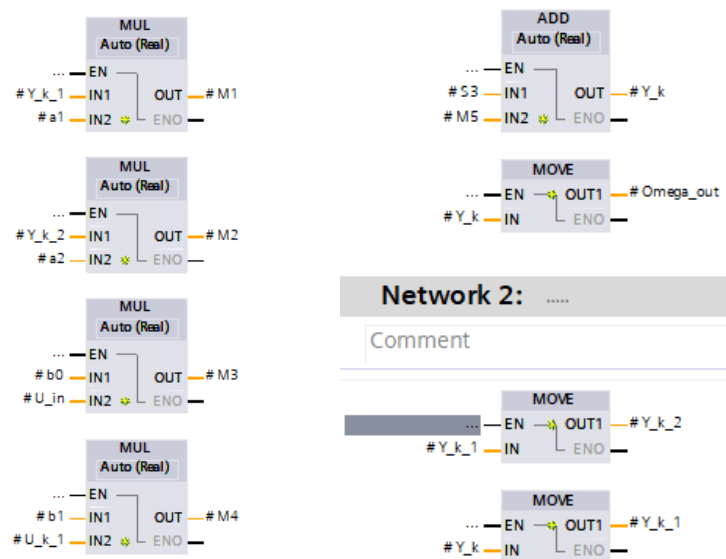


Figure 7.21 – Manual PID arithmetic networks

Figure 7.21 shows all four networks of the manual PID block, illustrating how each arithmetic operation maps directly to the discrete PID difference equation. The second implementation uses the Siemens PID_Compact function block, which encapsulates the same mathematical operations in a single certified block. The input connections of the PID_Compact block are shown in Figure 7.22.

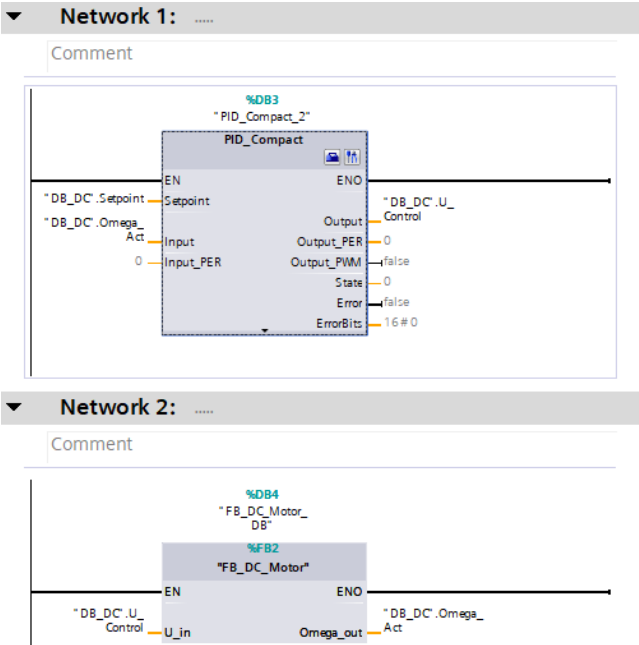


Figure 7.22 – PID_Compact function block input connections

Figure 7.22 presents the network where the setpoint from the HMI and the encoder feedback value are wired to the corresponding PID_Compact inputs. The output handling network, which routes the controller result to the motor drive, is shown in Figure 7.23.

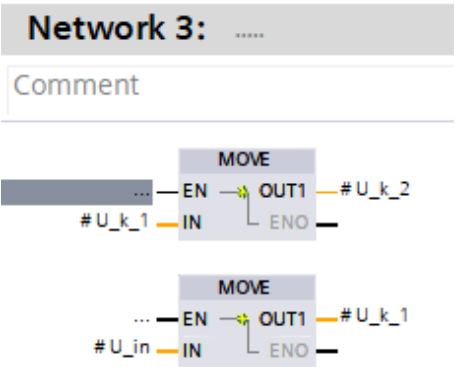


Figure 7.23 – PID_Compact output scaling and motor control network

It includes internal logic such as anti-windup, bumpless transfer, and autotuning. It is easier to use and requires fewer networks. The output of the PID_Compact block is then sent to the logic block for further handling.

7.6.3 PID's Human Machine Interface visualization

The PID controller is visualized on a dedicated HMI screen developed in WinCC Comfort. The operator can modify the speed setpoint, observe the actual measured value and read the current controller output. The screen also shows the active controller mode, whether the PID is running in automatic, manual or tuning state, and flags any internal error condition. The main PID control panel screen is shown in Figure 7.24.

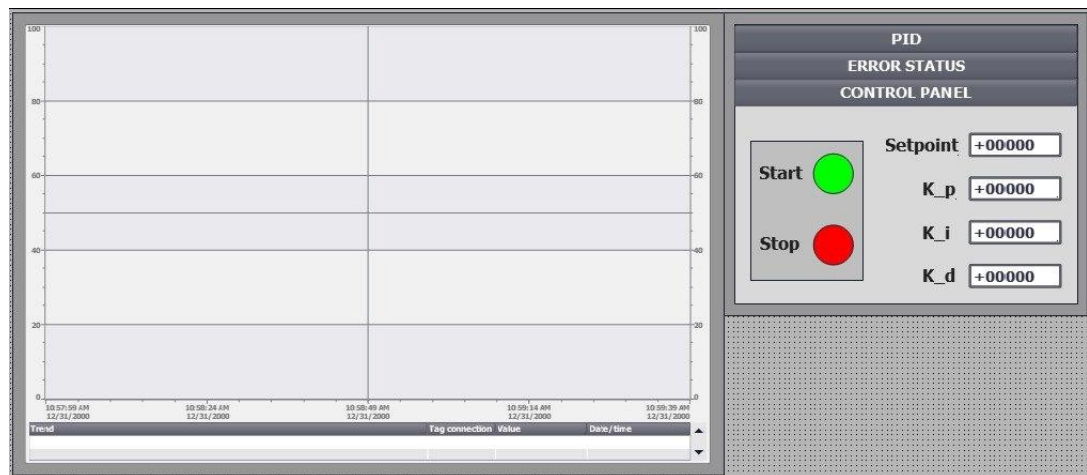


Figure 7.24 – PID HMI control panel

Figure 7.24 presents the numerical display area of the PID screen, where the setpoint, process value and output are shown as live updating fields. A trend graph is included to allow the operator to observe how these three signals evolve over time, making it straightforward to assess the quality of the speed regulation. The trend visualization is shown in Figure 7.25.

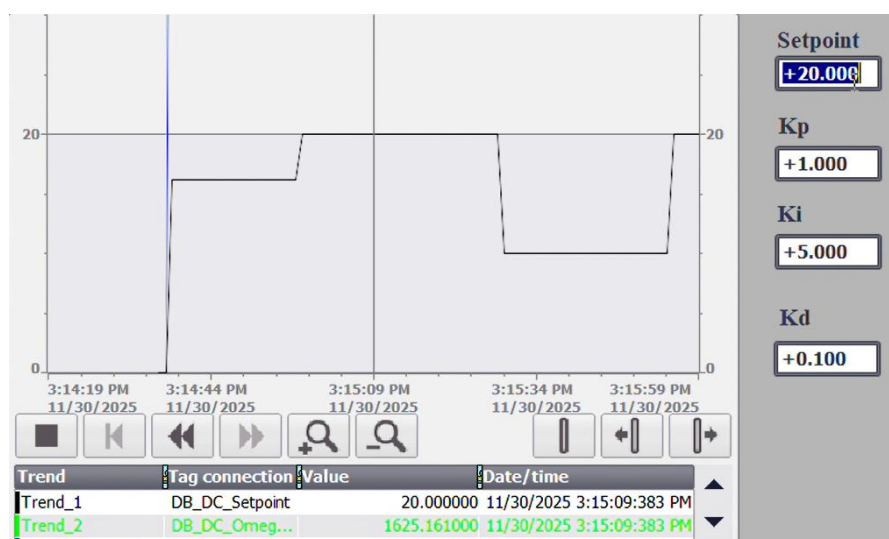


Figure 7.25 – PID trend visualization

Figure 7.25 shows the trend graph, which plots the setpoint, feedback and output signals on a shared time axis. This visualization helps the operator confirm that the conveyor speed tracks the setpoint closely and that no sustained oscillation or steady-state error is present, ensuring that the painting process remains stable and repeatable.

7.7 Real-life implementation

This chapter presents the practical implementation of the developed multi-robot painting system. The project was considered at two levels: an industrial-scale concept and a laboratory-scale realization. The industrial version shows how the system can be applied in a real automotive factory, while the laboratory version demonstrates the actual implementation using the available equipment.

Figure 7.26 shows the industrial-scale concept of the automated painting process inside an automotive factory. In this version, the technological process is clearly divided into three main stages: painting, drying, and pick-and-place operation. Each stage is performed by a separate subsystem, which allows the process to be organized in a structured and safe way.

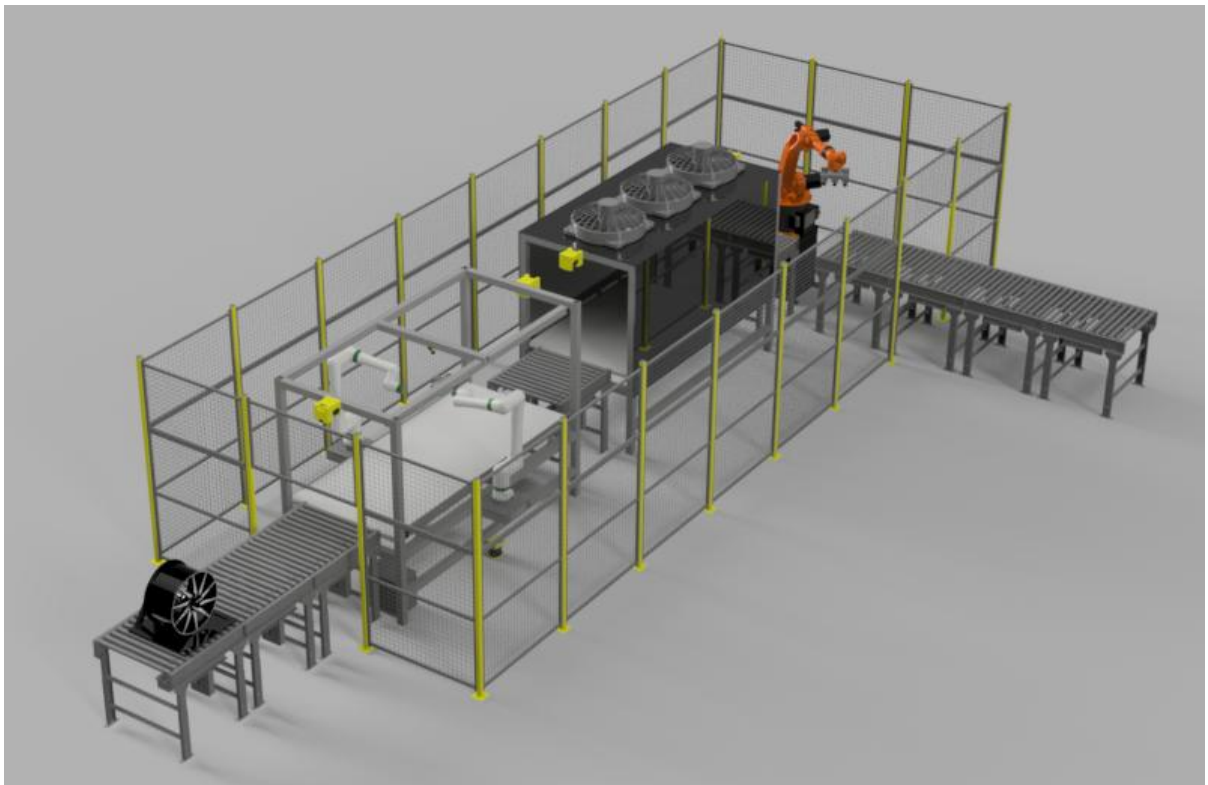


Figure 7.26 – Industrial scale project

The painting stage is performed by robotic manipulators that apply paint to the object with repeatable movement and stable trajectory. After painting, the object is moved to the drying zone, where the coating is stabilized before the final operation.

At the last stage, an industrial manipulator performs the pick-and-place process by moving the finished part to a selected position.

This model demonstrates how the proposed system can be scaled for real production. It also shows the correct placement of robots, conveyor equipment, sensors, and control devices. Therefore, the industrial-scale project can be used as a reference for future implementation in an automotive manufacturing environment.

Figure 7.27 presents the laboratory-scale version of the project. The full workstation was modeled in Fusion 360 before the physical implementation. This model made it possible to plan the placement of the main components, check the available working space, and prepare the system layout before assembly.



Figure 7.27 – Laboratory scale project

The laboratory workstation includes the SIMATIC S7-1500 educational stand, a conveyor belt, inductive sensors, two RoArm M2S manipulators, a KUKA manipulator, and a computer. The computer is used to run MATLAB software, the SCADA system, and the monitoring interface. The laboratory version repeats the main logic of the industrial process, but in a smaller and safer environment.

This scale was selected because it allows the main functions of the system to be tested without using a full industrial production line. At the same time, the control logic, communication structure, and robot coordination were developed according to industrial principles. Because of this, the laboratory model can be considered a practical prototype of the real automated painting system.

Figure 7.28 shows the fully assembled real-life implementation of the project. The system was built in a 1:1 scale relative to the planned laboratory model. All main parts of the workstation were installed and connected according to the designed structure.



Figure 7.28 – Real-life implementation

The inductive sensors were connected to the SIMATIC S7-1500 PLC and used to detect the object position on the conveyor. The conveyor belt was also connected to the controller, which allowed the PLC to start and stop the movement according to the process sequence. This made it possible to move the object between the painting, drying, and pick-and-place zones.

The RoArm M2S manipulators were connected wirelessly to the MATLAB program. The painting trajectory was generated and sent from MATLAB, allowing the manipulators to perform the required painting motion. This confirmed that the developed control approach can coordinate robot movement with the general process sequence.

At the final stage of the process, the KUKA manipulator performed the pick-and-place operation. The robot successfully picked up the object from the conveyor and placed it in the predefined position. This step confirmed that the final handling operation can be integrated into the same automated cycle.

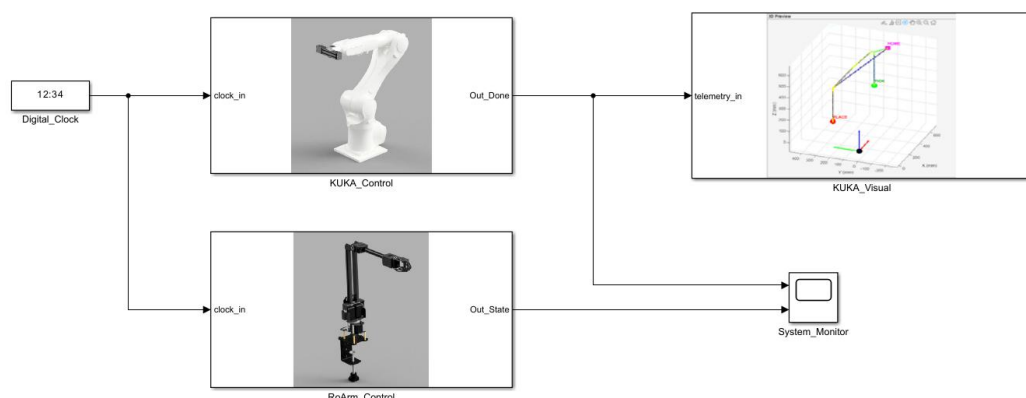


Figure 7.29 – Simulink of process

Figure 7.29 shows the Simulink logic used to start the whole technological process simultaneously. This logic allows the operator to launch the complete automated cycle by pressing only one button. As a result, the conveyor, sensors, PLC, MATLAB software, RoArm M2s manipulators, SCADA/HMI monitoring system, and KUKA robot operate as one coordinated system.

As a result, the laboratory implementation confirmed the main functionality of the proposed system. The conveyor, sensors, PLC, MATLAB software, RoArm manipulators, SCADA/HMI monitoring, and KUKA robot were combined into one working process. The completed prototype shows that the developed system can be used as a basis for further improvement and future industrial-scale implementation.

7.8 Conclusion

This section presented the complete software implementation of the multirobot painting control system. The MATLAB-based supervisory layer handles high-level trajectory generation and real-time command dispatch to the KUKA KR 10 R1100-2 via KukaVarProxy and to the two RoArm M2S manipulators via HTTP/JSON over Wi-Fi. The Siemens S7-1500 PLC, programmed in TIA Portal V19, implements the full technological sequence including conveyor indexing, drying zone management and robot handshake signals, with safety interlocks enforced at every transition. Conveyor motor speed is regulated by a PID controller implemented both as a manual ladder network and using the PID_Compact function block, providing reliable and tunable closed-loop speed control. The WinCC-based HMI provides the operator with a clear overview of the process, mode selection, manual controls and fault diagnostics, ensuring safe and efficient operation of the complete automated painting line.

8 LABOR PROTECTION

The use of robotic systems in industry makes it possible to increase productivity, accuracy and stability of the technological process. However, the operation of such a line requires special attention to occupational safety issues, as it includes manipulators, a conveyor, control cabinets, sensors, cameras and other technical means that may pose a danger to the operator and maintenance personnel. The main dangerous and harmful factors in this case include moving machinery, electric current, noise, risk of fire, as well as aerosol and chemical substances in the work area. Therefore, ensuring safe working conditions is an important part of the design of this system. Remote monitoring, alarms, interlocks and emergency stops play an essential role in reducing risk. Their use makes it possible to reduce the time spent by personnel in a hazardous area and increase the overall level of industrial safety.

8.1 Characteristics of the workplace of the automated line operator

The operator's workplace is designed to monitor and control the management of an automated painting line. The operator starts and stops the line, monitors the condition of the manipulators, conveyor, sensors, drives and control system, as well as monitors the signals and the general condition of the equipment. Most of the work is done from the control panel and computer, so the operator does not need to be constantly near the moving parts of the line.

The operator's job is to monitor the painting process, monitor the operation of the equipment and quickly respond to possible failures. Physical activity in the workplace is low, but such work requires attention and constant visual control.

The workplace should be comfortable and safe for work. It should include a control panel, a screen for displaying information, an emergency stop button, and light and sound alarms. Good lighting, ventilation and safe access to the controls must also be provided. During the operation of the line, the operator may be affected by noise, electric current, moving parts of equipment and substances located in the painting area.

8.2 Dangerous and harmful factors in the operation of the robotic system

During the operation of the robotic painting line, various dangerous and harmful factors can affect the operator and maintenance personnel. Their appearance is due to the presence of manipulators, conveyors, drives, control cabinets, paint spraying systems, as well as other electrical and mechanical equipment. Therefore, when designing such a system, it is necessary to determine in advance the main sources of danger and the possible consequences of their impact on humans. The main dangerous and harmful factors that occur during the operation of the robotic system are listed in Table 8.1.

Table 8.1 – Dangerous and harmful factors in the operation of a robotic system

№	Dangerous or harmful factor	Source of occurrence	Possible consequences
1	2	3	4
1	The working area of the manipulators	Movement of robots and actuators	Impact, pinching, injury
2	Conveyor movement area	Tape, rollers, drive elements	Clothing damage, pinching, mechanical injury
3	Electric current	Control cabinets, cables, drives, sensors	Electric shock
4	Aerosol and chemical substances	The painting process	Adverse effects on humans
5	Fire danger	Combustible materials, electrical equipment	Fire, emergency situation
6	Insufficient lighting and limited visibility	Operator's work area and individual sections of the line	Control errors, late detection of malfunctions

As can be seen from Table 8.1, the working area of the manipulators, the conveyor movement area and electrical equipment pose the greatest danger to personnel. During the movement of the manipulator, there is a risk of hitting or pinching a person if they accidentally find themselves in the area of movement.

The conveyor movement area is no less dangerous. If you are near moving parts, you may get clothes trapped, pinched or mechanically injured. An additional risk arises during line maintenance, if individual nodes have not been stopped in a timely manner.

According to Table 8.1, electric current is also an important hazard factor. The robotic system includes control cabinets, cable lines, sensors, actuators, cameras and other electrical devices. If the insulation is damaged, the connection is incorrect or maintenance regulations are violated, the risk of electric shock increases.

Special attention should be paid to aerosol and chemical substances that appear in the painting area. During coating, paint particles and other substances may enter the air of the work area. With insufficient ventilation, they accumulate in the air and can adversely affect a person.

As shown in Table 8.1, significant factors include fire hazard. It is associated with the presence of combustible materials, electrical equipment and heating elements. If there is a short circuit, equipment malfunction or accumulation of flammable substances, a fire may occur.

In addition, the safety of work is affected by insufficient lighting and a limited view of individual sections of the line. This can lead to operator errors and delayed detection of malfunctions.

Thus, the main dangerous and harmful factors in the operation of a robotic system are related to moving mechanisms, electrical equipment, noise, substances in the painting area and the risk of fire. Their consideration is necessary to choose protective measures and ensure safe working conditions.

8.3 Analysis of compliance of working conditions with regulatory requirements

The analysis of working conditions at the workplace of an automated line operator is based on the main factors that were discussed in the previous section. Since the projected system is considered as part of the graduation project, the assessment is carried out on the basis of the technical solutions adopted, the composition of the equipment and the requirements of regulatory documents. At the same time, the actual confirmation of the working environment parameters must be performed after the system is put into operation using the necessary measurements.

The basic requirements for the air of the working area and the content of harmful substances are established by ГОСТ 12.1.005–88, and hygienic requirements for the microclimate of workplaces are СанПиН 2.2.4.548–96. Noise requirements in workplaces are established by ГОСТ 12.1.003–83 and standards for natural and artificial lighting are СНиП 23-05-95 and СП 52.13330.2016. The requirements for protective measures of electrical safety and grounding of electrical equipment are defined by the ПУЭ, гл. 1.7 and ГОСТ 12.1.030–81. The results of the analysis of compliance of working conditions with the main regulatory requirements are shown in Table 8.2.

Table 8.2 – Analysis of compliance of working conditions with regulatory requirements

№	The factor	Regulatory document	Assessment for the projected system
1	2	3	4
1	Working area and air and harmful substances	ГОСТ 12.1.005–88	The presence of aerosol and chemical substances is possible in the painting area, therefore, the project should provide for ventilation and a limitation of the time spent by personnel in this area.
2	Microclimate	СанПиН 2.2.4.548–96	The operator's workplace must be located in a room with regulated parameters of temperature, humidity and air mobility.
3	Noise	ГОСТ 12.1.003–83	The noise sources are drives, conveyor belt and fans, so the noise level should be reduced to acceptable values due to equipment layout.

Continuation of table 8.2

1	2	3	4
4	Illumination	СНП 23-05-95, СП 52.13330.2016	The operator's workplace must be provided with sufficient artificial lighting; verification of this requirement is carried out in section 5.7.
5	Electrical safety	ПУЭ, гл. 1.7, ГОСТ 12.1.030–81	The metal parts of the equipment must be grounded and the control cabinets and supply circuits must have protective measures against electric shock.
6	Safety in the area of movement of mechanisms	Safety requirements for the organization of workplaces and fencing of hazardous areas	Access to the working area of the manipulators and conveyor should be limited; the project should provide for interlocks, emergency stop and warning alarms.

As can be seen from Table 8.2, the greatest attention in the project should be paid to the air of the working area, noise, lighting, electrical safety and restriction of access to the area of movement of manipulators and conveyor. For the painting area, the main condition is to avoid exceeding the permissible content of harmful substances in the air of the working area, since ГОСТ 12.1.005–88 requires that the content of harmful substances does not exceed the maximum permissible concentration used in the design of buildings, processes, equipment and ventilation.

The parameters of the microclimate at the operator's workplace must also comply with sanitary requirements. СанПиН 2.2.4.548–96 considers air temperature, relative humidity, air velocity and thermal radiation as the main indicators for assessing the microclimate in the workplace. Therefore, placing the operator outside the spray area and in a separate control area allows for more stable and safer working conditions.

Workplace lighting should be considered separately. Artificial lighting standards are established by СНП 23-05-95 and updated СП 52.13330.2016, while for general artificial lighting of premises it is recommended to use discharge light sources, and illumination from general lighting for a number of industrial premises should be at least 200 lux. For this reason, this project requires the calculation of artificial lighting, which is performed in section 6.7.

According to the noise requirements, working conditions must ensure sound exposure levels that do not exceed the permissible values in the workplace. ГОСТ 12.1.003–83 sets general noise safety requirements and monitoring of noise levels at workplaces is carried out according to established measurement methods. For the line in question, this means the need to reduce noise exposure due to the rational placement of equipment and remote control of the process.

One of the important requirements for a robotic line is to ensure electrical safety. The metal parts of control cabinets, drives and other equipment elements must be grounded, as dangerous voltage may appear on them if the insulation is damaged. Compliance with these requirements reduces the risk of electric shock and increases the safety of staff.

Thus, the decisions adopted in the draft generally make it possible to ensure compliance with the basic regulatory requirements for work area air, microclimate, lighting, noise and electrical safety. However, the final confirmation of compliance with working conditions should be carried out after installation and start-up of the line based on actual measurements and verification of the protective equipment.

8.4 Security measures

Taking into account the factors listed in Table 8.1, when organizing the operation of a robotic line, technical and organizational measures should be provided to reduce the risk to the operator and maintenance personnel. The main objective of these measures is to eliminate direct human contact with hazardous areas, reduce the impact of harmful factors and ensure a quick response in case of malfunctions.

One of the main measures is to restrict access to the working area of the manipulators and the conveyor movement area. Diagrams of the working areas of the KUKA manipulator, the RoArm M2S robot and the conveyor belt are given in Appendix M, R and U. An analysis of these schemes shows that when the equipment is operating, areas that are dangerous for the operator and maintenance personnel are formed around the robots and along the conveyor. Within these areas, there is a risk of impact, pinching and mechanical injury in case of accidental contact with moving parts, therefore, personnel access to them during the operation of the system should be excluded.

Fences, warning signs and visual controls should be provided to prevent access to dangerous areas. Access to such areas should be possible only after the system is completely shut down. To increase safety, locks should also be used to prevent equipment from starting when the fence is open or when the set operating mode is violated.

An emergency stop plays an important role. The emergency stop buttons should be located in accessible places and ensure that the movement of the manipulators, conveyor and other actuators is quickly shut down. This allows you to quickly stop the system when a dangerous situation occurs and reduce the risk of injury to personnel.

Risk reduction for the operator is also ensured by remote monitoring of the system. The main part of the control must be performed from the control panel and computer located outside the danger zone. This approach makes it possible to reduce the time spent by personnel near working equipment and make monitoring the process safer.

Special attention should be paid to the painting area. Ventilation and removal of polluted air from the work area must be provided to reduce exposure to aerosols

and chemicals. In addition, the presence of personnel near the coating site during the operation of the line should be limited.

The safety of work also depends on the proper organization of equipment maintenance. Before performing an inspection, adjustment or repair the line must be stopped and the moving and electrical parts disconnected. Maintenance work should be performed after checking the safe condition of the equipment.

Additional measures include adequate workplace lighting, good visibility of the main sections of the line, the presence of light and sound alarms, as well as free access to controls and emergency stop buttons. This allows the operator to notice deviations in the system faster and take the necessary measures in a timely manner.

8.5 Electrical and fire safety

When operating a robotic line, special attention should be paid to electrical and fire safety, as the system includes control cabinets, actuators, sensors, cable lines and other electrical devices. In addition, aerosols and flammable substances may be present in the painting area, which increases the risk of fire. Therefore, when organizing the operation of the line, measures should be taken to protect personnel and reduce the likelihood of an emergency.

Electrical safety is ensured by proper connection of equipment, grounding of metal parts, protection of cables and restriction of access to electrical components. The control cabinets must be closed and setup, inspection and repair must be performed only after the line is completely stopped and the power is turned off. This reduces the risk of electric shock and makes staff work safer.

Fire safety is ensured by ventilation of the painting area, removal of impurities, maintenance of cleanliness and monitoring of the condition of the equipment. There should be no open sources of fire near the line and if smoke, a burning smell or signs of fire appear, the system must be stopped immediately. For this purpose, an emergency stop, an alarm system and primary fire extinguishing equipment must be provided on the line.

8.6 Calculation of the ventilation of the painting area

Task statement: it is necessary to perform a preliminary calculation of the supply and exhaust ventilation of the painting area of the robotic line. The calculation is carried out in order to determine the required air flow required to remove aerosol paint particles, associated vapors and heated air from the work area. The calculation takes into account that the air requirements of the working area are established by ГОСТ 12.1.005–88, the requirements for the microclimate of workplaces are СанПиН 2.2.4.548–96, and the design of ventilation systems is carried out taking into account СНиП 2.04.05–91.

In the absence of accurate data on the amount of substances released, a preliminary calculation can be performed based on the multiplicity of air exchange. In this case, the required air flow is determined by the formula:

$$Q = n \cdot V \quad (8.1)$$

where Q is the air flow rate, m^3/h ; n is the air exchange rate; V is the volume of the painting area, m^3 .

For the calculation scheme, we take the dimensions of the painting area: room length $L=5m$, width $W=3m$, height $H=4m$. Then the volume of the room is determined by the formula:

$$V = L \cdot W \cdot H = 5 \cdot 3 \cdot 4 = 60m^3 \quad (8.2)$$

For a preliminary calculation, we take the air exchange ratio $n=10$. Then by formula (5.1) we obtain:

$$Q = n \cdot V = 10 \cdot 60 = 600 m^3/h \quad (8.3)$$

Therefore, it is necessary to provide an air exchange of at least $600 m^3/h$ for the painting area.

In order to prevent polluted air from spreading through the room, it is advisable to accept a little more inflow from the hood. In the calculation scheme, we can assume the supply air flow $Q_{supply} = 550 m^3/h$, and the exhaust air flow $Q_{exhaust} = 650 m^3/h$. This solution allows you to create a small vacuum in the painting area and direct the air movement towards the hood. The indoor air flow pattern is shown in Figure 8.1.

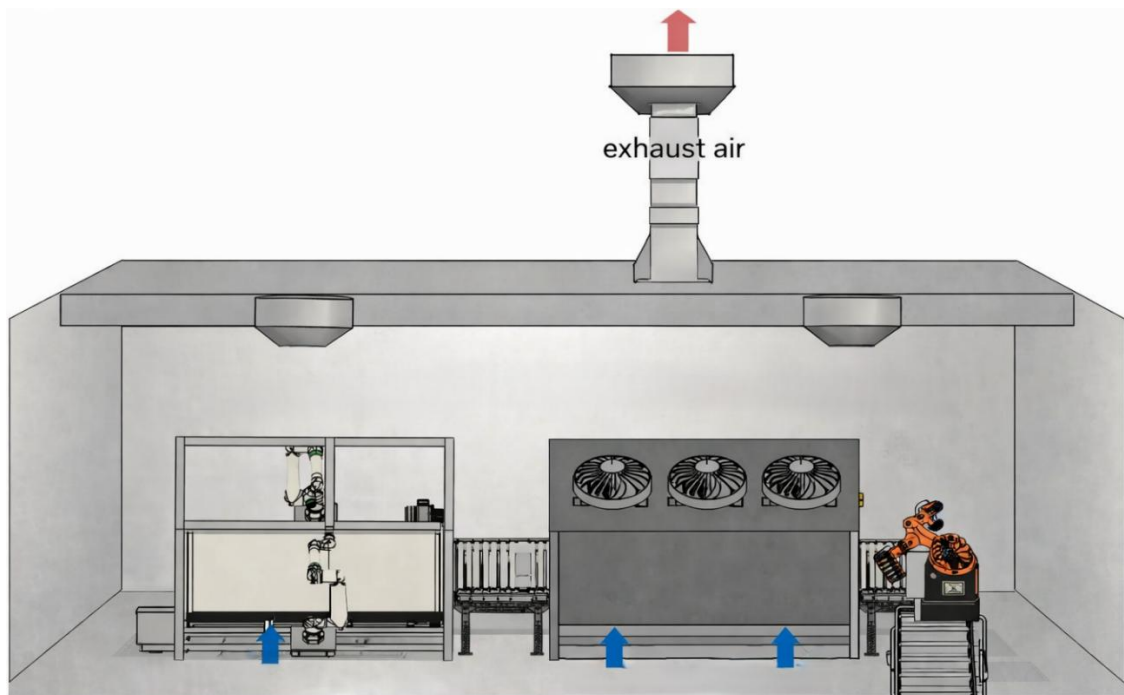


Figure 8.1– Scheme of supply and exhaust ventilation area

In the accepted version, fresh air is supplied to the lower part of the room, after which it passes through the work area. The removal of polluted air is carried out through exhaust ducts located above the painting area and above the drying area. This scheme makes it possible to reduce the concentration of paint aerosols and remove heated air from the most stressed sections of the line.

Thus, for the painting area under consideration, it is necessary to provide supply and exhaust ventilation with an estimated air exchange of at least 600 m³/h. In this case, the air supply must be supplied to the lower part of the room, and the hood must be arranged above the painting area and above the drying part of the line. This allows you to improve working conditions and increase the safety of the system.

8.7 Conclusion

The analysis carried out in this chapter showed that the robotic painting line is associated with a number of dangerous and harmful factors, including the movement of manipulators and conveyor elements, electric current, aerosol and chemical substances in the painting area, as well as the risk of fire. In this regard, labor protection is an important part of the design of the system. The characteristics of the operator's workplace, the main hazardous factors, and the compliance of the adopted technical solutions with regulatory requirements were considered.

Based on the identified risks, the main safety measures for the robotic line were determined. These include restricting access to hazardous areas, using fences, interlocks, emergency stop buttons, remote monitoring, ventilation and proper maintenance procedures. In addition, the preliminary ventilation calculation showed that the required air exchange for the painting area is at least 600 m³/h. The adopted solutions make it possible to improve working conditions, reduce the impact of hazardous factors on personnel, and increase the overall safety of the robotic line.

9 ECONOMIC SECTION

The economic justification for the introduction of this system is based on the reduction of manual labor, the shortening of the production cycle, the reduction of defect rates and paint overconsumption, and the improvement in the stability of coating quality. Each of the processes within the current production line are either performed manually or in a semi-automatic mode. The robotic cell will transfer these steps to the robot and eliminate the required labor for those steps.

9.1 Technical and Economic Justification of the Proposed System

The economic efficiency of the system is provided primarily through the reduction of the number of laborers that are required to produce the products, the elimination of the time spent in retooling the products, and waiting periods between steps in the production line, as well as through the reduction of defect rates during the production of each of the items in the production line. Additionally, the integration of the PLC and HMI into the paint robot will allow for the concentrated management of the robot, which will reduce the number of emergencies that occur within the production line and reduce the maintenance of the production line total.

Equipment cost data are taken from Section 6.6 of the diploma project, where the total cost of the complex is estimated at 50 874 USD. Staff salaries are adopted as indicative values based on open job postings in Almaty for 2026.

9.2 Calculation of Capital Expenditures for System Implementation

The capital expenditure calculation is one of the key metrics for assessing economic efficiency. It determines the total cost of implementing the automated process control system (APCS), accounting for labor, equipment, operating costs and electricity.

$$K = Z_{\text{lab}} + C_{\text{eq}} + C_{\text{op}} + C_{\text{el}} \quad (9.1)$$

where K is the total capital expenditure; Z_{lab} is the labor cost including mandatory employer contributions; C_{eq} is the equipment cost including installation and commissioning; C_{op} is the annual operating expenditure; C_{el} is the annual electricity cost.

9.2.1 Labor Cost Calculation for the Implementation Team

Labor costs cover the salaries of specialists involved in the design and commissioning of the multi-robot painting line. The team includes a project manager, an APCS engineer, a robotics engineer, a software engineer, an electrical commissioning engineer and a mechanical design engineer. Salaries are adopted as

indicative market values based on open vacancies in Almaty for 2026. The table below shows the labor cost of the implementation team

Table 9.1 - Labor cost of the implementation team

Position	Headcount	Period, months	Monthly salary, KZT	Total, KZT
Project Manager	1	3	600,000	1,800,000
APCS / PLC-HMI Engineer	1	3	600,000	1,800,000
Robotics Engineer	1	3	450,000	1,350,000
Software Engineer (MATLAB/ROS2)	1	3	500,000	1,500,000
Electrical Commissioning Engineer	1	3	350,000	1,050,000
Mechanical Design Engineer	1	3	400,000	1,200,000
Total S_base				8,700,000

The total labor cost consists of the base payroll and mandatory employer contributions:

$$Z_{lab} = S_{base} + S_{soc} \quad (9.2)$$

where Z_{lab} is the total labor cost for the implementation period; S_{base} is the base payroll fund; S_{soc} is the mandatory employer contributions.

Mandatory employer contributions applicable in the Republic of Kazakhstan in 2026 are: social contributions - 5%, employer contributions to compulsory medical insurance (OSMS) - 3% and employer mandatory pension contributions (OPVR) - 3.5%. The combined rate is 11.5%.

$$S_{soc} = 0.115 \times S_{base} = 0.115 \times 8,700,000 = 1,000,500 \text{ KZT} \quad (9.3)$$

where S_{soc} is the mandatory employer contributions; 0.115 is the combined statutory contribution rate; S_{base} is the base payroll fund.

$$Z_{lab} = 8,700,000 + 1,000,500 = 9,700,500 \text{ KZT} \quad (9.4)$$

Therefore the total labor cost for the implementation team amounts to 9,700,500 KZT.

9.2.2 Equipment Procurement and Installation Costs

The APCS of the multi-robot painting line includes a Siemens SIMATIC S7-1500 controller, I/O modules, an operator panel, a KUKA industrial robot, two

RoArm manipulators, sensors, a conveyor drive, communication equipment and auxiliary peripherals. The equipment cost table from Section 5.6 of the diploma project is reproduced below to confirm the total of 50 874 USD with a concrete component breakdown.

Table 9.2 - Main equipment costs of the diploma project

No.	Model / Specification	Qty	Price, USD	Total, USD	Total, KZT
1	KR10 R1100-2/SEL	1	42 000	42 000	21 000 000
2	RoArm M2s	2	307	614	307 000
3	57BLF01 motor + BLDC-8015A motor driver	1	65	65	32 500
4	SIMATIC PXI200M12	4	155	620	310 000
5	6ES7521-1BL10-0AA0	1	1 510	1 510	755 000
6	6ES7531-7KF00-0AB0	1	458	458	229 000
7	6ES7521-1BL00-0AB0	1	413	413	206 500
8	6ES7522-1BL01-0AB0f	1	653	653	326 500
9	6EP1332-4BA00	1	212	212	106 000
10	SIMATIC HMI TP700 Comfort, 6AV2124-0GC01-0AX0	1	900	900	450 000
11	TP-LINK TL-WR844N	1	15	15	7 500
12	TIA Portal V19	1	350	350	175 000
13	WinCC Runtime Advanced	1	2 500	2 500	1 250 000
14	Student Suite license	1	99	99	49 500
15	Electromagnetic lock 280 kg	1	25	25	12 500
16	Conveyor belt	1	400	400	200 000
17	Spray gun	2	20	40	20 000
	Total			50,874	25,437,000

The detailed table yields a total of 50,874 USD. An exchange rate of 1 USD = 500 KZT is applied for conversion to the national currency:

$$C_b = 50,874 \times 500 = 25,437,000 \text{ KZT} \quad (9.5)$$

where C_b is the base equipment cost. In addition to the base equipment cost, capital expenditures include allowances for unaccounted items such as cabling, connectors, fasteners and auxiliary materials. In engineering-economic calculations these are taken at 5% of the base equipment cost:

$$C_a = 0.05 \times C_b = 0.05 \times 25,437,000 = 1,271,850 \text{ KZT} \quad (9.6)$$

where C_a is the allowance for unaccounted equipment and peripherals. Adding the unaccounted allowance gives the total procurement cost:

$$C_{\text{proc}} = C_b + C_a = 25,437,000 + 1,271,850 = 26,708,850 \text{ KZT} \quad (9.7)$$

Installation and commissioning costs are taken at 25% of the procurement cost, consistent with the methodology used in the reference example [2]:

$$C_{\text{in}} = 0.25 \times C_{\text{proc}} = 0.25 \times 26,708,850 = 6,677,213 \text{ KZT} \quad (9.8)$$

where C_{in} is the installation and commissioning cost. The total equipment capital expenditure including procurement, unaccounted allowances and installation is:

$$C_{\text{eq}} = C_{\text{proc}} + C_{\text{in}} = 26,708,850 + 6,677,213 = 33,386,063 \text{ KZT} \quad (9.9)$$

Thus the total equipment cost of the APCS including all allowances and installation amounts to 33,386,063 KZT.

9.2.3 Operating Cost Calculation

Operating costs include depreciation charges, repair costs and maintenance expenses. The total is determined by:

$$C_{\text{op}} = A + R + M \quad (9.10)$$

where C_{op} is the total annual operating cost; A is depreciation; R is repair costs; M is maintenance costs.

Table 9.3 - Annual operating costs of the APCS equipment

Cost item	Amount, KZT
Depreciation of control and computing equipment (10%)	860,000
Depreciation of sensors and measurement devices (15.5%)	325,500
Repair costs (2.5% of $Z_{\text{lab}} + C_{\text{eq}}$)	1,077,164
Maintenance costs (2.3% of $Z_{\text{lab}} + C_{\text{eq}}$)	991,991
Total(C_{op})	3,254,655

The total annual operating cost for maintaining the APCS equipment in working condition amounts to 3,254,655 KZT.

9.2.4 Electricity Cost Calculation

Annual electricity costs are calculated using the total installed power of the equipment, the power utilization factor, daily operating hours and the number of working days per year.

First, annual electricity consumption is determined by multiplying the total installed power by the utilization factor, daily operating hours and the number of working days:

$$W_{el} = P_{total} \times k_u \times t_{day} \times D_{year} \quad (9.11.1)$$

$$W_{el} = 4.8 \times 0.75 \times 8 \times 250 = 7,200 \text{ kWh/year} \quad (9.11.2)$$

where W_{el} is the annual electricity consumption; P_{total} is the total installed power of the equipment, kW; k_u is the power utilization factor; t_{day} is the daily operating hours; D_{year} is the number of working days per year.

The annual electricity cost is then obtained by multiplying consumption by the applicable tariff:

$$C_{el} = W_{el} \times T_{el} = 7,200 \times 25 = 180,000 \text{ KZT/year} \quad (9.12)$$

where C_{el} is the annual electricity cost; T_{el} is the adopted tariff per kWh.

9.2.5 Total Capital Expenditure

All previously calculated components like labor, equipment, operating costs and electricity are combined into a single total:

$$K = 9,700,500 + 33,386,063 + 3,254,655 + 180,000 = 46,521,218 \text{ KZT} \quad (8.13)$$

Therefore the total capital expenditure for the design and implementation of the multi-robot painting line APCS amounts to 46,521,218 KZT.

9.2.6 Economic Efficiency of APCS Implementation

The primary economic benefit of deploying the robotic painting line lies in reducing the number of staff engaged in manual painting, part handling and inter-operation inspection as well as in cutting losses from defects and paint overconsumption. Indicative 2026 salary levels for Kazakhstan are used for production personnel.

Table 9.4 - Payroll fund before and after APCS implementation

Position	Headcount	Salary, KZT/month	Monthly payroll, KZT	Annual payroll, KZT
1	2	3	4	5
Payroll fund before APCS implementation				
Spray painter / operator	6	320,000	1,920,000	23,040,000
Conveyor line operator	1	300,000	300,000	3,600,000
Quality inspector	1	300,000	300,000	3,600,000
Part loading / unloading worker	2	280,000	560,000	6,720,000
Total S_bef				36,960,000
Payroll fund after APCS implementation				
Automated line operator	1	350,000	350,000	4,200,000
Robot maintenance engineer	1	450,000	450,000	5,400,000
APCS engineer	1	600,000	600,000	7,200,000
Quality inspector	1	300,000	300,000	3,600,000
Total S_aft				20,400,000

The total annual labor cost before implementation, including mandatory employer contributions of 11.5%, is:

$$Z_before = S_before + 0.115 \times S_before \quad (9.14.1)$$

$$Z_before = 36,960,000 + 4,250,400 = 41,210,400 \text{ KZT} \quad (9.14.2)$$

where Z_before is the total annual labor cost before APCS implementation; S_before is the annual base payroll before implementation.

After automation, total labor costs decrease because repetitive manual operations are replaced by robotic and automated equipment:

$$Z_after = S_after + 0.115 \times S_after \quad (9.15.1)$$

$$Z_after = 20,400,000 + 2,346,000 = 22,746,000 \text{ KZT} \quad (9.15.1)$$

where Z_{after} is the total annual labor cost after APCS implementation; S_{after} is the annual base payroll after implementation.

Payroll savings equal the difference between annual labor costs before and after automation:

$$E_{\text{sal}} = Z_{\text{before}} - Z_{\text{after}} \quad (9.16.1)$$

$$E_{\text{sal}} = 41,210,400 - 22,746,000 = 18,464,400 \text{ KZT} \quad (9.16.2)$$

where E_{sal} is the payroll saving due to headcount reduction.

Additional savings account for reduced paint consumption (precise robotic dosing eliminates overspray), lower defect rates (consistent coating thickness without human variability) and reduced line downtime (automatic diagnostics and real-time monitoring):

$$E_{\text{add}} = 2,500,000 \text{ KZT} \quad (9.17)$$

Total annual savings combine direct payroll savings and additional operational savings:

$$E_{\text{total}} = E_{\text{sal}} + E_{\text{add}} = 18,464,400 + 2,500,000 = 20,964,400 \text{ KZT} \quad (9.18)$$

where E_{total} is the total annual saving from APCS implementation. The annual economic effect is calculated using the standard efficiency coefficient. A value of $E_n = 0.32$ is adopted, consistent with the reference methodology:

$$E_{\text{year}} = E_{\text{total}} - E_n \times K \quad (9.19.1)$$

$$E_{\text{year}} = 20,964,400 - 0.32 \times 46,521,218 = 6,077,610 \text{ KZT} \quad (9.19.2)$$

where E_{year} is the annual economic effect; E_n is the standard economic efficiency coefficient; K is the total capital expenditure. The payback period shows how many years of accumulated savings are needed to recover the initial investment:

$$T_{\text{pb}} = K / E_{\text{total}} = 46,521,218 / 20,964,400 = 2.22 \text{ years} \quad (9.20)$$

where T_{pb} is the investment payback period. The results demonstrate a positive annual economic effect and a payback period of just over two years, confirming the economic viability of the project.

9.3 Investment Attractiveness Assessment

To provide a more complete project evaluation, taxable profit, corporate income tax, net profit and annual cash flow are determined. In this simplified

calculation, taxable profit equals the difference between total annual savings and annual operating and electricity costs.

Taxable profit equals the difference between annual savings and annual operating and electricity expenditures:

$$P_{\text{tax}} = E_{\text{total}} - C_{\text{op}} - C_{\text{el}} \quad (9.21.1)$$

$$P_{\text{tax}} = 20,964,400 - 3,254,655 - 180,000 = 17,529,745 \text{ KZT} \quad (9.21.2)$$

where P_{tax} is the taxable profit. Corporate income tax is calculated at the statutory rate of 20%:

$$\text{CIT} = 0.20 \times P_{\text{tax}} = 0.20 \times 17,529,745 = 3,505,949 \text{ KZT} \quad (9.22)$$

where CIT is the corporate income tax; 0.20 is the applicable CIT rate. Net profit is the portion of profit remaining after tax payment:

$$P_{\text{net}} = P_{\text{tax}} - \text{CIT} = 17,529,745 - 3,505,949 = 14,023,796 \text{ KZT} \quad (9.23)$$

Annual cash flow equals net profit plus depreciation. Depreciation is added back because it is a non-cash accounting charge:

$$\text{CF} = P_{\text{net}} + A = 14,023,796 + 1,185,500 = 15,209,296 \text{ KZT} \quad (9.24)$$

where CF is the annual cash flow and A is depreciation charges. The summary key performance indicators of the project are presented in Table 9.5.

Table 9.5 - Summary technical and economic indicators of the project

Indicator	Value, KZT
Total project cost (K)	46,521,218
Total annual savings (E_{total})	20,964,400
Annual economic effect (E_{year})	6,077,610
Payback period (T_{pb})	2.22 years
Net profit (P_{net})	14,023,796
Annual cash flow (CF)	15,209,296

9.4 Conclusion

The economic assessment of the developed system has established that implementing the multi-robot painting line with a Siemens S7-1500 controller, two RoArm manipulators, a KUKA industrial robot and a machine vision system is economically justified. The calculation shows that the total cost of project design and implementation amounts to 46,521,218 KZT. The largest share of expenditure falls on equipment procurement and installation, which reflects the project structure

whose main components are the KUKA industrial robot, RoArm manipulators, Siemens S7-1500 controller, I/O modules and visualization tools. Through reduction of maintenance staff, lower defect rates, decreased paint consumption and fewer line stoppages, annual savings reach 20,964,400 KZT. The annual economic effect is 6,077,610 KZT and the project payback period is 2.22 years. The annual cash flow amounts to 15,209,296 KZT. Thus the proposed system not only raises the technological level of production and the quality of part coating but also delivers a positive economic result. This confirms the feasibility of further deployment and development of the project in an academic production or industrial setting.

CONCLUSION

The development of a distributed control system for a multirobot painting line has demonstrated how the integration of robotics, industrial automation, and real-time monitoring can significantly improve manufacturing efficiency and quality. By combining RoArm manipulators for detailed coating operations and a KUKA KR 10 R1100-2 for handling, the system ensures both precision in surface finishing and reliability in material flow. The layered architecture-encompassing sensing, communication, supervisory control, and analytics-provides stable synchronization of all subsystems, minimizes downtime, and supports adaptive corrections during operation.

Practical results of the project include improved consistency of coating, reduced material waste through accurate spray control, and higher safety through distributed decision-making and fault tolerance. The use of MATLAB and TIA Portal has allowed integration of advanced algorithms with industrial-grade logic. Altogether, the project establishes a scalable framework that can be extended with additional robots, advanced sensors, or data-driven technology in future work. This ensures the system remains relevant for industrial painting lines seeking higher throughput, better quality, and smart manufacturing readiness.

References

1. International Federation of Robotics, “Collaborative Robots – How Robots Work alongside Humans,” IFR Press Release, Dec. 4, 2024.
2. C. Urrea and J. Kern, “Recent Advances and Challenges in Industrial Robotics: A Systematic Review of Technological Trends and Emerging Applications,” *Processes*, vol. 13, no. 3, Art. 832, 2025, doi:10.3390/pr13030832.
3. SIMATIC S7-1500, ET 200MP, ET 200SP, ET 200AL, ET 200pro, ET 200eco PN Diagnostics Function Manual.
4. J. Wu, X. Wang, B. Zhang, and T. Huang, “Multi-objective optimal design of a novel 6-DOF spray-painting robot,” *Robotica*, vol. 39, no. 12, pp. 2268–2282, 2021, doi: 10.1017/S026357472100031X.
5. Y. Liu, B. Zi, Z. Wang, S. Qian, L. Zheng, and L. Jiang, “Adaptive lead-through teaching control for spray-painting robot with closed control system,” *Robotica*, vol. 41, no. 4, pp. 1295–1312, 2023, doi: 10.1017/S0263574722001710.
6. C. Wang, X. Jian, Q. Yang, K. Sun, and S. Zhang, “Theoretical and Experimental Study on Coating Uniformity in Automatic Spray-Coating of Pipeline Weld Repairs,” *Coatings*, vol. 15, no. 10, Art. 1193, Oct. 2025, doi: 10.3390/coatings15101193.
7. Gabbar, H.A., and Idrees, M., “ARSIP: Automated Robotic System for Industrial Painting,” *Technologies*, vol. 12, no. 2, Art. 27, 2024, doi: 10.3390/technologies12020027.
8. C. Wang, X. Jian, Q. Yang, K. Sun, and S. Zhang, “Theoretical and Experimental Study on Coating Uniformity in Automatic Spray-Coating of Pipeline Weld Repairs,” *Coatings*, vol. 15, no. 10, Art. 1193, Oct. 2025, doi: 10.3390/coatings15101193.
9. Wu H, Tang Q. Robotic spray painting path planning for complex surface: boundary fitting approach. *Robotica*. 2023;41(6):1794-1811. doi:10.1017/S0263574723000152
10. Elmaraghy, Hoda & Monostori, Laszlo & Schuh, Guenther & Elmaraghy, W.. (2021). Evolution and future of manufacturing systems. *CIRP Annals*. 70. 635-658. 10.1016/j.cirp.2021.05.008.
11. Zbiss, Khalil & Kacem, Amal & Santillo, Mario & Mohammadi, Alireza. (2022). Automatic Collision-Free Trajectory Generation for Collaborative Robotic Car-Painting. *IEEE Access*. 10. 9950-9959. 10.1109/ACCESS.2022.3144631.
12. Zbiss, K.; Kacem, A.; Santillo, M.; Mohammadi, A. Automatic Optimal Robotic Base Placement for Collaborative Industrial Robotic Car Painting. *Appl. Sci.* 2024, 14, 8614. <https://doi.org/10.3390/app14198614>
13. Ghobakhloo S, Khoshakhlagh AH, Morais S, Mazaheri Tehrani A. Exposure to Volatile Organic Compounds in Paint Production Plants: Levels and Potential Human Health Risks. *Toxics*. 2023;11(2):111. Published 2023 Jan 24. doi:10.3390/toxics11020111
14. İritaş, S.B.; Bacanlı, M.G.; Taner, G.; Türksoy, V.A.; Tutkun, L.; Yilmaz, Ö.H.; Başaran, A.N. Chronic Occupational Exposure to Chemical Mixtures Induces

Genomic Instability in Paint Workers. *Toxics* 2025, 13, 785. <https://doi.org/10.3390/toxics13090785>

15. S. Borys and W. Kaczmarek, “Integration of the GoFa collaborative robot with a linear track using PROFINET,” *Przegląd Elektrotechniczny*, vol. 100, no. 12, pp. 32–36, 2024.

16. P. Obal and P. Gierlak, “EGM Toolbox—Interface for Controlling ABB Robots in Simulink,” *Sensors*, vol. 21, no. 22, Art. 7463, Nov. 2021, doi: 10.3390/s21227463.

17. MathWorks, “Drive Robot Using Inverse Kinematics in Simulink,” *Robotics System Toolbox*, 2022.

18. “Robot Programming - MATLAB Simulink,” MathWorks, Oct. 2025.

19. EL Zerk, A., Ouassaid, M., and Zidani, Y., “Development of a real-time framework between MATLAB and PLC through OPC-UA: A case study of a microgrid energy management system,” *Scientific African*, vol. 21, Art. e01846, 2023, doi: 10.1016/j.sciaf.2023.e01846.

20. AutomationDirect, “PLC Handbook: Practical Guide to Programmable Logic Controllers,” AutomationDirect, 2024.

21. J. Stenerson, *Industrial Automation and Process Control*, 2nd ed. Pearson, 2021.

22. Controlling a KUKA Industrial Robot Using a SIMATIC S7-1500. SIMATIC S7-1500 / TIA Portal V13 SP1 / KUKA.PLC mxAutomation.

23. Programming a KUKA Robot with a SIMATIC S7-1500. SIMATIC S7-1500 / TIA Portal V15.1 KUKA.PLC mxAutomation.

24. Use Case 1: Connecting Simulink Models to SIMATIC PLCSIM Advanced via API.

25. Use Case 2: Connecting Simulink Models to SIMATIC PLCSIM Advanced via OPC UA.

26. Hetai motor, Brushless DC motor 57BLF series. <https://www.promoco-motors.com/products/BLDC/57BLF%20Series.pdf>

27. Miklós Kuczmán, *Review of DC Motor Modeling and Linear Control: Theory with Laboratory Tests*, 2024.

28. К.Ю. Поляков, ТЕОРИЯ АВТОМАТИЧЕСКОГО УПРАВЛЕНИЯ ДЛЯ «ЧАЙНИКОВ», 2008.

29. PID Control System Design and Automatic Tuning using MATLAB/Simulink, Liuping Wang, 2020.

30. Ziegler-Nichols' Closed-Loop Method, Finn Haugen TechTeach, 2010.

31. Ziegler-Nichols Tuning of the PID controller, Dr. Eng. Mustafa Turki Hussein, 2015.

32. LQR Control, R. M. Murray, 2006.

33. ГОСТ 12.1.005–88. Система стандартов безопасности труда (ССБТ). Общие санитарно-гигиенические требования к воздуху рабочей зоны.

34. СанПиН 2.2.4.548–96. Гигиенические требования к микроклимату производственных помещений.

35. ГОСТ 12.1.003–83. Система стандартов безопасности труда (ССБТ). Шум. Общие требования безопасности.
36. ГОСТ 12.1.030–81. Система стандартов безопасности труда (ССБТ). Электробезопасность. Защитное заземление, зануление.
37. Правила устройства электроустановок (ПУЭ). Глава 1.7. Заземление и защитные меры электробезопасности.
38. СНиП 2.04.05–91*. Отопление, вентиляция и кондиционирование.
39. СНиП 23-05-95*. Естественное и искусственное освещение.
40. СП 52.13330.2016. Естественное и искусственное освещение. Актуализированная редакция СНиП 23-05-95.
41. Department of State Revenue of East Kazakhstan Region. Social payments in the Republic of Kazakhstan. 26.11.2025.
42. UNPF. Kazakhstan increases the mandatory employer pension contribution. 06.01.2026.
43. Department of State Revenue of Karaganda Region. On compulsory social health insurance.
44. HeadHunter Kazakhstan. Spray painter vacancies in Almaty, 2026.
45. HeadHunter Kazakhstan. Production line operator vacancies in Almaty, 2026.
46. HeadHunter Kazakhstan. Quality inspector vacancies in Almaty, 2026.
47. HeadHunter Kazakhstan. APCS engineer vacancies in Almaty, 2026.
48. HeadHunter Kazakhstan. Instrumentation and controls engineer vacancies in Almaty, 2026.
49. HeadHunter Kazakhstan. Project manager vacancies in Almaty, 2026.

APPENDIX A

The MATLAB code for Linearized model and eigenvalues.

```
clc; clear; close all;

fprintf('=====\n');
fprintf(' SECTION 1 – MOTOR PARAMETERS & MODEL\n');
fprintf('=====\n\n');

% --- Physical parameters ---
J = 1.2e-5; % Rotor inertia      [kg.m^2]
b = 1e-4;  % Viscous damping     [N.m.s/rad]
R = 1.3275; % Armature resistance [Ohm]
L = 0.00075; % Armature inductance [H]
Kt = 0.065; % Torque constant     [N.m/A]
Ke = Kt; % Back-EMF constant     [V.s/rad]
% --- Single consistent setpoint used throughout ALL sections ---
setpoint = 16.80955; % [rad/s]  $\approx 1/K_e$  at nominal operating point

s = tf('s');

fprintf(' J      = %.2e kg.m^2\n', J);
fprintf(' b      = %.2e N.m.s/rad (non-zero: realistic damping)\n', b);
fprintf(' R      = %.4f Ohm\n', R);
fprintf(' L      = %.5f H\n', L);
fprintf(' Kt     = %.4f N.m/A\n', Kt);
fprintf(' Ke     = %.4f V.s/rad (enforced = Kt)\n', Ke);
fprintf(' Setpoint = %.5f rad/s\n\n', setpoint);

% --- Transfer function:  $G(s) = K_t / (J*L*s^2 + (b*L+J*R)*s + (b*R+K_t*K_e))$  ---
num_G = Kt;
den_G = [J*L, (b*L + J*R), (b*R + Kt*Ke)];
G = tf(num_G, den_G);

fprintf('Open-loop poles of G(s):\n');
disp(pole(G).')
fprintf('DC gain G(0) = %.6f (rad/s)/V\n\n', dcgain(G));
G_dc = dcgain(G); % stored for fair open-loop scaling in plots

% --- State-space:  $x = [i_a ; \omega]$ ,  $y = \omega$  ---
A = [-R/L, -Ke/L;
     Kt/J, -b/J];
B = [1/L; 0];
```

```

C = [0, 1];
D = 0;
sys_ol = ss(A, B, C, D);

% --- Open-loop eigenvalue analysis ---
eigA = eig(A);
fprintf('Eigenvalues of A:\n');
disp(eigA.')
if all(real(eigA) < 0)
    disp('Open-loop plant: STABLE (all eigenvalues in LHP).');
else
    disp('Open-loop plant: UNSTABLE.');
```

end

APPENDIX B

The MATLAB code for calculate the Controllability and Observability.

```
% --- Controllability ---
Co = ctrb(A, B);
rankCo = rank(Co);
fprintf('\nControllability matrix rank = %d / %d --> ', rankCo, size(A,1));
if rankCo == size(A,1)
    disp('FULLY CONTROLLABLE.');
```

```
else
    disp('NOT fully controllable.');
```

```
end

% --- Observability ---
Ob = obsv(A, C);
rankOb = rank(Ob);
fprintf('Observability matrix rank = %d / %d --> ', rankOb, size(A,1));
if rankOb == size(A,1)
    disp('FULLY OBSERVABLE.');
```

```
else
    disp('NOT fully observable.');
```

```
end
```

APPENDIX C

The MATLAB code for the Bode plot.

```
% ----- Open-Loop Bode Plot -----

[GM, PM, Wcg, Wcp] = margin(G);

w = logspace(0, 5, 3000);
[mag, phase, wout] = bode(G, w);
mag_dB = squeeze(20*log10(squeeze(mag)));
wout = squeeze(wout);
phase_raw = squeeze(phase);

% --- Robust phase unwrapping (correct for any plant order) ---
phase_plot = unwrap(phase_raw * pi/180) * 180/pi;
phase_plot = phase_plot - 360 * round(phase_plot(1) / 360);

if isinf(GM)
    fprintf('Gain Margin = Inf dB (no phase-crossover for 2nd-order plant)\n');
else
    fprintf('Gain Margin = %.4f dB at %.4f rad/s\n', 20*log10(GM), Wcg);
end
fprintf('Phase Margin = %.4f deg at %.4f rad/s\n\n', PM, Wcp);

% --- Bode Plot ---

figure('Color','w','Name','Open-Loop Bode Plot');

% -- Magnitude subplot --
subplot(2,1,1)
hMag = semilogx(wout, mag_dB, 'Color',[0.00 0.20 0.80], 'LineWidth',2);
hold on; grid on;
h0 = yline(0, 'k:', 'LineWidth',1.2);

if ~isinf(Wcp) && Wcp > 0
    mag_Wcp = 20*log10(abs(squeeze(freqresp(G, Wcp))));
    hGcPt = plot(Wcp, mag_Wcp, 'o', 'Color',[0.10 0.85 0.10], ...
        'MarkerSize',8, 'LineWidth',2);
    hGcLn = xline(Wcp, ':', 'Color',[0.10 0.85 0.10], 'LineWidth',1.2);
    text(Wcp*1.05, mag_Wcp+3, sprintf('\omega_{gc}=%.2f', Wcp), ...
        'Color',[0.10 0.85 0.10], 'FontSize',11, 'FontWeight','bold');
else
    hGcPt = plot(nan,nan,'o','Color',[0.10 0.85 0.10],'LineWidth',2);
```

```

hGcLn = plot(nan,nan,':','Color',[0.10 0.85 0.10],'LineWidth',1.2);
end

if ~isinf(Wcg) && Wcg > 0
    mag_Wcg = 20*log10(abs(squeeze(freqresp(G, Wcg))));
    hPcPt = plot(Wcg, mag_Wcg, 'o', 'Color',[1.00 0.30 0.00], ...
        'MarkerSize',8, 'LineWidth',2);
    hPcLn = xline(Wcg, ':', 'Color',[1.00 0.30 0.00], 'LineWidth',1.2);
    if ~isinf(GM)
        text(Wcg*1.05, mag_Wcg+3, sprintf('GM=%.2f dB', 20*log10(GM)), ...
            'Color',[1.00 0.30 0.00], 'FontSize',11, 'FontWeight','bold');
    end
else
    hPcPt = plot(nan,nan,'o','Color',[1.00 0.30 0.00],'LineWidth',2);
    hPcLn = plot(nan,nan,':','Color',[1.00 0.30 0.00],'LineWidth',1.2);
end

title('Open-Loop Bode Plot', 'FontSize',16, 'FontWeight','bold');
ylabel('Magnitude (dB)', 'FontSize',13);
legend([hMag h0 hGcPt hGcLn hPcPt hPcLn], ...
    {'Magnitude','0 dB line','Gain crossover point','Gain crossover freq.', ...
    'Phase crossover point','Phase crossover freq.'}, ...
    'Location','northeast', 'FontSize',10);
set(gca, 'FontSize',12, 'LineWidth',1.2);

% -- Phase subplot --
subplot(2,1,2)
hPhase = semilogx(wout, phase_plot, 'Color',[0.55 0.00 0.75], 'LineWidth',2);
hold on; grid on;
h180 = yline(-180, 'k:', 'LineWidth',1.2);

if ~isinf(Wcp) && Wcp > 0
    ph_Wcp = angle(squeeze(freqresp(G,Wcp))) * 180/pi;
    ph_Wcp = ph_Wcp - 360*ceil(ph_Wcp/360);
    hPmPt = plot(Wcp, ph_Wcp, 'o', 'Color',[0.10 0.85 0.10], ...
        'MarkerSize',8, 'LineWidth',2);
    hPmLn = xline(Wcp, ':', 'Color',[0.10 0.85 0.10], 'LineWidth',1.2);
    text(Wcp*1.05, ph_Wcp+10, sprintf('PM=%.2f deg', PM), ...
        'Color',[0.10 0.85 0.10], 'FontSize',11, 'FontWeight','bold');
else
    hPmPt = plot(nan,nan,'o','Color',[0.10 0.85 0.10],'LineWidth',2);
    hPmLn = plot(nan,nan,':','Color',[0.10 0.85 0.10],'LineWidth',1.2);
end
end

```

```

if ~isinf(Wcg) && Wcg > 0
    ph_Wcg = angle(squeeze(freqresp(G,Wcg))) * 180/pi;
    ph_Wcg = ph_Wcg - 360*ceil(ph_Wcg/360);
    hGmPt = plot(Wcg, ph_Wcg, 'o', 'Color',[1.00 0.30 0.00], ...
        'MarkerSize',8, 'LineWidth',2);
    hGmLn = xline(Wcg, ':', 'Color',[1.00 0.30 0.00], 'LineWidth',1.2);
    text(Wcg*1.05, ph_Wcg+10, sprintf('\omega_{pc}=%.2f', Wcg), ...
        'Color',[1.00 0.30 0.00], 'FontSize',11, 'FontWeight','bold');
else
    hGmPt = plot(nan,nan,'o','Color',[1.00 0.30 0.00],'LineWidth',2);
    hGmLn = plot(nan,nan,':','Color',[1.00 0.30 0.00],'LineWidth',1.2);
end

xlabel('Frequency (rad/s)', 'FontSize',13);
ylabel('Phase (deg)', 'FontSize',13);
legend([hPhase h180 hPmPt hPmLn hGmPt hGmLn], ...
    {'Phase','-180 deg line','PM point','Gain crossover freq.', ...
    'GM point','Phase crossover freq.'}, ...
    'Location','northeast', 'FontSize',10);
set(gca, 'FontSize',12, 'LineWidth',1.2);

if isinf(GM)
    sgtitle(sprintf('Bode Diagram | GM = Inf | PM = %.2f deg', PM), ...
        'FontSize',15, 'FontWeight','bold');
else
    sgtitle(sprintf('Bode Diagram | GM = %.2f dB | PM = %.2f deg', ...
        20*log10(GM), PM), 'FontSize',15, 'FontWeight','bold');
end

```

APPENDIX D

The MATLAB code for the Nyquist plot

```
----- Open-Loop Nyquist Plot -----
w_ny = logspace(-1, 5, 4000);
resp = squeeze(freqresp(G, w_ny));

nyqColor   = [0.00 0.20 0.80];
pmPointColor = [0.00 0.00 1.00];
pmDirColor  = [0.50 0.00 0.80];
pmArcColor  = [1.00 0.50 0.00];
gmColor     = [0.10 0.90 0.10];
smColor     = [1.00 0.00 1.00];

figure('Color','w','Name','Open-Loop Nyquist Plot');
hNyg = plot(real(resp), imag(resp), 'Color',nyqColor, 'LineWidth',2);
hold on;
plot(real(resp), -imag(resp), 'Color',nyqColor, 'LineWidth',2, ...
      'HandleVisibility','off');
grid on;

hCrit = plot(-1, 0, 'rx', 'MarkerSize',10, 'LineWidth',2);
th     = linspace(0, 2*pi, 1000);
hCircle = plot(cos(th), sin(th), 'k:', 'LineWidth',1.5);

% Phase margin point (on unit circle)
[~, idx_pm] = min(abs(abs(resp) - 1));
Ppm         = resp(idx_pm);
PM_actual   = 180 + atan2d(imag(Ppm), real(Ppm));

hPMpoint = plot(real(Ppm), imag(Ppm), 'o', 'Color',pmPointColor, ...
                'MarkerSize',9, 'LineWidth',2);
hPMdir    = plot([0 real(Ppm)], [0 imag(Ppm)], '--', ...
                'Color',pmDirColor, 'LineWidth',1.6);

% Gain margin point (where phase = -180 deg)
phase_ny = unwrap(angle(resp)) * 180/pi;
phase_ny = phase_ny - 360*round(phase_ny(1)/360);
[~, idx_gm] = min(abs(phase_ny + 180));
Pgm         = resp(idx_gm);
hGMpoint = plot(real(Pgm), imag(Pgm), 'o', 'Color',gmColor, ...
                'MarkerSize',9, 'LineWidth',2);
hGMdir    = plot([0 real(Pgm)], [0 imag(Pgm)], '--', ...
```

```

'Color',gmColor, 'LineWidth',1.4);

% Stability margin (closest point to critical -1+0j)
[SM, idx_sm] = min(abs(resp + 1));
Psm          = resp(idx_sm);
hSMpoint     = plot(real(Psm), imag(Psm), 'o', 'Color',smColor, ...
    'MarkerSize',9, 'LineWidth',2);
hSMdir       = plot([real(Psm) -1], [imag(Psm) 0], '--', ...
    'Color',smColor, 'LineWidth',1.6);

% PM arc
theta_point = atan2(imag(Ppm), real(Ppm));
if theta_point < 0, theta_point = theta_point + 2*pi; end
if theta_point >= pi
    arc_theta = linspace(pi, theta_point, 120);
else
    arc_theta = linspace(theta_point, pi, 120);
end
hPMarc = plot(0.18*cos(arc_theta), 0.18*sin(arc_theta), ...
    'Color',pmArcColor, 'LineWidth',2.4);

text(real(Ppm)-0.08, imag(Ppm)-0.05, sprintf('PM = %.1f deg', PM_actual), ...
    'Color',pmPointColor, 'FontSize',12, 'FontWeight','bold');
text(real(Pgm)-0.05, imag(Pgm)+0.05, 'GM', ...
    'Color',gmColor, 'FontSize',12, 'FontWeight','bold');
text(real(Psm)-0.05, imag(Psm)-0.08, sprintf('SM = %.3f', SM), ...
    'Color',smColor, 'FontSize',12, 'FontWeight','bold');
text(-1.08, 0.05, '(-1,0)', 'Color','r', 'FontSize',12, 'FontWeight','bold');

title('Open-Loop Nyquist Plot', 'FontSize',16, 'FontWeight','bold');
xlabel('Real Axis', 'FontSize',14);
ylabel('Imaginary Axis', 'FontSize',14);
legend([hNyq hCrit hCircle hPMpoint hPMdir hGMpoint hGMdir hSMpoint
hSMdir hPMarc], ...
    {'Nyquist curve','Critical point (-1,0)','Unit circle', ...
    'PM point','PM direction','GM point','GM direction', ...
    'SM point','Stability margin','PM angle'}, ...
    'Location','best', 'FontSize',11);
set(gca, 'FontSize',13, 'LineWidth',1.2);

```

APPENDIX E

The MATLAB code for Step response , Impulse response and Pole-zero map.

```
sys_cl = feedback(G, 1);

% --- Step response ---
t_cl = linspace(0, 0.05, 3000);
[y_cl, t_cl] = step(setpoint * sys_cl, t_cl);
y_cl = squeeze(y_cl);
ss_value = dcgain(setpoint * sys_cl); % exact theoretical SS value
ss_error = setpoint - ss_value;
info = stepinfo(setpoint * sys_cl);

fprintf('Steady-state value = %.6f rad/s\n', ss_value);
fprintf('Steady-state error = %.6f rad/s (%.2f%%)\n', ss_error, ...
    100*ss_error/setpoint);
fprintf('Rise Time = %.6f s\n', info.RiseTime);
fprintf('Settling Time = %.6f s\n', info.SettlingTime);
fprintf('Overshoot = %.4f %%\n\n', info.Overshoot);

% --- Step Response Plot ---
respColor = [0.00 0.20 0.80];
setpColor = [0.00 0.00 0.00];
ssColor = [0.75 0.00 0.75];
riseColor = [0.10 0.75 0.10];
peakColor = [1.00 0.35 0.00];
settleColor = [0.00 0.75 0.85];

figure('Color','w','Name','CL Step Response - No Controller');
hold on; grid on; box on;

h_response = plot(t_cl, y_cl, 'Color',respColor, 'LineWidth',2.2);
h_setpoint = yline(setpoint, '--', 'Color',setpColor, 'LineWidth',1.4);
h_ss = yline(ss_value, '--', 'Color',ssColor, 'LineWidth',1.4);

x_rise = info.RiseTime;
y_rise = interp1(t_cl, y_cl, x_rise);
h_rise = plot([x_rise x_rise], [0 y_rise], '--', 'Color',riseColor, 'LineWidth',1.5);
plot(x_rise, y_rise, 'o', 'Color',riseColor, 'MarkerSize',8, 'LineWidth',2);

x_settle = info.SettlingTime;
h_settle = plot([x_settle x_settle], [0 ss_value], '--', ...
    'Color',settleColor, 'LineWidth',1.5);
```

```

plot(x_settle, ss_value, 'o', 'Color',settleColor, 'MarkerSize',8, 'LineWidth',2);

if info.Overshoot > 1e-3
    [peak_val, peak_idx] = max(y_cl);
    t_peak = t_cl(peak_idx);
    h_peak = plot([t_peak t_peak], [0 peak_val], '--', ...
        'Color',peakColor, 'LineWidth',1.5);
    plot(t_peak, peak_val, 'o', 'Color',peakColor, 'MarkerSize',8, 'LineWidth',2);
    text(t_peak*1.02, peak_val+0.4, ...
        sprintf('Overshoot = %.2f%%', info.Overshoot), ...
        'Color',peakColor, 'FontSize',12, 'FontWeight','bold');
else
    h_peak = plot(nan,nan,'--','Color',peakColor,'LineWidth',1.5);
end

text(x_rise*1.08, y_rise*0.55, ...
    sprintf('Rise Time = %.4f s', info.RiseTime), ...
    'Color',riseColor, 'FontSize',12, 'FontWeight','bold');
text(x_settle+0.001, ss_value*0.87, ...
    sprintf('Settling Time = %.4f s', info.SettlingTime), ...
    'Color',settleColor, 'FontSize',12, 'FontWeight','bold');
text(t_cl(end)*0.01, ss_value+0.6, ...
    sprintf('Steady-state = %.4f', ss_value), ...
    'Color',ssColor, 'FontSize',12, 'FontWeight','bold');
text(t_cl(end)*0.01, ss_value+1.4, ...
    sprintf('SS error = %.4f (%.2f%%)', ss_error, 100*ss_error/setpoint), ...
    'Color',setpColor, 'FontSize',12);

title(sprintf('Step Response - No Controller (Setpoint = %.5f rad/s)', setpoint), ...
    'FontSize',16, 'FontWeight','bold');
xlabel('Time [s]', 'FontSize',14);
ylabel('\omega [rad/s]', 'FontSize',14);
legend([h_response h_setpoint h_ss h_rise h_peak h_settle], ...
    {'Response','Setpoint','Steady-state','Rise time','Overshoot','Settling time'}, ...
    'Location','best', 'FontSize',11);
set(gca, 'FontSize',13, 'LineWidth',1.2);

% --- Impulse Response ---
t_imp = linspace(0, 0.03, 3000);
[y_imp, t_imp] = impulse(sys_cl, t_imp);
y_imp = squeeze(y_imp);

figure('Color','w','Name','CL Impulse Response');

```

```

hold on; grid on; box on;

h_imp = plot(t_imp, y_imp, 'Color',respColor, 'LineWidth',2.2);
h_zero = yline(0, '--', 'Color',setpColor, 'LineWidth',1.2);

[imp_peak, idx_ip] = max(y_imp);
[imp_min, idx_im] = min(y_imp);
t_imp_peak = t_imp(idx_ip);
t_imp_min = t_imp(idx_im);

h_imp_peak = plot(t_imp_peak, imp_peak, 'o', 'Color',peakColor, ...
    'MarkerSize',8, 'LineWidth',2);
plot([t_imp_peak t_imp_peak], [0 imp_peak], '--', 'Color',peakColor,
'LineWidth',1.4);
plot(t_imp_min, imp_min, 'o', 'Color',[0.55 0.00 0.75], 'MarkerSize',8,
'LineWidth',2);
plot([t_imp_min t_imp_min], [0 imp_min], '--', 'Color',[0.55 0.00 0.75],
'LineWidth',1.4);

text(t_imp_peak*1.05, imp_peak*0.90, sprintf('Peak = %.2f', imp_peak), ...
    'Color',peakColor, 'FontSize',12, 'FontWeight','bold');
text(t_imp_min*1.05, imp_min*0.90, sprintf('Min = %.2f', imp_min), ...
    'Color',[0.55 0.00 0.75], 'FontSize',12, 'FontWeight','bold');

title('Closed-Loop Impulse Response', 'FontSize',16, 'FontWeight','bold');
xlabel('Time [s]', 'FontSize',14);
ylabel('\omega [rad/s]', 'FontSize',14);
legend([h_imp h_zero h_imp_peak], ...
    {'Impulse response','Zero line','Peak'}, 'Location','best', 'FontSize',11);
set(gca, 'FontSize',13, 'LineWidth',1.2);

% --- Pole-Zero Map ---
fprintf('=== POLE-ZERO MAP (closed-loop, no controller) ===\n');
poles_cl = pole(sys_cl);
zeros_cl = zero(sys_cl);
fprintf('Closed-loop poles:\n'); disp(poles_cl.)

figure('Color','w','Name','CL Pole-Zero Map');
hold on; grid on; box on;

h_poles = plot(real(poles_cl), imag(poles_cl), 'x', ...
    'Color',respColor, 'MarkerSize',14, 'LineWidth',2);
if ~isempty(zeros_cl)
    h_zeros = plot(real(zeros_cl), imag(zeros_cl), 'o', ...

```

```

        'Color',peakColor, 'MarkerSize',12, 'LineWidth',2);
else
    h_zeros = plot(nan,nan,'o','Color',peakColor,'MarkerSize',12,'LineWidth',2);
end
xline(0,'--','Color',setpColor,'LineWidth',1.1);
yline(0,'--','Color',setpColor,'LineWidth',1.1);

for k = 1:length(poles_cl)
    text(real(poles_cl(k))+15, imag(poles_cl(k))+50*(-1)^k, ...
        sprintf('%.2f%+.2fj', real(poles_cl(k)), imag(poles_cl(k))), ...
        'Color',respColor, 'FontSize',11, 'FontWeight','bold');
end
for k = 1:length(zeros_cl)
    text(real(zeros_cl(k))+15, imag(zeros_cl(k))+50, ...
        sprintf('%.2f%+.2fj', real(zeros_cl(k)), imag(zeros_cl(k))), ...
        'Color',peakColor, 'FontSize',11, 'FontWeight','bold');
end

title('Closed-Loop Pole-Zero Map', 'FontSize',16, 'FontWeight','bold');
xlabel('Real Axis ( $s^{-1}$ )', 'FontSize',14);
ylabel('Imaginary Axis ( $s^{-1}$ )', 'FontSize',14);
legend([h_poles h_zeros], {'Poles','Zeros'}, 'Location','best', 'FontSize',11);
set(gca, 'FontSize',13, 'LineWidth',1.2);
if all(real(poles_cl) < 0)
    disp('Pole-Zero: STABLE (all poles in LHP).');
else
    disp('Pole-Zero: UNSTABLE.');
```

end

```

zpk(sys_cl)
```

APPENDIX F

The MATLAB code for Hurwitz and Lyapunov stability check

```
fprintf('\n=== HURWITZ STABILITY CHECK ===\n');
[~,
 den_tf] = tfdata(sys_cl, 'v');
if iscell(den_tf), den_tf = den_tf{1}; end
fprintf('Characteristic polynomial coefficients:\n'); disp(den_tf);

sys_cl_ss_form = ss(sys_cl);
Acl = sys_cl_ss_form.A;
if all(real(eig(Acl)) < 0)
    disp('Hurwitz criterion: STABLE (all eigenvalues in LHP).');
else
    disp('Hurwitz criterion: UNSTABLE.');
```

end

% --- Lyapunov Stability ---

```
fprintf('\n=== LYAPUNOV STABILITY CHECK ===\n');
Q_lyap = eye(size(Acl));
P_lyap = lyap(Acl, Q_lyap);
if all(eig(P_lyap) > 0) && all(real(eig(Acl)) < 0)
    disp('Lyapunov: P is positive definite --> closed-loop is STABLE.');
```

else

```
    disp('Lyapunov: stability condition NOT satisfied.');
```

end

```
fprintf('\n=== SENSITIVITY ANALYSIS – NO CONTROLLER ===\n');
S_no = feedback(1, G); % S = 1/(1+G)
T_no = feedback(G, 1); % T = G/(1+G)
Ms_no = 20*log10(getPeakGain(S_no));
Mt_no = 20*log10(getPeakGain(T_no));
fprintf('Peak Sensitivity Ms    = %.2f dB (< 6 dB = good robustness)\n', Ms_no);
fprintf('Peak Comp. Sens. Mt    = %.2f dB\n', Mt_no);
if Ms_no < 6
    disp('Sensitivity: GOOD robustness margin.');
```

else

```
    disp('Sensitivity: POOR robustness -- a controller is needed.');
```

end

APPENDIX G

The MATLAB code for P, PI, PD, PID and I controllers with auto-tuning settings in simulink

```
% Auto-Tuned Controllers (P / PI / PD / PID / I)
fprintf('\n===== \n');
fprintf(' SECTION 4 – AUTO-TUNED CONTROLLERS \n');
fprintf('===== \n \n');

% --- Controller gains ---
Kp_pid = 0.126990104339988;
Ki_pid = 31.5401187377579;
Kd_pid = 7.42813189240244e-05;

Kp_pi = 0.127327769462067;
Ki_pi = 11.4913466320886;

Kp_p = 0.379840196142128;

Kp_pd = 1.93638309848532;
Kd_pd = 0.00113297986811313;

Ki_i = 8.83054574193016;

Auto_PID = Kp_pid + Kd_pid*s + Ki_pid/s;
Auto_PI = Kp_pi + Ki_pi/s;
Auto_P = Kp_p;
Auto_PD = Kp_pd + Kd_pd*s;
Auto_I = Ki_i/s;

% --- Closed-loop systems ---
sys_cl_PID = feedback(Auto_PID*G, 1);
sys_cl_PI = feedback(Auto_PI*G, 1);
sys_cl_P = feedback(Auto_P*G, 1);
sys_cl_PD = feedback(Auto_PD*G, 1);
sys_cl_I = feedback(Auto_I*G, 1);

% --- Exact steady-state values from DC gain ---
ss_PID = dcgain(setpoint * sys_cl_PID);
ss_PI = dcgain(setpoint * sys_cl_PI);
ss_P = dcgain(setpoint * sys_cl_P);
ss_PD = dcgain(setpoint * sys_cl_PD);
ss_I = dcgain(setpoint * sys_cl_I);
```

```

fprintf('Steady-state tracking (setpoint = %.5f rad/s):\n', setpoint);
fprintf(' PID : SS = %.4f error = %.4f (0.2f%%)\n', ...
    ss_PID, setpoint-ss_PID, 100*(setpoint-ss_PID)/setpoint);
fprintf(' PI : SS = %.4f error = %.4f (0.2f%%)\n', ...
    ss_PI, setpoint-ss_PI, 100*(setpoint-ss_PI)/setpoint);
fprintf(' P : SS = %.4f error = %.4f (0.2f%%) <-- inherent SS error\n', ...
    ss_P, setpoint-ss_P, 100*(setpoint-ss_P)/setpoint);
fprintf(' PD : SS = %.4f error = %.4f (0.2f%%) <-- inherent SS error\n', ...
    ss_PD, setpoint-ss_PD, 100*(setpoint-ss_PD)/setpoint);
fprintf(' I : SS = %.4f error = %.4f (0.2f%%)\n\n', ...
    ss_I, setpoint-ss_I, 100*(setpoint-ss_I)/setpoint);

% --- Step responses ---
Tend = 0.06;
t = linspace(0, Tend, 2000);

% Open-loop: scale the input voltage so steady-state equals setpoint
% (fair visual comparison against closed-loop curves)
[y_open, t_open] = step((setpoint/G_dc)*G, t);
[y_noPID, t_noPID] = step(setpoint * sys_cl, t);
[y_PID, t_PID] = step(setpoint * sys_cl_PID, t);
[y_PI, t_PI] = step(setpoint * sys_cl_PI, t);
[y_P, t_P] = step(setpoint * sys_cl_P, t);
[y_PD, t_PD] = step(setpoint * sys_cl_PD, t);
[y_I, t_I] = step(setpoint * sys_cl_I, t);

orange = [0.8500 0.3250 0.0980];

figure('Color','w','Name','Step Responses - Auto-Tuned Controllers');
plot(t_open, y_open, 'g', 'LineWidth',1.8); hold on;
plot(t_noPID, y_noPID, 'b', 'LineWidth',1.8);
plot(t_PID, y_PID, 'r', 'LineWidth',2.2);
plot(t_PI, y_PI, 'c', 'LineWidth',1.8);
plot(t_P, y_P, 'm', 'LineWidth',1.8);
plot(t_PD, y_PD, 'y', 'LineWidth',1.8);
plot(t_I, y_I, '', 'Color',orange, 'LineWidth',1.8);
yline(setpoint, 'k--', 'LineWidth',1.4);

hold off;
title(sprintf('Step Response with Auto-Tuned Controllers (Setpoint = %.5f rad/s)',
...
    setpoint), 'FontSize',14, 'FontWeight','bold');
xlabel('Time (s)', 'FontSize',13);

```

```

ylabel('\omega [rad/s]', 'FontSize',13);
legend('Open-loop (no feedback)', 'Closed-loop(No controller)', ...
'PID controller(Closed-loop)', 'PI controller(Closed-loop)', 'P controller(Closed-
loop)', ...
'PD controller(Closed-loop)', 'I controller(Closed-loop)', 'Setpoint', ...
'Location','best', 'FontSize',10);
grid on;
set(gca, 'FontSize',12, 'LineWidth',1.2);

% --- Sensitivity analysis for each auto-tuned controller ---
fprintf('=== SENSITIVITY ANALYSIS with AUTO-TUNED CONTROLLERS
===\n');
fprintf('%-6s %10s %10s %s\n','Ctrl','Ms (dB)','Mt (dB)','Robustness');
fprintf('%s\n', repmat('-',1,52));

ctrl_auto = {Auto_PID, Auto_PI, Auto_P, Auto_PD, Auto_I};
cname_auto = {'PID','PI','P','PD','I'};
for k = 1:numel(ctrl_auto)
    S_k = feedback(1, ctrl_auto{k}*G);
    T_k = feedback(ctrl_auto{k}*G, 1);
    Ms_k = 20*log10(getPeakGain(S_k));
    Mt_k = 20*log10(getPeakGain(T_k));
    if Ms_k < 6, rob = 'Good'; else, rob = 'Poor'; end
    fprintf('%-6s %10.2f %10.2f %s\n', cname_auto{k}, Ms_k, Mt_k, rob);
end

```

APPENDIX H

The MATLAB code for P, PI, PD and PID controllers with Ziegler-Nichols method

```
% Ziegler-Nichols Controllers
fprintf('\n=====\\n');
fprintf(' SECTION 5 – ZIEGLER-NICHOLS CONTROLLERS\\n');
fprintf('=====\\n\\n');

Td = 1e-3; % [s] assumed transport + computation delay
fprintf('Assumed transport delay Td = %.1f ms\\n', Td*1e3);
fprintf('(Must be justified by PWM period or ADC sample rate)\\n\\n');

[n_pade, d_pade] = pade(Td, 1); % 1st-order Pade approximation
Gd = G * tf(n_pade, d_pade); % plant with delay

[GM_zn, ~, Wgm_zn, ~] = margin(Gd);
Ku = GM_zn; % ultimate gain (linear ratio – correct)
Pu = 2*pi / Wgm_zn; % ultimate period [s]

fprintf('Ultimate gain Ku = %.6f (linear scale, not dB)\\n', Ku);
fprintf('Ultimate period Pu = %.6f s\\n\\n', Pu);

% --- ZN formulas ---
Kp_ZN_P = 0.50*Ku;
ZN_P = Kp_ZN_P;

Kp_ZN_PI = 0.45*Ku;
Ki_ZN_PI = 1.2*Kp_ZN_PI / Pu;
ZN_PI = Kp_ZN_PI + Ki_ZN_PI/s;

Kp_ZN_PID = 0.60*Ku;
Ki_ZN_PID = 2*Kp_ZN_PID / Pu;
Kd_ZN_PID = Kp_ZN_PID*Pu / 8;
ZN_PID = Kp_ZN_PID + Kd_ZN_PID*s + Ki_ZN_PID/s;

Kp_ZN_PD = 0.80*Ku;
Kd_ZN_PD = Kp_ZN_PD*Pu / 8;
ZN_PD = Kp_ZN_PD + Kd_ZN_PD*s;

fprintf('ZN Gains:\\n');
fprintf(' P Kp = %.6f\\n', Kp_ZN_P);
fprintf(' PI Kp = %.6f Ki = %.6f\\n', Kp_ZN_PI, Ki_ZN_PI);
```

```

fprintf(' PID Kp = %.6f Ki = %.6f Kd = %.6e\n', ...
        Kp_ZN_PID, Ki_ZN_PID, Kd_ZN_PID);
fprintf(' PD Kp = %.6f Kd = %.6e\n\n', Kp_ZN_PD, Kd_ZN_PD);

% --- Closed-loop systems ---
sys_cl_ZN_P = feedback(ZN_P*Gd, 1);
sys_cl_ZN_PI = feedback(ZN_PI*Gd, 1);
sys_cl_ZN_PID = feedback(ZN_PID*Gd, 1);
sys_cl_ZN_PD = feedback(ZN_PD*Gd, 1);
sys_Ku = feedback(Ku*Gd, 1); % at ultimate gain (reference only)

% --- Steady-state values ---
ss_ZN_PID = dcgain(setpoint * sys_cl_ZN_PID);
ss_ZN_PI = dcgain(setpoint * sys_cl_ZN_PI);
ss_ZN_P = dcgain(setpoint * sys_cl_ZN_P);
ss_ZN_PD = dcgain(setpoint * sys_cl_ZN_PD);

fprintf('ZN Steady-state tracking:\n');
fprintf(' PID : SS = %.4f error = %.4f (0.2f%%)\n', ...
        ss_ZN_PID, setpoint-ss_ZN_PID, 100*(setpoint-ss_ZN_PID)/setpoint);
fprintf(' PI : SS = %.4f error = %.4f (0.2f%%)\n', ...
        ss_ZN_PI, setpoint-ss_ZN_PI, 100*(setpoint-ss_ZN_PI)/setpoint);
fprintf(' P : SS = %.4f error = %.4f (0.2f%%) <-- inherent SS error\n', ...
        ss_ZN_P, setpoint-ss_ZN_P, 100*(setpoint-ss_ZN_P)/setpoint);
fprintf(' PD : SS = %.4f error = %.4f (0.2f%%) <-- inherent SS error\n\n', ...
        ss_ZN_PD, setpoint-ss_ZN_PD, 100*(setpoint-ss_ZN_PD)/setpoint);

% --- Step responses ---
t_zn = linspace(0, Tend, 2000);
[y_ZN_open, ~] = step((setpoint/G_dc)*G, t_zn);
[y_ZN_noCl, ~] = step(setpoint*sys_cl, t_zn);
[y_ZN_PID, ~] = step(setpoint*sys_cl_ZN_PID, t_zn);
[y_ZN_PI, ~] = step(setpoint*sys_cl_ZN_PI, t_zn);
[y_ZN_P, ~] = step(setpoint*sys_cl_ZN_P, t_zn);
[y_ZN_PD, ~] = step(setpoint*sys_cl_ZN_PD, t_zn);
[y_Ku_resp, ~] = step(setpoint*sys_Ku, t_zn);

figure('Color','w','Name','Step Responses of Ziegler-Nichols method');
plot(t_zn, y_Ku_resp, 'r', 'LineWidth',1.5);

figure('Color','w','Name','Step Responses of Ziegler-Nichols method');
plot(t_zn, y_ZN_open, 'g', 'LineWidth',1.8); hold on;
plot(t_zn, y_ZN_noCl, 'b', 'LineWidth',1.8);
plot(t_zn, y_ZN_PID, 'r', 'LineWidth',2.2);

```

```

plot(t_zn, y_ZN_PI, 'c', 'LineWidth',1.8);
plot(t_zn, y_ZN_P, 'm', 'LineWidth',1.8);
plot(t_zn, y_ZN_PD, 'y', 'LineWidth',1.8);
yline(setpoint, 'k--', 'LineWidth',1.4);

hold off;
title(sprintf('Step Response of Ziegler-Nichols method (Td = %.1f ms, Setpoint =
%.5f rad/s)', ...
Td*1e3, setpoint), 'FontSize',13, 'FontWeight','bold');
xlabel('Time (s)', 'FontSize',13);
ylabel('\omega [rad/s]', 'FontSize',13);
legend('Open-loop (no feedback)', 'Closed-loop(No controller)', ...
'ZN-PID controller(Closed-loop)', 'ZN-PI controller(Closed-loop)', 'ZN-P
controller(Closed-loop)',...
'ZN-PD controller(Closed-loop)', 'Setpoint', 'Location','best', 'FontSize',10);
grid on;
set(gca, 'FontSize',12, 'LineWidth',1.2);

% --- Sensitivity analysis for ZN controllers ---
fprintf('\n=== SENSITIVITY ANALYSIS – ZN CONTROLLERS ===\n');
fprintf('%-8s %-10s %-10s %s\n', 'Ctrl', 'Ms (dB)', 'Mt (dB)', 'Robustness');
fprintf('%s\n', repmat('-',1,52));

ctrl_zn = {ZN_PID, ZN_PI, ZN_P, ZN_PD};
cname_zn = {'ZN-PID', 'ZN-PI', 'ZN-P', 'ZN-PD'};
for k = 1:numel(ctrl_zn)
    S_k = feedback(1, ctrl_zn{k}*G);
    T_k = feedback(ctrl_zn{k}*G, 1);
    Ms_k = 20*log10(getPeakGain(S_k));
    Mt_k = 20*log10(getPeakGain(T_k));
    if Ms_k < 6, rob = 'Good'; else, rob = 'Poor'; end
    fprintf('%-8s %-10.2f %-10.2f %s\n', cname_zn{k}, Ms_k, Mt_k, rob);
end

```

APPENDIX I

The MATLAB code for controllers with LQR methods

```
% LQR: Standard + Integral Augmented + Luenberger Observer

fprintf('\n===== \n');
fprintf(' SECTION 6 – LQR CONTROLLER \n');
fprintf('===== \n \n');

% -----
% --- 6a. Standard LQR (Bryson's Rule) ---
% -----
ia_max = 6; % allowable peak armature current [A]
omega_max = 314; % max speed (~3000 rpm) [rad/s]
u_max = 24; % supply voltage limit [V]

Q_lqr = diag([1/ia_max^2, 1/omega_max^2]);
R_lqr = 1/u_max^2;

[K_lqr, ~, cl_poles_lqr] = lqr(A, B, Q_lqr, R_lqr);

% Pre-scaling: drives omega to setpoint for constant r, NO disturbances.
% With load disturbances a steady-state error remains -- see LQR+I below.
Nbar = -1 / (C * ((A - B*K_lqr) \ B));

sys_LQR = ss(A - B*K_lqr, B*Nbar, C, 0);
sys_cl_noLQR = feedback(sys_ol, 1); % FIXED: was 'sys' (undefined)

fprintf('--- Standard LQR --- \n');
fprintf('Q = diag([1/ia_max^2, 1/omega_max^2]), R = 1/u_max^2 \n');
fprintf('LQR gain K = [%0.4f, %0.4f] \n', K_lqr(1), K_lqr(2));
fprintf('Pre-scale Nbar = %0.6f \n', Nbar);
fprintf('Closed-loop poles (A-BK): \n'); disp(eig(A-B*K_lqr).');

% -----
% --- 6b. Integral-Augmented LQR ---
% Augmented state: x_aug = [ia ; omega ; e_int]
% e_int_dot = setpoint - omega
% This eliminates steady-state error under constant load disturbances.
% -----
A_aug = [A, zeros(2,1);
         -C, 0];
B_aug = [B; 0];
```

```

C_aug = [C, 0]; % y = omega

Q_aug = blkdiag(Q_lqr, 500); % extra weight on integrator state
R_aug = R_lqr;

[K_aug, ~, cl_poles_aug] = lqr(A_aug, B_aug, Q_aug, R_aug);
K_x_aug = K_aug(1:2); % state feedback gains
K_i_aug = K_aug(3); % integral gain

% Closed-loop augmented system
A_cl_aug = A_aug - B_aug*K_aug;
B_r_aug = [zeros(2,1); 1]; % reference enters through integrator
sys_LQR_I = ss(A_cl_aug, B_r_aug, C_aug, 0);

fprintf('\n--- Integral-Augmented LQR ---\n');
fprintf('K_state = [%0.4f, %0.4f], K_integral = %0.4f\n', ...
    K_x_aug(1), K_x_aug(2), K_i_aug);
fprintf('Augmented CL poles:\n'); disp(cl_poles_aug.);

% -----
% --- 6c. Luenberger Observer ---
% Only omega (speed) is measured; ia (current) must be estimated.
% Observer poles placed obs_speed x faster than LQR poles.
% -----
lqr_pole_mag = max(abs(cl_poles_lqr));
obs_speed = 8; % observer ~8x faster than controller poles
observer_poles = [-obs_speed * lqr_pole_mag * 1.0;
    -obs_speed * lqr_pole_mag * 1.2]; % distinct to avoid ill-conditioning

L_obs = place(A', C', observer_poles);

fprintf('\n--- Luenberger Observer ---\n');
fprintf('Observer desired poles: [%0.2f, %0.2f]\n', ...
    observer_poles(1), observer_poles(2));
fprintf('Observer gain L = [%0.4f; %0.4f]\n', L_obs(1), L_obs(2));

obs_eigs = eig(A - L_obs*C);
fprintf('Observer eigenvalues:\n'); disp(obs_eigs.);
if all(real(obs_eigs) < 0)
    disp('Observer error dynamics: STABLE');
else
    disp('Observer error dynamics: UNSTABLE -- recheck pole placement.');
```

```

% Observer state-space (inputs: [u ; y_measured], outputs: x_hat)
%  $\dot{x}_{\hat{}} = (A - L \cdot C) \cdot x_{\hat{}} + B \cdot u + L \cdot y$ 
A_obs = A - L_obs * C;
B_obs = [B, L_obs]; % columns: [control input; measured output]
C_obs = eye(2); % both states estimated
D_obs = zeros(2, 2);
sys_observer = ss(A_obs, B_obs, C_obs, D_obs);

% --- 6d. Step Response Comparison ---
t_lqr = linspace(0, Tend, 2000);

[y_ol_lqr, ~] = step((setpoint/G_dc)*sys_ol, t_lqr);
[y_noLQR, ~] = step(setpoint*sys_cl_noLQR, t_lqr);
[y_LQR, ~] = step(setpoint*sys_LQR, t_lqr);
[y_LQR_I, ~] = step(setpoint*sys_LQR_I, t_lqr);

y_ol_lqr = squeeze(y_ol_lqr);
y_noLQR = squeeze(y_noLQR);
y_LQR = squeeze(y_LQR);
y_LQR_I = squeeze(y_LQR_I);

% Steady-state values
ss_LQR = dcgain(setpoint * sys_LQR);
ss_LQR_I = dcgain(setpoint * sys_LQR_I);

fprintf('\nLQR steady-state tracking:\n');
fprintf(' Standard LQR (Nbar scaling): SS = %.4f error = %.4f (0.2f%%)\n', ...
    ss_LQR, setpoint-ss_LQR, 100*(setpoint-ss_LQR)/setpoint);
fprintf(' Integral LQR : SS = %.4f error = %.4f (0.2f%%)\n', ...
    ss_LQR_I, setpoint-ss_LQR_I, 100*(setpoint-ss_LQR_I)/setpoint);

figure('Color','w','Name','Step Responses - LQR');
plot(t_lqr, y_ol_lqr, 'g', 'LineWidth',1.8); hold on;
plot(t_lqr, y_noLQR, 'b', 'LineWidth',1.8);
plot(t_lqr, y_LQR, 'r', 'LineWidth',2.2);
plot(t_lqr, y_LQR_I, 'm', 'LineWidth',2.2);
yline(setpoint, 'k--', 'LineWidth',1.4);

hold off;
title('Step Responses of LQR method (Standard vs Integral-Augmented)', ...
    'FontSize',14, 'FontWeight','bold');
xlabel('Time (s)', 'FontSize',13);
ylabel('\omega [rad/s]', 'FontSize',13);
legend('Open-loop (no feedback)', 'Closed-loop(No controller)', ...

```

```

'LQR (Nbar scaling)', 'LQR + Integral', 'Setpoint', ...
'Location', 'best', 'FontSize', 11);
grid on;
set(gca, 'FontSize', 12, 'LineWidth', 1.2);

% --- 6e. Sensitivity Analysis – LQR ---
fprintf('\n=== SENSITIVITY ANALYSIS – LQR ===\n');

% Standard LQR: convert to unity-feedback equivalent loop TF
L_lqr = K_lqr * (s*eye(2) - (A - B*K_lqr))^( -1) * B;
% Use input-output TF from voltage to speed directly
S_LQR = feedback(1, sys_LQR);
T_LQR = feedback(sys_LQR, 1);
Ms_LQR = 20*log10(getPeakGain(S_LQR));
Mt_LQR = 20*log10(getPeakGain(T_LQR));
fprintf('Standard LQR: Ms = %.2f dB, Mt = %.2f dB', Ms_LQR, Mt_LQR);
if Ms_LQR < 6, fprintf(' --> Good\n'); else, fprintf(' --> Poor\n'); end

S_LQR_I = feedback(1, sys_LQR_I);
T_LQR_I = feedback(sys_LQR_I, 1);
Ms_LQR_I = 20*log10(getPeakGain(S_LQR_I));
Mt_LQR_I = 20*log10(getPeakGain(T_LQR_I));
fprintf('Integral LQR: Ms = %.2f dB, Mt = %.2f dB', Ms_LQR_I, Mt_LQR_I);
if Ms_LQR_I < 6, fprintf(' --> Good\n'); else, fprintf(' --> Poor\n'); end

% Controller Comparison Summary Table
fprintf('\n===== \n');
fprintf(' SECTION 7 – CONTROLLER COMPARISON SUMMARY \n');
fprintf('===== \n \n');

all_systems = {sys_cl, sys_cl_PID, sys_cl_PI, sys_cl_P, ...
               sys_cl_PD, sys_cl_I, ...
               sys_cl_ZN_PID, sys_cl_ZN_PI, sys_cl_ZN_P, sys_cl_ZN_PD, ...
               sys_LQR, sys_LQR_I};

all_names = {'No Ctrl', 'Auto-PID', 'Auto-PI', 'Auto-P', 'Auto-PD', 'Auto-I', ...
             'ZN-PID', 'ZN-PI', 'ZN-P', 'ZN-PD', ...
             'LQR', 'LQR+I'};

fprintf('%-12s %8s %8s %8s %8s %8s %8s \n', ...
        'Controller', 'Rise(s)', 'Settle(s)', 'OS(%)', 'SS_val', 'SS_err%', 'Ms(dB)');
fprintf('%s \n', repmat('-', 1, 76));

for k = 1:numel(all_systems)

```

```

sys_k = all_systems{k};
name_k = all_names{k};

try
    inf_k = stepinfo(setpoint * sys_k);
    rt_k = inf_k.RiseTime;
    st_k = inf_k.SettlingTime;
    os_k = inf_k.Overshoot;
    ss_k = dcgain(setpoint * sys_k);
    err_k = 100*(setpoint - ss_k)/setpoint;
    S_k = feedback(1, sys_k);
    Ms_k = 20*log10(getPeakGain(S_k));

    fprintf('%-12s %8.4f %8.4f %8.2f %8.4f %8.2f %8.2f\n', ...
        name_k, rt_k, st_k, os_k, ss_k, err_k, Ms_k);
catch
    fprintf('%-12s (could not compute)\n', name_k);
end
end

```

APPENDIX J

Pseudo code for Pick and Place KUKA manipulator

```
FUNCTION kuka_pick_and_place_gui_magnetic()

// ----- UI и состояние -----
create UI figure
S = struct with:
    robot_model, max_reach, min_reach
    z_table, z_clearance
    home, pick, place poses
    connection settings (ip, port, tcp, connected)
    robot_s, robot_t
    key_poses, waypoints
    running flags
    magnetic flags + delays + status

build left panel with controls:
    robot model dropdown
    environment inputs
    motion inputs (points per segment, dt, trigger pulse)
    magnetic control inputs + test button
    connection inputs + connect/disconnect buttons
    pose fields for home/pick/place
    preview/execute/stop buttons

build center 3D preview axes
build log panel
build right telemetry axes (torque, cartesian, velocity, current)

refreshPreview()
log ready messages

// ----- callbacks -----
FUNCTION onModelChanged()
    update S.max_reach / S.robot_model
    refreshPreview()
    log model change
END

FUNCTION onMagneticEnableChanged()
    toggle magnetic controls
    log enable/disable
```

END

```
FUNCTION onConnect()
  read ip/port
  connect tcp client
  update UI states
  setMagnetic(FALSE) // safety
  read S/T from robot
  log connected
END
```

```
FUNCTION doDisconnect()
  setMagnetic(FALSE) // safety
  close tcp
  update UI states
  log disconnected
END
```

```
FUNCTION doTestMagnetic()
  if not connected -> alert
  setMagnetic(TRUE), wait, setMagnetic(FALSE)
  log test complete
END
```

```
FUNCTION doPreview()
  readUIToState()
  buildTrajectory()
  drawTrajectory()
  log preview stats
END
```

```
FUNCTION doExecute()
  if not connected -> alert
  readUIToState()
  buildTrajectory()
  drawTrajectory()
  confirm dialog with safety checklist
  if confirmed -> runStreaming()
END
```

APPENDIX K

Pseudo code for Painting process RoArm M2-S manipulators

```
FUNCTION roarms_painting_process_updated()
    clear console, close figures
    cfg = buildConfig()
    buildMainGUI(cfg)
END

// ----- CONFIG -----
FUNCTION buildConfig()
    define geometry of table, conveyor, object
    define robot geometry and reach limits
    define station position (single station in center)
    define painting params (standoff, stripe height, approach, speed)
    define HTTP comms settings (IPs, timeouts, retries)
    define motion gripper angle offset
    define settle criteria (tolerance, timeout, poll)
    return cfg
END

// ----- GUI -----
FUNCTION buildMainGUI(cfg)
    create UI figure and top bar
    create ARM1 panel (inputs + Move button + labels)
    create ARM2 panel (inputs + Move button + labels)
    create settings panel (stripe, standoff, speed, IPs, buttons)
    create 3D preview panel and draw workspace
    create log panel
    create telemetry plots (positions/torques for both arms)
    init state (stopFlag, step counters)
    wire callbacks (Move/Ping/Home/Telemetry/Start/Stop)
END

FUNCTION makeArmPanel(...)
    build input fields X/Y/Z and Move button
    build labels for IK status and telemetry
    return handles
END

FUNCTION makeConfigPanel(...)
    build controls for stripe/standoff/speed/IPs
    build buttons: Ping, Home, Telemetry, Start Painting, Stop
```

```
    return handles
END
```

```
FUNCTION rewireConfigButtons(fig, cfg)
    set button callbacks to cbPing/cbHome/cbTel/cbStart/cbStop
END
```

```
// ----- COORDINATES & REACH -----
FUNCTION globalToLocal(gx, gy, gz, armIdx, cfg)
    if arm1: local = global - base1
    if arm2: mirror about base2
    return lx, ly, lz
END
```

```
FUNCTION checkReach(lx, ly, lz, cfg)
    compute r, d3, z-range
    enforce constraints:
        not behind base, within reach, not too close, Z range ok
    return ok, message
END
```

```
FUNCTION computeJointAngles(lx, ly, lz, cfg)
    solve planar 2-link IK (theta1, theta2, theta3)
    return angles
END
```

```
FUNCTION computeStaticTorques(theta2, theta3, cfg)
    estimate tau1, tau2, tau3 with simple mass model
    return torques
END
```

```
// ----- HTTP COMM -----
FUNCTION httpSend(ip, baseUrl, jsonCmd, timeout, maxRet, retDly)
    for k in 1..maxRet:
        try webread(url)
        if ok -> return response, success
        else wait retDly
    return response empty, success false
END
```

```
FUNCTION httpReadStatus(ip, timeout)
    send T=105 status request
    decode JSON
    return data struct or empty
```

END

FUNCTION parseTel(data)

 extract X/Y/Z from data

 return x,y,z

END

FUNCTION validateConnection(ip1, ip2, timeout)

 read status from each arm

 return connected + per-arm flags

END

FUNCTION stopRequested(fig)

 return fig.UserData.stopFlag

END

FUNCTION waitForArrival(ip, targetLocal, cfg, fig)

 loop until timeout:

 if stop -> return false

 read status

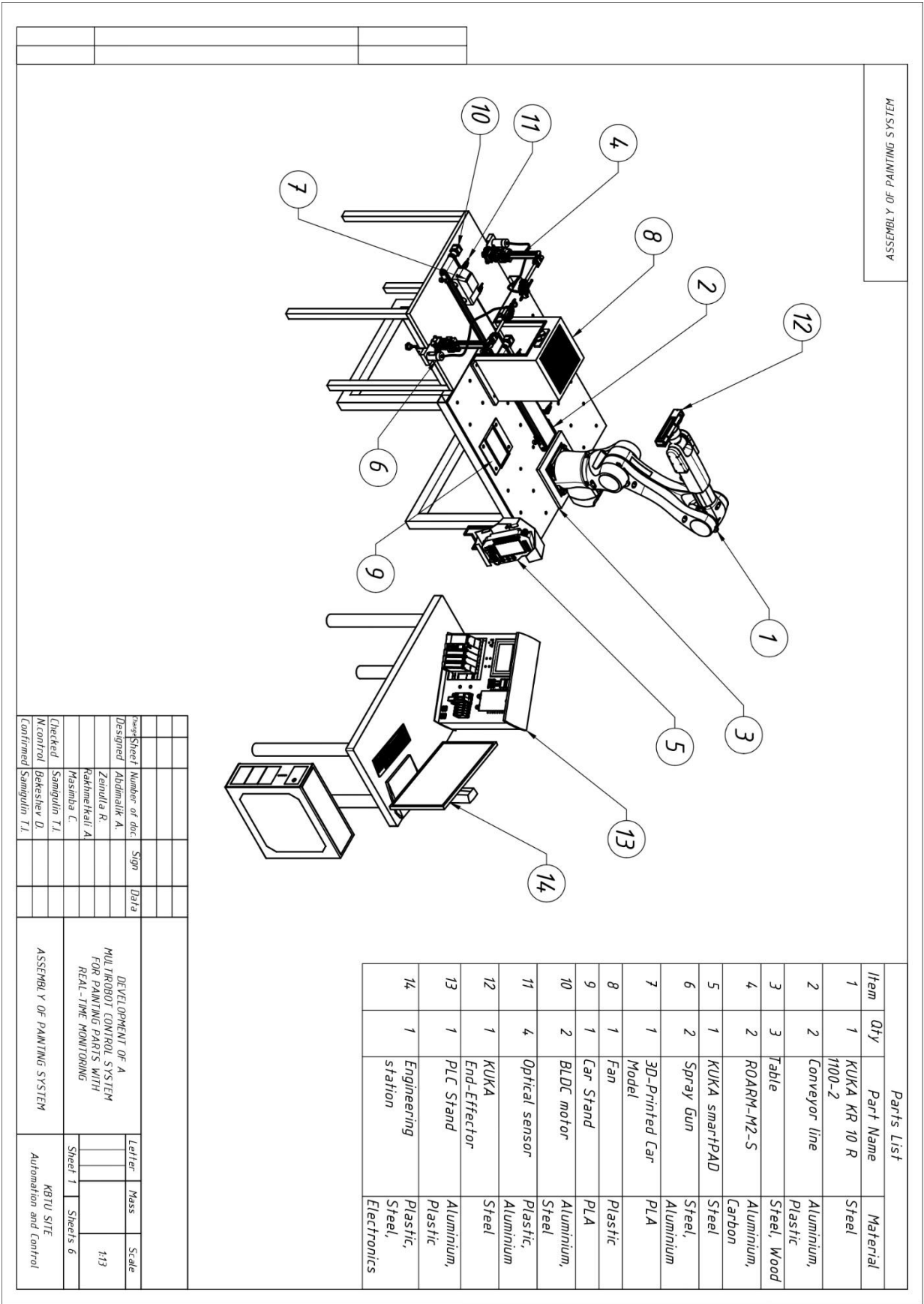
 if distance <= tolerance -> return true

 return false

END

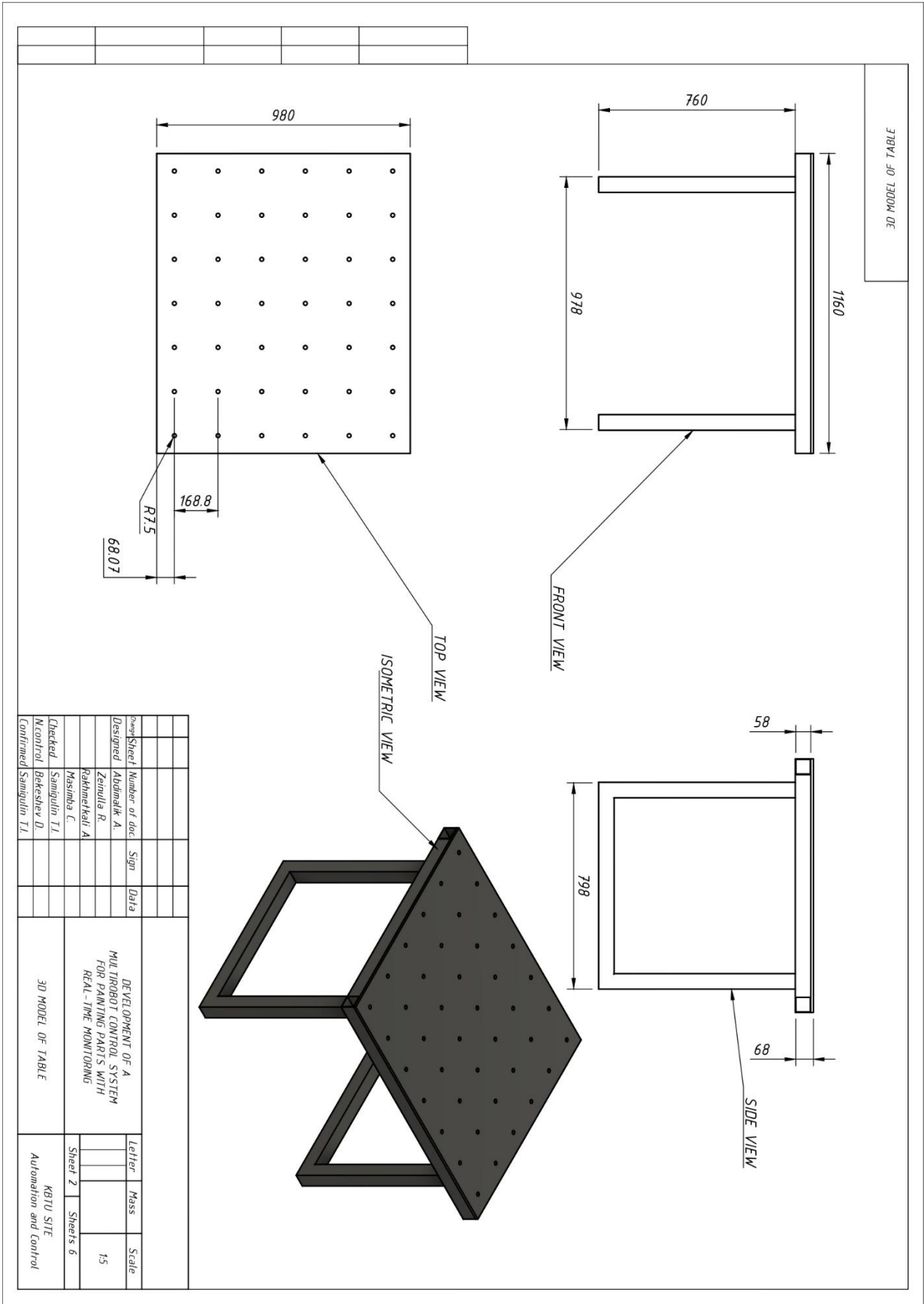
APPENDIX L

Assembly of painting system



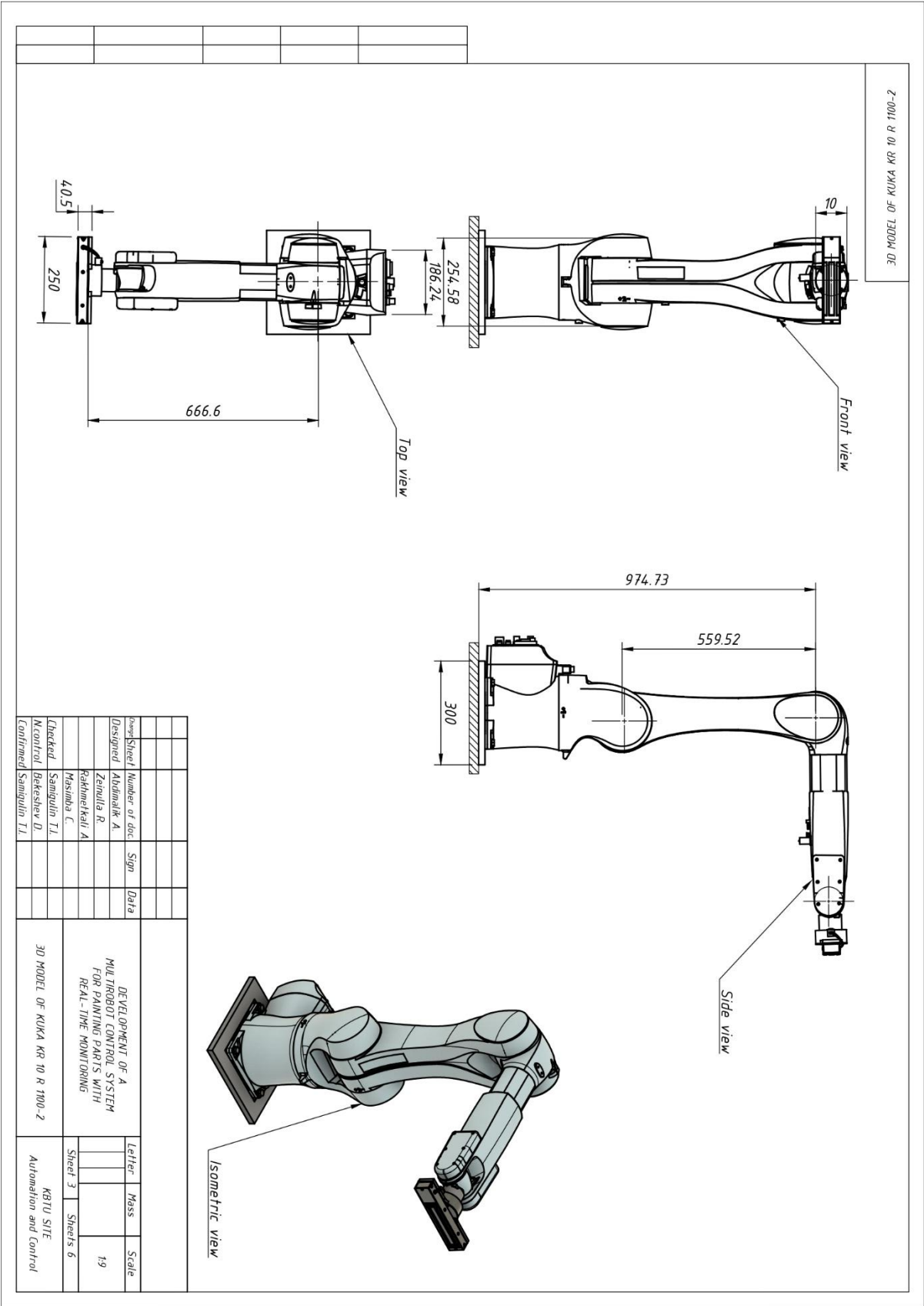
APPENDIX M

3D model of the table



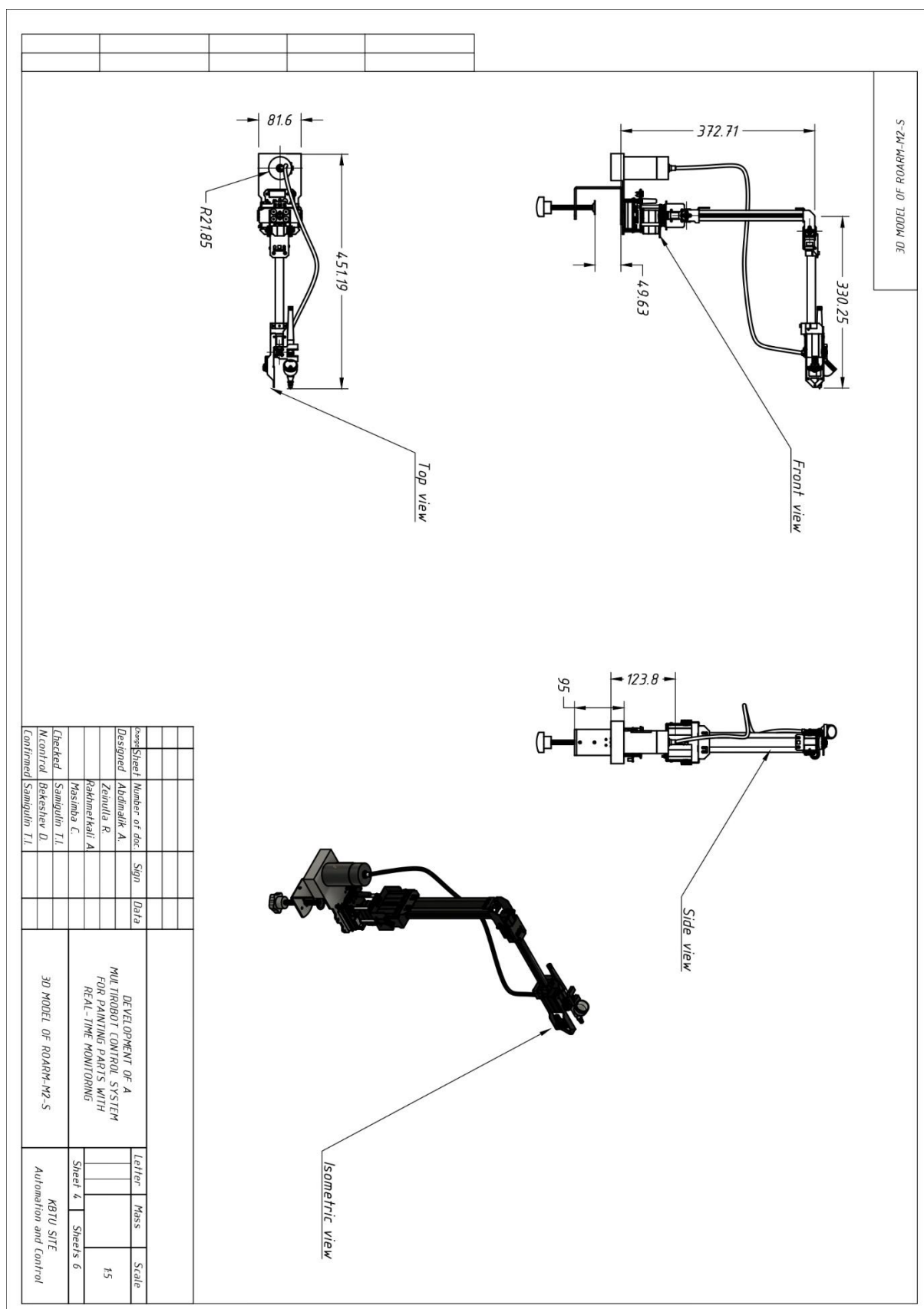
APPENDIX N

3D model of KUKA KR 10 R 1100-2



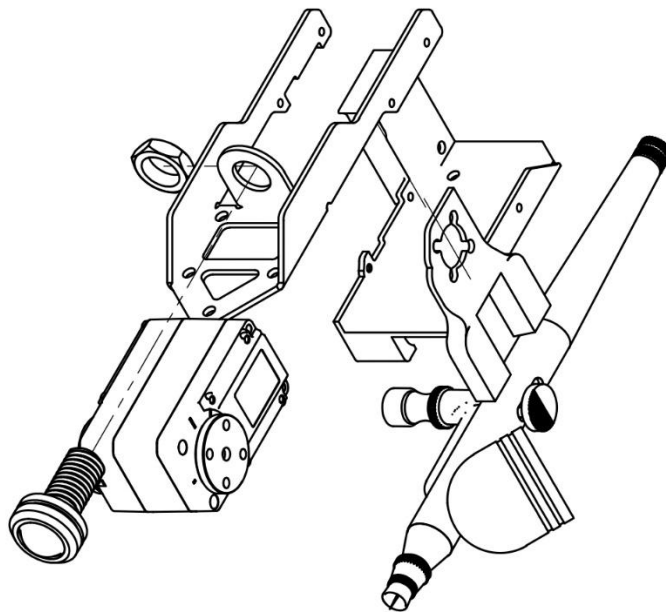
APPENDIX O

3D model of RoArm-M2-S



APPENDIX P

Exploded view of RoArm-M2-S End Effector



						DEVELOPMENT OF A MULTIROBOT CONTROL SYSTEM FOR PAINTING PARTS WITH REAL-TIME MONITORING			
Design	Sheet	Number of doc	Sign	Date					
		Abdulk A.							
		Zenilla R.							
		Kashimekail A							
		Mosina J.							
Checked		Samquin T.I.							
Mccontrol		Bekesley D.							
confirmed		Samquin T.I.							

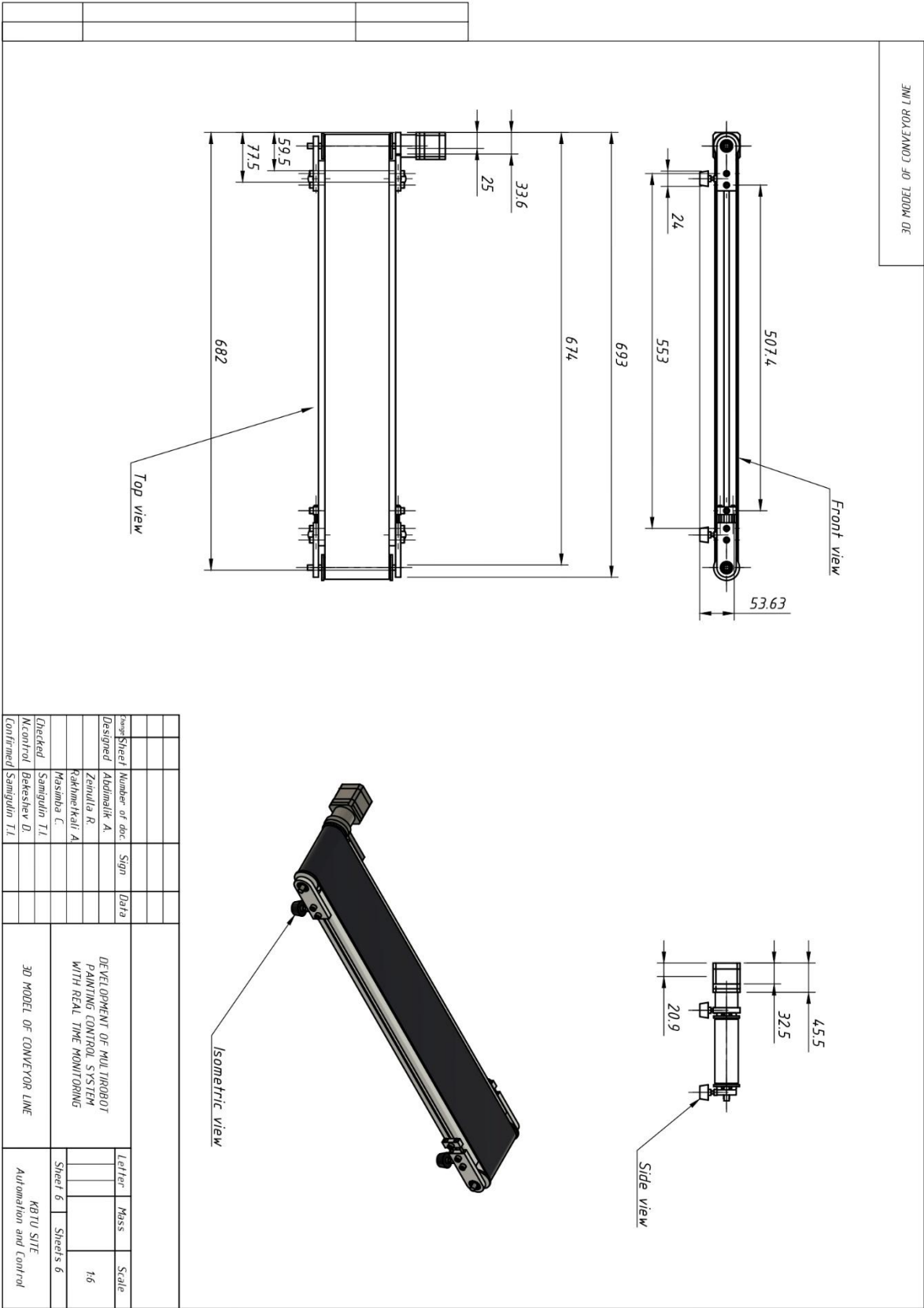
Letter	Mass	Scale
		1:1
Sheet 5		Sheet 6

KBU SITE Automation and Control

EXPLODED VIEW OF
ROARM-M2-S END EFFECTOR

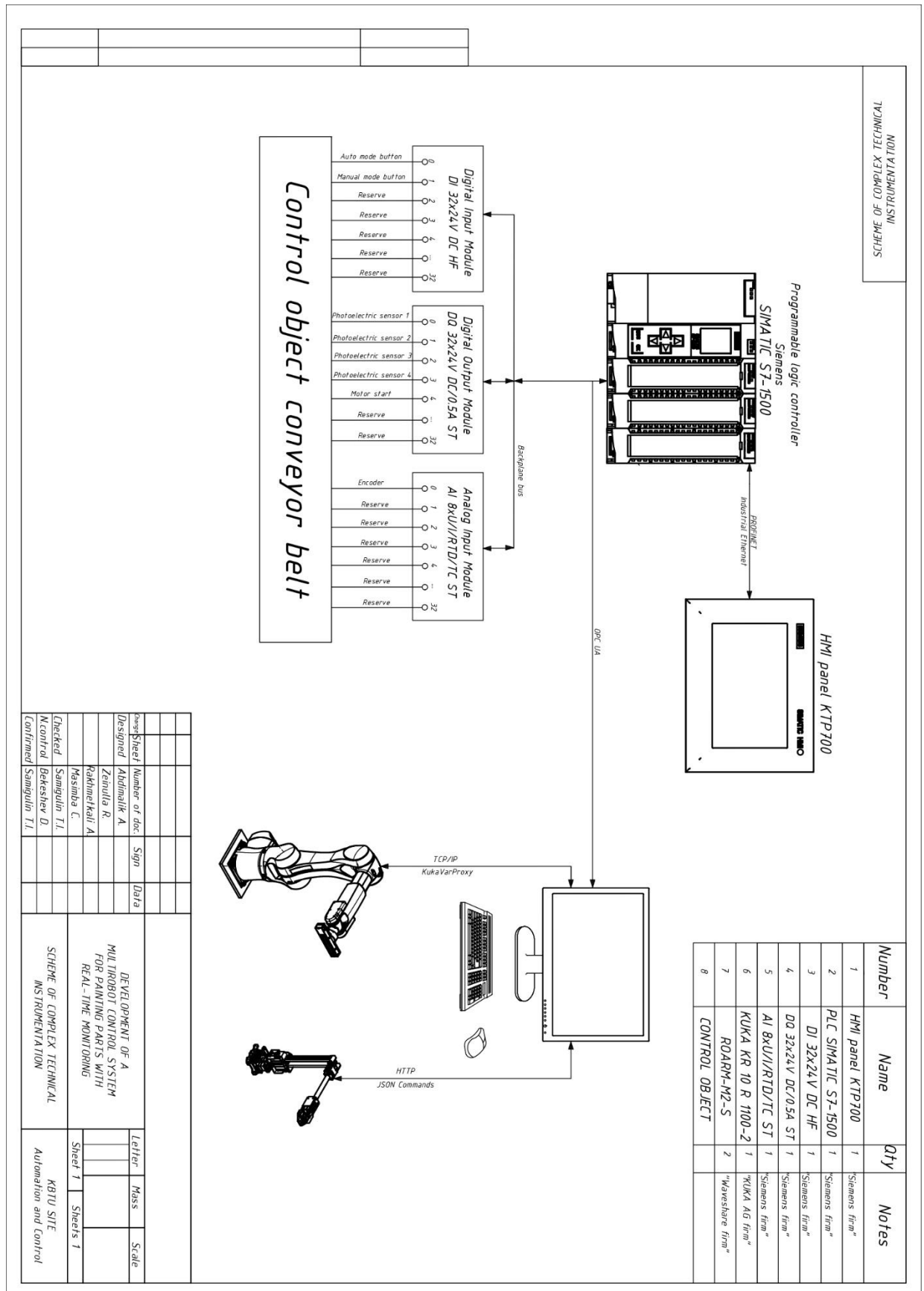
APPENDIX Q

3D model of conveyor line



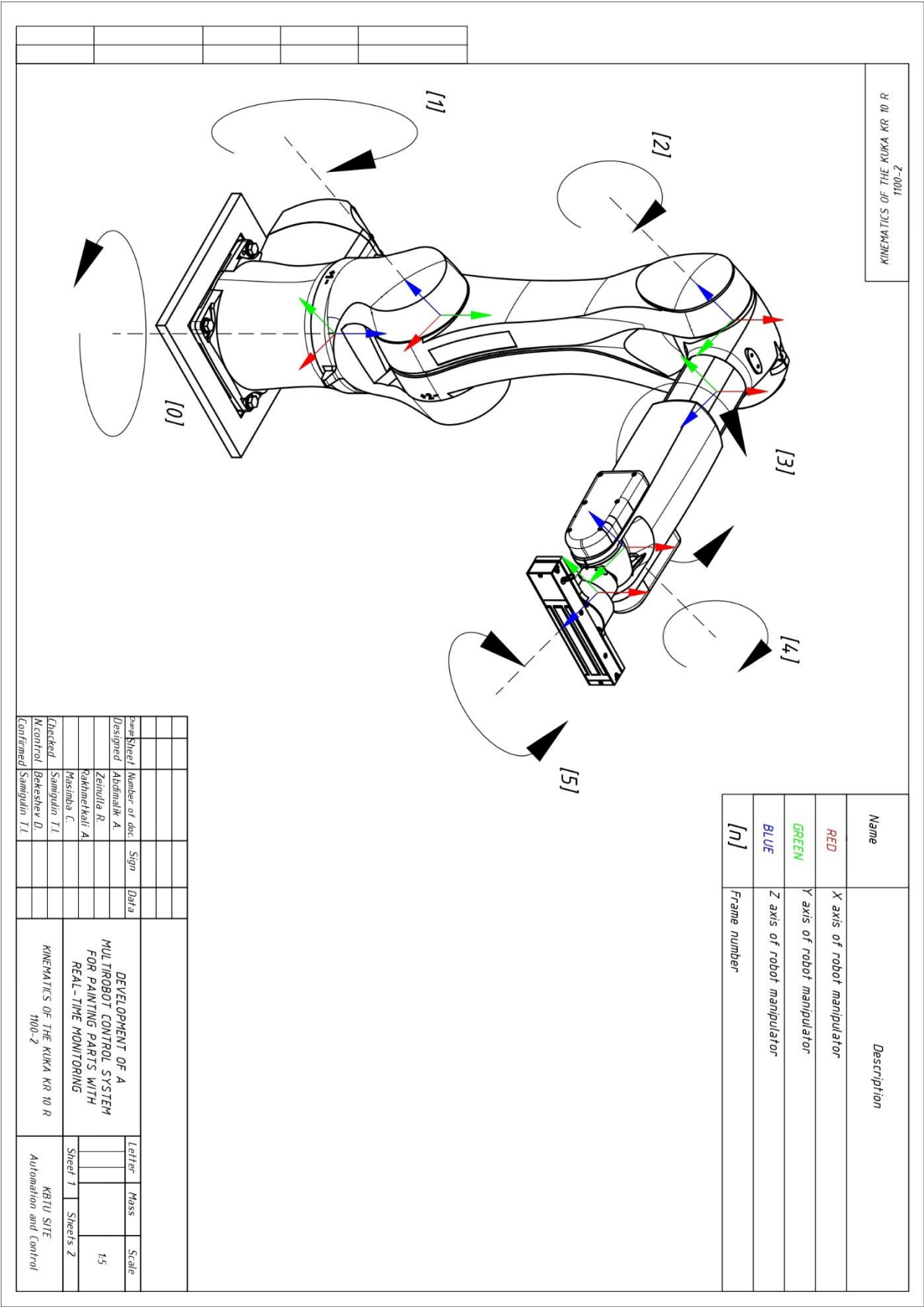
APPENDIX R

Scheme of complex technical instrumentation



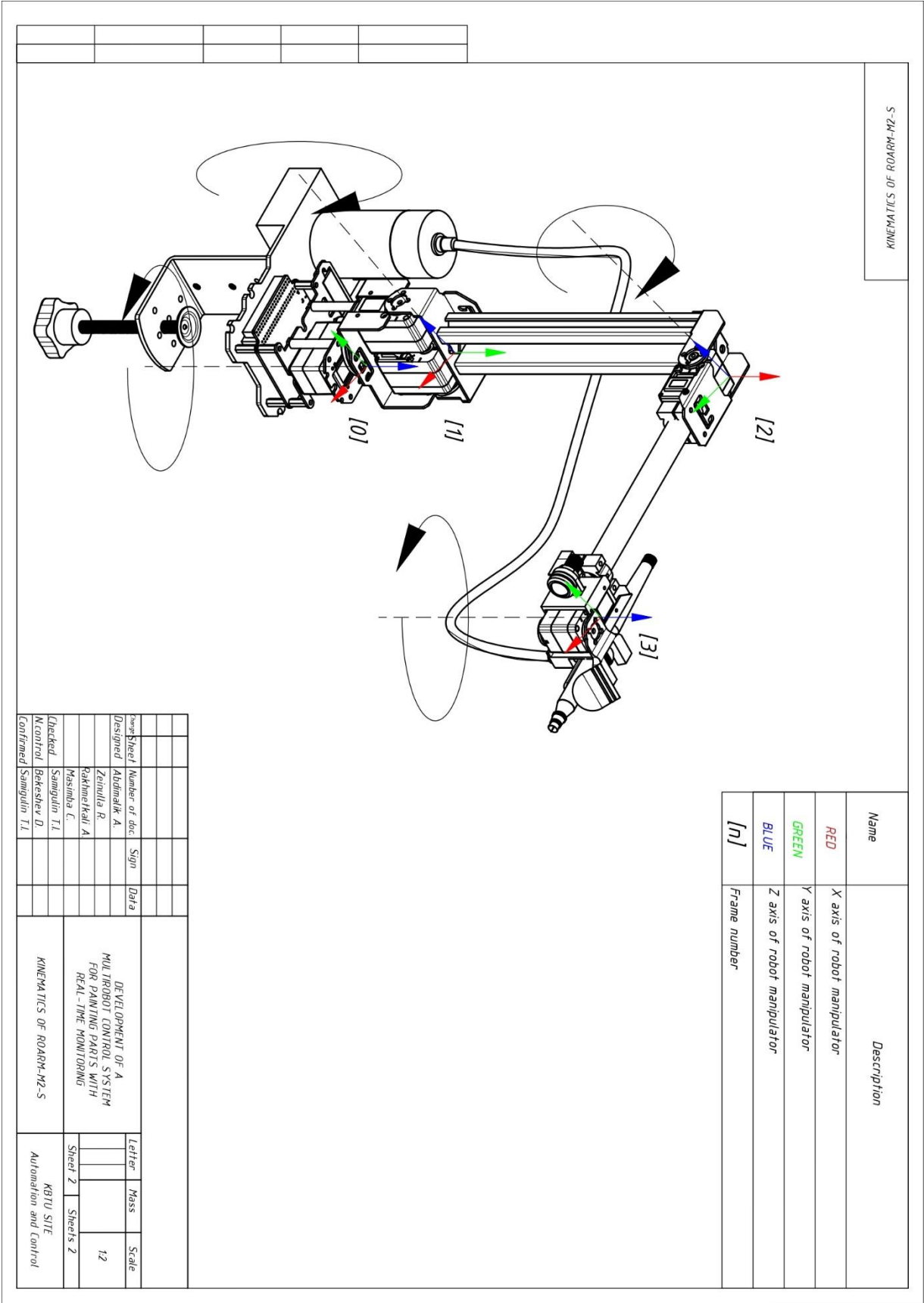
APPENDIX S

Kinematics of the KUKA KR 10 R 1100-2



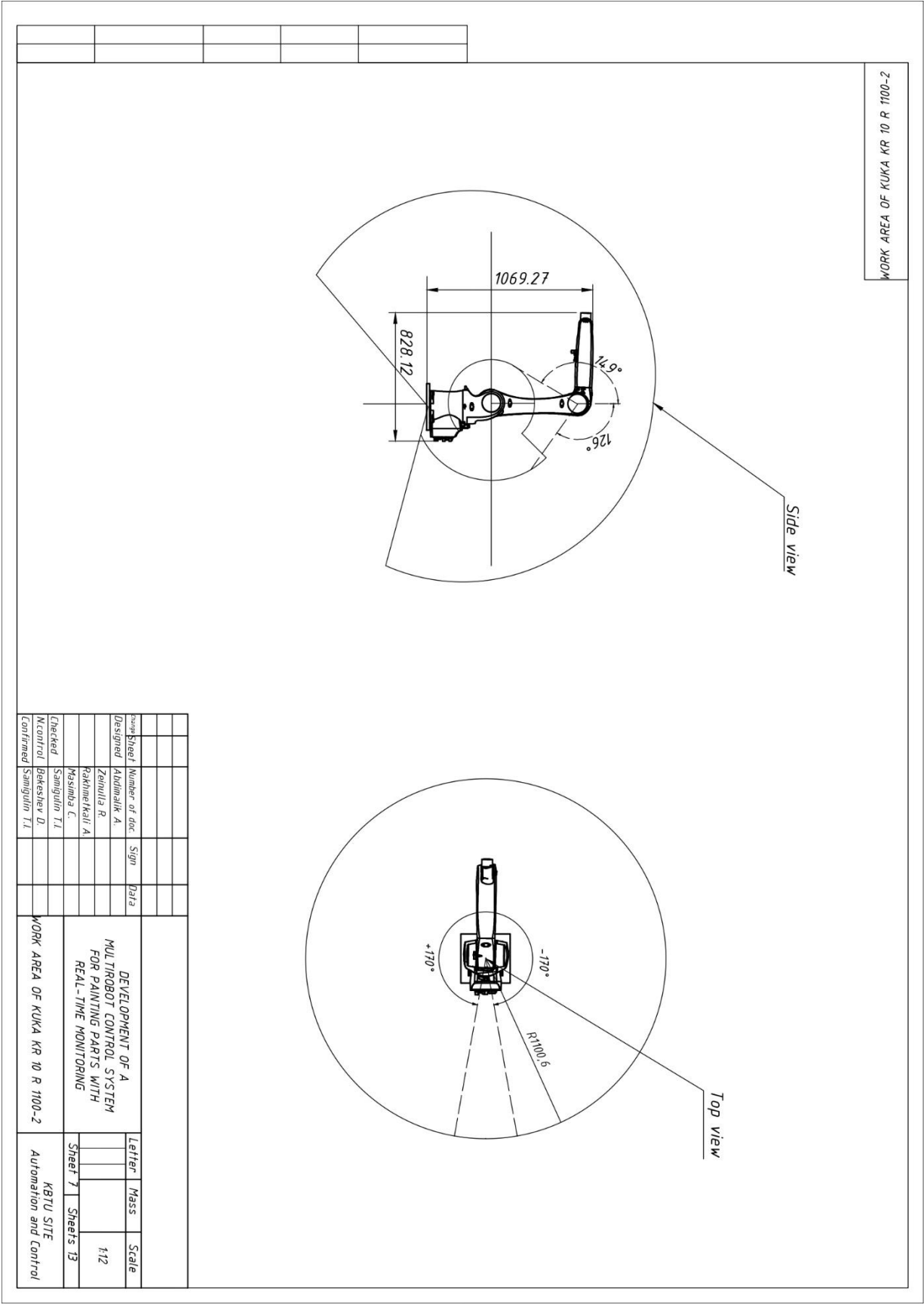
APPENDIX T

Kinematics of the RoArm-M2-S



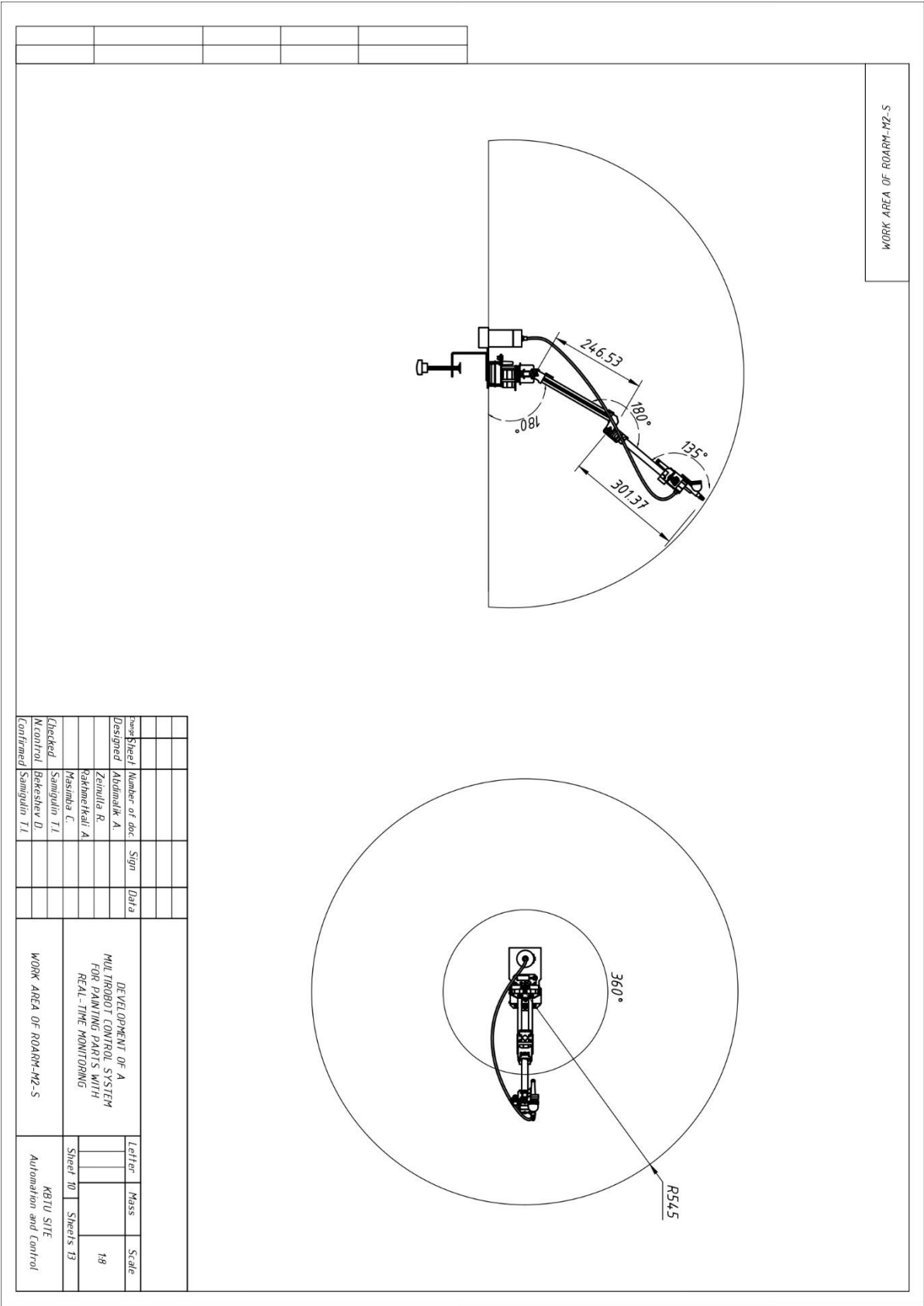
APPENDIX U

Work area of KUKA KR 10 R 1100-2



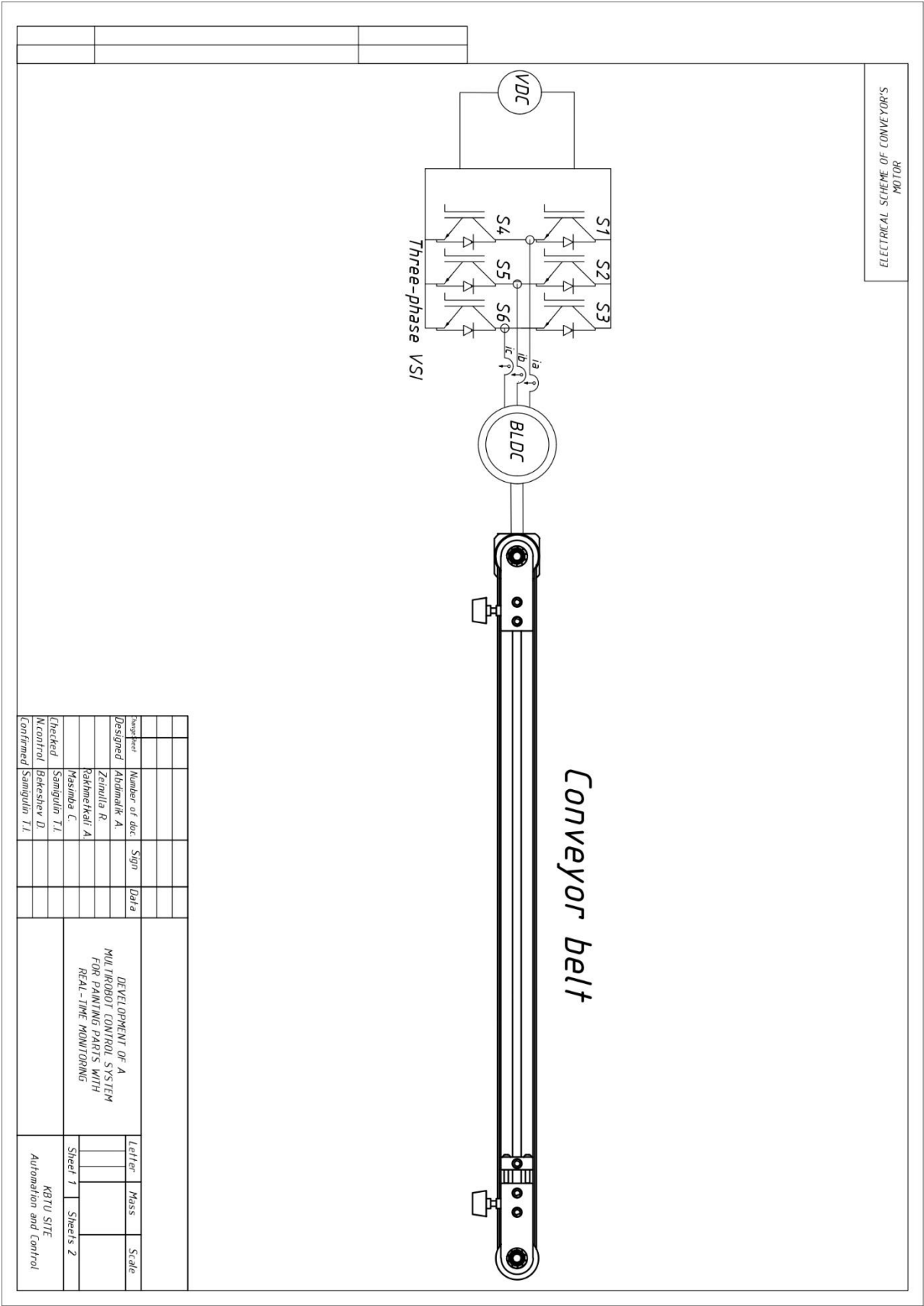
APPENDIX V

Work area of RoArm-M2-S



APPENDIX W

Electrical scheme of Conveyor's motor



APPENDIX X

Electrical scheme of RoArm-M2-S

